

# XCPC Template Library

Morning Flower  
and Evening Oath

Tongji University

vector 算法模板与数学速查

建设稿 · 2026-09-12

## 状态说明

本稿包含首批已进行本地验证的模块与完整来源覆盖表的入口。尚未完成 kuangbin、WIDA 全部条目的迁移，尚未完成近三年中国赛站逐题覆盖审计，最大流、平衡树、双连通分量、可持久化等部分接口已有在线 AC 记录，具体题号、源码摘要及验证范围见 `verification/oj.json`。其余接口须分别核验，不能由同文件中某算法通过推断全部通过。本地对拍通过不代表对所有输入的绝对正确性证明；适用范围、数值上限与退化约定属于接口的一部分。

# 目录

|                             |           |
|-----------------------------|-----------|
| 使用与验证                       | ix        |
| <b>第一部分 vector 模板</b>       | <b>1</b>  |
| <b>第一章 Tarjan 与缩点</b>       | <b>2</b>  |
| 1.1 Tarjan 强连通分量与缩点         | 2         |
| <b>第二章 函数图</b>              | <b>4</b>  |
| 2.1 函数图分解、跳转与到达             | 4         |
| <b>第三章 双连通分量与圆方树</b>        | <b>7</b>  |
| 3.1 Tarjan 点双、边双与圆方树        | 7         |
| <b>第四章 奇环顶点判定</b>           | <b>10</b> |
| 4.1 属于简单奇环的全部顶点             | 10        |
| <b>第五章 删点连通性</b>            | <b>12</b> |
| 5.1 由点双分解计算删点连通块数           | 12        |
| <b>第六章 边双缩点与边定向</b>         | <b>13</b> |
| 6.1 边双缩点的大小与内部边数            | 13        |
| 6.2 无向边定向与强连通定向             | 14        |
| <b>第七章 桥树最少加边</b>           | <b>15</b> |
| 7.1 连通无向图最少加边方案             | 15        |
| <b>第八章 数据结构</b>             | <b>16</b> |
| 8.1 并查集                     | 16        |
| 8.2 可撤销并查集                  | 17        |
| 8.3 树状数组与第 k 小              | 18        |
| 8.4 区间加与区间和                 | 19        |
| 8.5 64 位异或线性基               | 20        |
| <b>第九章 带修改的莫队</b>           | <b>23</b> |
| 9.1 带修改的莫队                  | 23        |
| <b>第十章 带位置线性基</b>           | <b>25</b> |
| 10.1 带位置限制的异或线性基            | 25        |
| <b>第十一章 线性空间求交</b>          | <b>27</b> |
| 11.1 异或线性空间求交               | 27        |
| 11.2 Zassenhaus: 同时求和空间与交空间 | 28        |
| <b>第十二章 线段树常用修改</b>         | <b>29</b> |
| 12.1 线段树: 区间乘、加、赋值与模意义求和    | 29        |

|                               |           |
|-------------------------------|-----------|
| <b>第十三章 网络流</b>               | <b>31</b> |
| 13.1 最大流与方案                   | 31        |
| 13.2 势能最小费用最大流                | 33        |
| 13.3 上下界可行循环流                 | 35        |
| <b>第十四章 最大权闭合子图</b>           | <b>36</b> |
| 14.1 最大权闭合子图与选点方案             | 36        |
| <b>第十五章 图论</b>                | <b>37</b> |
| 15.1 非负权最短路                   | 37        |
| 15.2 强连通分量                    | 39        |
| 15.3 2-SAT 与任意可行方案            | 40        |
| 15.4 Hopcroft-Karp 最大匹配与最小点覆盖 | 40        |
| 15.5 割点与桥                     | 43        |
| <b>第十六章 开放时间最短路</b>           | <b>45</b> |
| 16.1 顶点开放时间的单位边最短路            | 46        |
| <b>第十七章 字典序最小 2-SAT</b>       | <b>47</b> |
| 17.1 递归染色与回滚求最小赋值             | 47        |
| <b>第十八章 有向欧拉路</b>             | <b>49</b> |
| 18.1 有向图字典序最小欧拉路              | 49        |
| <b>第十九章 字典序单词链</b>            | <b>51</b> |
| 19.1 字典序最小单词链                 | 51        |
| <b>第二十章 无向欧拉路</b>             | <b>52</b> |
| 20.1 无向图欧拉路与边方向               | 52        |
| <b>第二十一章 混合图欧拉定向</b>          | <b>54</b> |
| 21.1 欧拉回路与指定端点定向              | 54        |
| 21.2 任意起终点的混合欧拉路径             | 56        |
| <b>第二十二章 图上异或行走</b>           | <b>57</b> |
| 22.1 无向图最大异或行走                | 57        |
| <b>第二十三章 树上算法</b>             | <b>59</b> |
| 23.1 重链剖分与 LCA                | 59        |
| <b>第二十四章 离线最近公共祖先</b>         | <b>61</b> |
| 24.1 离线 Tarjan LCA            | 61        |
| <b>第二十五章 在线最近公共祖先</b>         | <b>63</b> |
| 25.1 欧拉序与 ST 表 LCA            | 63        |
| <b>第二十六章 树上点集摘要</b>           | <b>66</b> |
| 26.1 树上点集直径摘要                 | 66        |
| <b>第二十七章 倍增祖先与路径最值</b>        | <b>68</b> |
| 27.1 倍增 LCA、祖先与最大边权           | 68        |

|                          |            |
|--------------------------|------------|
| <b>第二十八章 树上路径交</b>       | <b>71</b>  |
| 28.1 两条树路径的交集            | 71         |
| <b>第二十九章 点分治</b>         | <b>72</b>  |
| 29.1 点分治：树上距离点对计数        | 72         |
| <b>第三十章 树上启发式合并</b>      | <b>75</b>  |
| 30.1 树上启发式合并：子树众数颜色和     | 75         |
| <b>第三十一章 虚树</b>          | <b>77</b>  |
| 31.1 虚树：关键点压缩与路径信息       | 77         |
| <b>第三十二章 字符串</b>         | <b>80</b>  |
| 32.1 前缀函数                | 80         |
| 32.2 KMP 匹配              | 80         |
| 32.3 Z 函数                | 81         |
| 32.4 Manacher 回文半径       | 81         |
| 32.5 最小循环表示              | 82         |
| 32.6 AC 自动机出现次数          | 82         |
| 32.7 倍增后缀数组与 LCP         | 85         |
| 32.8 后缀自动机               | 87         |
| <b>第三十三章 后缀查询与重复子串</b>   | <b>89</b>  |
| 33.1 后缀 LCP 与子串比较        | 89         |
| 33.2 两个前缀的最长公共后缀         | 90         |
| 33.3 重复子串 AA 的起止位置计数     | 91         |
| <b>第三十四章 回文树</b>         | <b>92</b>  |
| 34.1 回文树 PAM：回文后缀与出现次数   | 92         |
| <b>第三十五章 数论</b>          | <b>94</b>  |
| 35.1 模运算、素性、CRT 与类欧几里德   | 94         |
| 35.2 Pollard-Rho 因数分解    | 97         |
| 35.3 线性筛、欧拉函数与 Möbius 函数 | 98         |
| 35.4 静态模整数               | 99         |
| 35.5 阶乘组合数               | 100        |
| <b>第三十六章 二元一次不定方程</b>    | <b>101</b> |
| 36.1 二元一次不定方程            | 101        |
| <b>第三十七章 线性同余方程</b>      | <b>102</b> |
| 37.1 线性同余方程              | 102        |
| <b>第三十八章 区间筛素数</b>       | <b>103</b> |
| 38.1 区间筛素数               | 103        |
| <b>第三十九章 批量模逆元</b>       | <b>104</b> |
| 39.1 任意数组的批量逆元           | 104        |
| <b>第四十章 连续整数逆元表</b>      | <b>105</b> |
| 40.1 连续整数的线性逆元表          | 105        |

|                              |            |
|------------------------------|------------|
| <b>第四十一章 混合进制中国剩余定理</b>      | <b>106</b> |
| 41.1 Garner: 大周期 CRT 与混合进制   | 106        |
| <b>第四十二章 乘法阶与原根</b>          | <b>107</b> |
| 42.1 乘法阶与原根                  | 107        |
| <b>第四十三章 矩形 GCD 计数</b>       | <b>110</b> |
| 43.1 矩形 GCD 计数               | 110        |
| <b>第四十四章 带权类欧几里德</b>         | <b>111</b> |
| 44.1 整除和、带权和与平方和             | 111        |
| <b>第四十五章 模几何级数与约数和</b>       | <b>112</b> |
| 45.1 模幂与几何级数同时求值             | 112        |
| 45.2 幂的约数和                   | 113        |
| <b>第四十六章 单个数的欧拉函数</b>        | <b>114</b> |
| 46.1 单个数的欧拉函数                | 114        |
| <b>第四十七章 Carmichael 函数</b>   | <b>115</b> |
| 47.1 Carmichael: 模乘法群的公共指数   | 115        |
| <b>第四十八章 整数分拆</b>            | <b>116</b> |
| 48.1 整数分拆与重数上限               | 116        |
| <b>第四十九章 素数模数组组合数</b>        | <b>117</b> |
| 49.1 Lucas: 小素数模的大下标组合数      | 117        |
| <b>第五十章 合数模数组组合数</b>         | <b>119</b> |
| 50.1 合数模数组组合数 (扩展 Lucas)     | 119        |
| <b>第五十一章 二次剩余</b>            | <b>121</b> |
| 51.1 素数模平方根 (Cipolla)        | 121        |
| <b>第五十二章 素数模高次剩余</b>         | <b>123</b> |
| 52.1 素数模高次剩余: 全部根的紧凑表示       | 123        |
| <b>第五十三章 素数幂模高次剩余</b>        | <b>124</b> |
| 53.1 素数幂模高次剩余: 含非互素被开方数      | 124        |
| <b>第五十四章 合数求根的准备</b>         | <b>127</b> |
| 54.1 合数求根准备: 分解模数与素数幂原根      | 127        |
| <b>第五十五章 合数模高次剩余</b>         | <b>128</b> |
| 55.1 一般合数模高次剩余: CRT 组合与按索引取根 | 128        |
| <b>第五十六章 多项式插值</b>           | <b>130</b> |
| 56.1 拉格朗日插值与连续点优化            | 130        |
| <b>第五十七章 参数化 NTT 卷积</b>      | <b>132</b> |
| 57.1 可指定素数与原根的 NTT 卷积        | 132        |
| <b>第五十八章 多项式平移</b>           | <b>134</b> |
| 58.1 多项式系数平移                 | 134        |

|                                     |            |
|-------------------------------------|------------|
| <b>第五十九章 等比点求值</b>                  | <b>135</b> |
| 59.1 Chirp Z: 等比点求值                 | 135        |
| <b>第六十章 斯特林数整行计算</b>                | <b>137</b> |
| 60.1 第二类斯特林数: 同一行前缀                 | 137        |
| 60.2 无符号第一类斯特林数: 同一行                | 138        |
| <b>第六十一章 集合变换与卷积</b>                | <b>139</b> |
| 61.1 按位 OR、AND、XOR 卷积               | 139        |
| <b>第六十二章 不相交子集卷积</b>                | <b>141</b> |
| 62.1 不相交子集卷积                        | 141        |
| <b>第六十三章 多项式</b>                    | <b>142</b> |
| 63.1 NTT、形式幂级数、FWT 与递推              | 142        |
| <b>第六十四章 代数与数论进阶</b>                | <b>146</b> |
| 64.1 模高斯消元、行列式、矩阵树定理                | 146        |
| 64.2 杜教筛                            | 149        |
| 64.3 扩展 BSGS                        | 150        |
| <b>第六十五章 任意模数行列式</b>                | <b>151</b> |
| 65.1 任意正模数行列式: 欧几里得消元               | 151        |
| <b>第六十六章 带权生成树计数</b>                | <b>152</b> |
| 66.1 带权矩阵树: 无向树与内外向树形图              | 152        |
| <b>第六十七章 合数模生成树计数</b>               | <b>153</b> |
| 67.1 任意正模数带权矩阵树                     | 153        |
| <b>第六十八章 计算几何</b>                   | <b>154</b> |
| 68.1 整数精确二维几何                       | 154        |
| 68.2 浮点点线圆构造                        | 156        |
| <b>第六十九章 优化与可持久化</b>                | <b>160</b> |
| 69.1 Max-plus 矩阵幂                   | 160        |
| 69.2 离散李超树与最优直线编号                   | 160        |
| 69.3 主席树: 静态区间第 $k$ 小               | 162        |
| <b>第七十章 可持久化数组</b>                  | <b>164</b> |
| 70.1 可持久化数组: 历史版本分叉与单点修改            | 164        |
| <b>第七十一章 可持久化区间操作</b>               | <b>167</b> |
| 71.1 可持久化区间加与版本拼接                   | 167        |
| <b>第七十二章 树上路径第 <math>k</math> 小</b> | <b>170</b> |
| 72.1 树上路径点权第 $k$ 小                  | 170        |
| <b>第七十三章 动态区间顺序统计</b>               | <b>173</b> |
| 73.1 动态区间第 $k$ 小 (树状数组套权值线段树)       | 173        |
| <b>第七十四章 区间不同数统计</b>                | <b>176</b> |
| 74.1 静态区间不同数计数 (前驱位置主席树)            | 176        |

|                               |            |
|-------------------------------|------------|
| <b>第七十五章 图论进阶</b>             | <b>178</b> |
| 75.1 矩形二分图最小权匹配               | 178        |
| 75.2 有向最小树形图                  | 179        |
| 75.3 无向全局最小割                  | 182        |
| <b>第七十六章 最小割树</b>             | <b>184</b> |
| 76.1 Gomory–Hu 最小割树           | 184        |
| <b>第七十七章 最小割树查询</b>           | <b>185</b> |
| 77.1 最小割树全点对查询                | 185        |
| <b>第七十八章 带权二分图匹配</b>          | <b>186</b> |
| 78.1 二分图最大权匹配（缺边与可选匹配）        | 186        |
| <b>第七十九章 精确有界半平面交</b>         | <b>188</b> |
| 79.1 整数半平面的有界多边形交             | 188        |
| <b>第八十章 圆与多边形面积</b>           | <b>191</b> |
| 80.1 圆与简单多边形相交面积              | 191        |
| <b>第八十一章 最小覆盖圆</b>            | <b>193</b> |
| 81.1 随机增量最小覆盖圆                | 193        |
| <b>第八十二章 圆的切线</b>             | <b>195</b> |
| 82.1 两圆公切线与过点切线               | 195        |
| <b>第八十三章 浮点最近点对</b>           | <b>197</b> |
| 83.1 浮点平面最近点对                 | 197        |
| <b>第八十四章 精确几何进阶</b>           | <b>199</b> |
| 84.1 整数最近点对与 Minkowski 和      | 199        |
| 84.2 整数三维共线、共面与方向             | 202        |
| <b>第八十五章 凸包支撑点查询</b>          | <b>203</b> |
| 85.1 支撑点查询恢复下凸包               | 203        |
| <b>第八十六章 动态树</b>              | <b>205</b> |
| 86.1 Link-cut tree: 动态森林与路径异或 | 205        |
| <b>第八十七章 随机平衡树</b>            | <b>208</b> |
| 87.1 FHQ Treap: 可重集合与顺序统计     | 208        |
| 87.2 隐式 Treap: 序列翻转、加值与区间和    | 210        |
| <b>第八十八章 伸展树</b>              | <b>214</b> |
| 88.1 Splay: 可重集合与顺序统计         | 214        |
| <b>第八十九章 状态序列维护</b>           | <b>218</b> |
| 89.1 状态序列 Treap: 分组区间 GCD     | 218        |
| <b>第九十章 一般图匹配</b>             | <b>222</b> |
| 90.1 一般图最大基数匹配: 带花树           | 222        |

|                               |            |
|-------------------------------|------------|
| <b>第九十一章 线性递推</b>             | <b>225</b> |
| 91.1 Berlekamp–Massey: 最短线性递推 | 225        |
| 91.2 特征多项式求线性递推第 $k$ 项        | 226        |
| <b>第九十二章 快速线性递推</b>           | <b>227</b> |
| 92.1 快速递推与有理生成函数单项            | 227        |
| <b>第二部分 数学速查与维护</b>           | <b>229</b> |
| <b>第九十三章 数学速查: 条件先于公式</b>     | <b>230</b> |
| 93.1 整除、卷积与筛法                 | 230        |
| 93.2 杜教筛: 预筛与缓存都不可省略          | 230        |
| 93.3 64 位判素与质因数分解             | 230        |
| 93.4 单个数的欧拉函数                 | 231        |
| 93.5 线性筛: 三张表的约定              | 231        |
| 93.6 区间筛: 只保存目标区间             | 232        |
| 93.7 矩形 GCD 计数: 缩放后容斥         | 232        |
| 93.8 整数分拆: 重数限制的差一问题          | 233        |
| 93.9 幂的约数和: 不除以 $p-1$         | 233        |
| 93.10 类欧几里德: 有符号整除求和          | 233        |
| 93.11 类欧几里德的三个求和量             | 234        |
| 93.12 模运算的适用条件                | 234        |
| 93.13 任意数组的批量逆元               | 235        |
| 93.14 连续整数的线性逆元表              | 235        |
| 93.15 一次不定方程: 特解与正整数解范围       | 235        |
| 93.16 线性同余: 最小解与最小周期          | 236        |
| 93.17 扩展 CRT: 逐式合并            | 236        |
| 93.18 Garner: 总周期可以超过整数范围     | 237        |
| 93.19 素数模平方根                  | 237        |
| 93.20 乘法阶与原根                  | 237        |
| 93.21 Carmichael: 最小公共指数      | 238        |
| 93.22 Lucas: 按素数进制逐位计算        | 238        |
| 93.23 合数模数组合数与插值              | 238        |
| 93.24 阶乘表: 组合数与排列数            | 239        |
| 93.25 组合计数                    | 239        |
| 93.26 群作用计数                   | 239        |
| 93.27 生成函数与递推                 | 239        |
| 93.28 线性代数与生成树                | 240        |
| 93.29 模线性方程组: 工厂工期            | 240        |
| 93.30 概率、期望与博弈                | 240        |
| 93.31 数值算法与几何                 | 240        |
| 93.32 线性递推: 系数方向与外推条件         | 241        |
| 93.33 素数模高次剩余与单位根             | 242        |
| 93.34 素数幂模高次剩余与根的提升           | 243        |
| 93.35 一般合数模高次剩余: CRT 组合       | 244        |
| 93.36 自动准备模数分解与原根             | 245        |
| 93.37 带权矩阵树定理与方向约定            | 246        |
| 93.38 任意正模数行列式                | 247        |

|                             |                         |            |
|-----------------------------|-------------------------|------------|
| 93.39                       | 任意正模数的矩阵树计数             | 248        |
| 93.40                       | 斯特林数的快速整行计算             | 249        |
| 93.41                       | 参数化 NTT 与模数适配           | 250        |
| 93.42                       | Bostan–Mori 与快速递推单项     | 251        |
| 93.43                       | 集合变换与不相交子集卷积            | 252        |
| 93.44                       | Taylor 平移与等比点求值         | 253        |
| 93.45                       | 可持久化区间拼接的懒标记            | 254        |
| 93.46                       | 杭州 2023 K：跨版本拼接应用       | 255        |
| 93.47                       | 非负权树的点集直径摘要             | 256        |
| 93.48                       | 函数图の入环链与到达查询            | 257        |
| 93.49                       | 杭州 2023 H：随机事件顺序与函数图    | 258        |
| 93.50                       | 杭州 2023 G：开放时间的单位边最短路   | 259        |
| 93.51                       | 带修莫队：时间轴与区间状态           | 259        |
| 93.52                       | SCC 模板题与速度榜口径           | 260        |
| 93.53                       | 并查集模板题的接口范围             | 260        |
| 93.54                       | ACL 的组合约定               | 260        |
| <b>第九十四章 计算几何：精确判定与适用范围</b> |                         | <b>261</b> |
| 94.1                        | 整数系数的有界半平面交             | 261        |
| 94.2                        | 最近点对、Minkowski 和与三维整数判定 | 262        |
| 94.3                        | 浮点最近点对                  | 263        |
| 94.4                        | 公切线与过点切线                | 264        |
| 94.5                        | 随机增量最小覆盖圆               | 265        |
| 94.6                        | 圆与简单多边形相交面积             | 266        |
| <b>第九十五章 来源与覆盖维护</b>        |                         | <b>267</b> |
| 95.1                        | 参考基线                    | 267        |
| 95.2                        | 后续覆盖项                   | 267        |

# 使用与验证

英文署名已于 QOJ 队伍账号 `ucup-team7502` 核对。按最新要求只保留 `vector` 实现，风格依据训练提交 2932575 提取，不擅自将共享队号的提交归属到个人。

代码采用 C++20。以赛时快速抄写为先，短语句可适度合并，复杂控制流与函数体保持清晰。无状态的简单算法使用独立函数，有状态算法使用轻量 `struct`。默认需要 `bits/stdc++.h` 和 `cassert`；各头文件列明依赖。抄写单个函数或结构体时需同时抄写其依赖结构。历史静态版仅留在 Git 历史与评测档案中，不属于当前模板库。点号除特别注明外为 1-based，字符串和多项式为 0-based；所有区间约定应以条目说明为准。

测试源码位于 `tests/`，固定种子便于复现。验证包含独立暴力参考、边界案例、静态与动态实现、AddressSanitizer 和 UndefinedBehaviorSanitizer。`verification/` 保存实际运行记录与源码摘要；OJ 状态只在获得可查提交结果后登记。最大流记录为 <https://www.luogu.com.cn/record/297501480> 与 <https://www.luogu.com.cn/record/297502364>。

原始来源目录共登记 713 项，见 `docs/coverage.csv`。它包含不同版本、例题及公式，不应将其数目视为独立算法数，也不能将“已登记”视为“已完成”。

# 第一部分

## vector 模板

# 第一章 Tarjan 与缩点

## 1.1 Tarjan 强连通分量与缩点

TarjanSCC(n), 点号 1..n; add(u,v) 加有向边, run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号, 跨分量边总从大编号指向小编号, 可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表, 保留重边; 若 DP 需要对不同邻点去重, 调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储  $O(n+m)$ , 递归 DFS 另需  $O(n)$  调用栈; 本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
1 struct TarjanSCC
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<int>>> g;
5     vector<int> dfn, low, bel, st;
6
7     TarjanSCC(int n) : n(n), g(n + 1) {}
8
9     void add(int u, int v)
10    {
11        assert(1 <= u && u <= n && 1 <= v && v <= n);
12        g[u].push_back(v);
13    }
14
15    void dfs(int u)
16    {
17        dfn[u] = low[u] = ++timer;
18        st.push_back(u);
19        for (int v : g[u])
20        {
21            if (!dfn[v])
22            {
23                dfs(v);
24                low[u] = min(low[u], low[v]);
25            }
26            else if (!bel[v])
27                low[u] = min(low[u], dfn[v]);
28        }
29        if (low[u] == dfn[u])
30        {
31            ++cnt;
32            while (true)
33            {
34                int v = st.back();
35                st.pop_back();
36                bel[v] = cnt;
37                if (v == u) break;
38            }
39        }
40    }
41
42    // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
43    void run()
```

```
44     {
45         timer = 0;
46         cnt = 0;
47         dfn.assign(n + 1, 0);
48         low = dfn;
49         bel = dfn;
50         st.clear();
51         for (int u = 1; u <= n; u++)
52         {
53             if (!dfn[u]) dfs(u);
54         }
55     }
56
57     // Retains parallel edges. Use after run().
58     vector<vector<int>> dag() const
59     {
60         vector<vector<int>> result(cnt + 1);
61         for (int u = 1; u <= n; u++)
62         {
63             for (int v : g[u])
64             {
65                 if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
66             }
67         }
68         return result;
69     }
70 };
```

## 第二章 函数图

### 2.1 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT\_MAX，实际规模须满足内存。

```
1 struct FunctionalGraph
2 {
3     using U = unsigned long long;
4     int n;
5     vector<int> component, position, entry, depth, order;
6     vector<vector<int>> cycles, up;
7
8     explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }
9
10    void build(vector<int> to)
11    {
12        assert(to.size() <= INT_MAX);
13        n = to.size();
14        vector<int> degree(n);
15        for (int v : to)
16        {
17            assert(0 <= v && v < n);
18            degree[v]++;
19        }
20        order.clear();
21        for (int u = 0; u < n; u++)
22            if (degree[u] == 0) order.push_back(u);
23        for (int i = 0; i < int(order.size()); i++)
24        {
25            int v = to[order[i]];
26            if (--degree[v] == 0) order.push_back(v);
27        }
28    }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```

28     component.assign(n, -1);
29     position.assign(n, -1);
30     entry.resize(n);
31     depth.assign(n, 0);
32     cycles.clear();
33     for (int u = 0; u < n; u++)
34         if (degree[u] && component[u] == -1)
35             {
36                 vector<int> cycle;
37                 int v = u, id = cycles.size();
38                 do
39                     {
40                         component[v] = id;
41                         position[v] = cycle.size();
42                         entry[v] = v;
43                         cycle.push_back(v);
44                         v = to[v];
45                     } while (v != u);
46                 cycles.push_back(move(cycle));
47             }
48     for (int i = int(order.size()) - 1; i >= 0; i--)
49     {
50         int u = order[i], v = to[u];
51         component[u] = component[v];
52         entry[u] = entry[v];
53         depth[u] = depth[v] + 1;
54     }
55     up.assign(1, to);
56     for (int k = 1; (1ULL << k) <= U(n); k++)
57     {
58         vector<int> row(n);
59         for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
60         up.push_back(move(row));
61     }
62 }
```

跳转与最少步数接口（接上页同一结构体）:

```

64     int advance(int u, U k) const
65     {
66         assert(0 <= u && u < n);
67         if (k >= U(depth[u]))
68         {
69             k -= depth[u];
70             u = entry[u];
71             const auto &cycle = cycles[component[u]];
72             return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
73         }
74         for (int bit = 0; k; bit++, k >>= 1)
75             if (k & 1) u = up[bit][u];
76         return u;
77     }
78
79     int steps(int u, int v) const
80     {
81         assert(0 <= u && u < n && 0 <= v && v < n);
82         if (component[u] != component[v] || depth[u] < depth[v]) return -1;
83         if (depth[v] > 0)
84         {
85             int d = depth[u] - depth[v];
86             return advance(u, d) == v ? d : -1;
87         }
88         int offset = position[v] - position[entry[u]];
89         if (offset < 0) offset += int(cycles[component[u]].size());
90         return depth[u] + offset;
91     }
92 };

```

## 第三章 双连通分量与圆方树

### 3.1 Tarjan 点双、边双与圆方树

Biconnected(n), 点号 1..n; add(u,v) 返回原边的 0-based 编号, run() 从头计算。blocks 为点双的顶点列表: 桥的两端构成一个块, 孤立点独立成块; 同一个割点可出现在多个块。cut[u] 标记割点, bridge[id] 标记桥; bel[u] 是 1..cnt 的边双编号。block\_forest() 返回圆方森林, 原点 1..n, blocks[i] 的方点编号 n+i+1; 孤立点也连接自己的单点方块。bridge\_forest() 返回边双缩点森林, 邻接项为 (相邻分量, 原桥编号)。重边按边号处理; 自环保留原边编号但不参与 DFS, 不是桥, 不额外产生点双。全程  $O(n+m)$ , 递归 DFS, 分量栈不替代递归调用。两种森林必须在 run 后获取。

```
1 struct Biconnected
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<vector<int>> blocks;
7     vector<int> dfn, low, cut, bridge, bel, st;
8
9     Biconnected(int n) : n(n), g(n + 1) {}
10
11     int add(int u, int v)
12     {
13         assert(1 <= u && u <= n && 1 <= v && v <= n);
14         int id = edges.size();
15         edges.push_back({u, v});
16         if (u != v)
17         {
18             g[u].push_back({v, id});
19             g[v].push_back({u, id});
20         }
21         return id;
22     }
23
24     void dfs(int u, int parent_edge)
25     {
26         dfn[u] = low[u] = ++timer;
27         st.push_back(u);
28         int children = 0;
29         for (auto [v, id] : g[u])
30         {
31             if (id == parent_edge) continue;
32             if (!dfn[v])
33             {
34                 children++;
35                 dfs(v, id);
36                 low[u] = min(low[u], low[v]);
37                 if (low[v] > dfn[u]) bridge[id] = 1;
38                 if (low[v] >= dfn[u])
39                 {
40                     if (parent_edge != -1 || children > 1) cut[u] = 1;
41                     blocks.push_back({u});
```

```

42         while (true)
43         {
44             int x = st.back();
45             st.pop_back();
46             blocks.back().push_back(x);
47             if (x == v) break;
48         }
49     }
50 }
51 else
52     low[u] = min(low[u], dfn[v]);
53 }
54 if (parent_edge == -1 && children == 0) blocks.push_back({u});
55 }
56
57 void paint(int u)
58 {
59     bel[u] = cnt;
60     for (auto [v, id] : g[u])
61     {
62         if (!bridge[id] && !bel[v]) paint(v);
63     }
64 }
65
66 void run()
67 {
68     timer = 0;
69     cnt = 0;
70     dfn.assign(n + 1, 0);
71     low = dfn;
72     cut = dfn;
73     bel = dfn;
74     bridge.assign(edges.size(), 0);
75     blocks.clear();
76     st.clear();
77     for (int u = 1; u <= n; u++)
78     {
79         if (!dfn[u])
80         {
81             dfs(u, -1);
82             st.pop_back();
83         }
84     }
85     for (int u = 1; u <= n; u++)
86     {
87         if (bel[u]) continue;
88         ++cnt;
89         paint(u);
90     }
91 }
92
93 // Original vertices 1..n; block i has node n+i+1. Includes isolated vertices.
94 vector<vector<int>> block_forest() const
95 {
96     vector<vector<int>> tree(n + blocks.size() + 1);
97     for (int i = 0; i < (int)blocks.size(); i++)

```

```
98     {
99         int b = n + i + 1;
100         for (int u : blocks[i])
101         {
102             tree[u].push_back(b);
103             tree[b].push_back(u);
104         }
105     }
106     return tree;
107 }
108
109 // Edge-connected component IDs 1..cnt; forest edges carry original edge IDs.
110 vector<vector<pair<int, int>>> bridge_forest() const
111 {
112     vector<vector<pair<int, int>>> tree(cnt + 1);
113     for (int id = 0; id < (int)edges.size(); id++)
114     {
115         if (!bridge[id]) continue;
116         auto [u, v] = edges[id];
117         u = bel[u];
118         v = bel[v];
119         tree[u].push_back({v, id});
120         tree[v].push_back({u, id});
121     }
122     return tree;
123 }
124 };
```

## 第四章 奇环顶点判定

### 4.1 属于简单奇环的全部顶点

先对 Biconnected 调用 run, 再调用 odd\_cycle\_vertices(graph), 返回 answer[1..n], 值为 1 表示该点属于某个长度至少为 3 的简单奇环, answer[0]=0。允许重边、断开图及空图; 自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环, 此时该点双每个顶点都属于某个奇环; 答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双: 两端中森林深度较大者的父方点即该块, 同深时二者父方点相同。随后只对块内边染色, 避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归, 分解后额外  $O(n+m)$  时间和空间。可重排 blocks, 不能改变分解内容。六点以内所有简单图枚举奇环对拍, 另有重边/自环、长链、星形和大奇环测试; POJ2942 补图驱动本地验证, 在线待补。依赖 biconnected.hpp。

```
1 // After run(). Simple odd cycles have length >= 3; self-loops are ignored.
2 inline vector<int> odd_cycle_vertices(const Biconnected &graph)
3 {
4     int n = graph.n;
5     auto forest = graph.block_forest();
6     vector<int> parent(forest.size(), depth(forest.size()));
7     auto root_tree = [&](auto &&self, int u, int p) -> void
8     {
9         parent[u] = p;
10        for (int v : forest[u])
11            if (v != p)
12            {
13                depth[v] = depth[u] + 1;
14                self(self, v, u);
15            }
16    };
17    for (int u = 1; u <= n; u++)
18        if (!depth[u])
19        {
20            depth[u] = 1;
21            root_tree(root_tree, u, 0);
22        }
```

```

23     vector<vector<pair<int, int>>> edges(graph.blocks.size());
24     for (auto [u, v] : graph.edges)
25         if (u != v)
26         {
27             int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
28             assert(0 <= block && block < (int)edges.size());
29             edges[block].push_back({u, v});
30         }
31     vector<vector<int>> g(n + 1);
32     vector<int> color(n + 1), answer(n + 1);
33     for (int block = 0; block < (int)edges.size(); block++)
34     {
35         for (int u : graph.blocks[block])
36         {
37             g[u].clear();
38             color[u] = 0;
39         }
40         for (auto [u, v] : edges[block])
41         {
42             g[u].push_back(v);
43             g[v].push_back(u);
44         }
45         bool bipartite = true;
46         auto paint = [&](auto &&self, int u) -> void
47         {
48             for (int v : g[u])
49                 if (!color[v])
50                 {
51                     color[v] = -color[u];
52                     self(self, v);
53                 }
54             else if (color[v] == color[u])
55                 bipartite = false;
56         };
57         int root = graph.blocks[block][0];
58         color[root] = 1;
59         paint(paint, root);
60         if (!bipartite)
61             for (int u : graph.blocks[block]) answer[u] = 1;
62     }
63     return answer;
64 }

```

## 第五章 删点连通性

### 5.1 由点双分解计算删点连通块数

在 `Biconnected.run()` 后调用 `removal_components(graph)`, 返回 (原连通块数 `before`, 删点结果 `after`)。 `after[u]` 为独立删除原顶点 `u` 后剩余连通块数, 点号 `1..n`; `after[0]=0` 不参与计算。 `before` 等于边双数量减去桥数量; `after[u]` 等于 `before-1` 加包含 `u` 的非单点点双数量。单点块必须跳过, 因此孤立点删去后会少一个连通块。空图返回 `(0,{0})`。不修改原分解,  $O(n+m)$  时间、 $O(n)$  额外空间。若只需要割点、桥和删点增量, 直接用更短的 `Lowlink/Low_Link`; 已经需要点双分解时可用本函数, 避免再次 DFS。

```
1 // Call after run(). Each vertex is deleted independently from the original graph.
2 inline pair<int, vector<int>> removal_components(const Biconnected &graph)
3 {
4     int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
5     vector<int> after(graph.n + 1, before - 1);
6     after[0] = 0;
7     for (const auto &block : graph.blocks)
8         if (block.size() > 1)
9             for (int u : block) after[u]++;
10    return {before, after};
11 }
```

## 第六章 边双缩点与边定向

### 6.1 边双缩点的大小与内部边数

在 `Biconnected.run()` 后构造 `EdgeCompression(graph)`。`n` 为边双数量；`size[c]` 为第 `c` 块的原顶点数，`inside[c]` 为块内逻辑无向边数，分量号 `1..n`，`0` 号槽忽略。`edges` 每项为 {块 `u`, 块 `v`, 原边 `id`}，只含跨块的桥，每条桥出现一次，按原边号递增；是无向森林，不承诺端点为父子方向。重边逐条计数，自环在所属块内部计一次，不能用邻接表度数直接计边而漏掉这些约定。空图返回 `n=0`、长度 `1` 的零计数数组和空边表。时间  $O(N+M)$ ，输出空间  $O(N+M)$ ， $N/M$  指原图规模。

```
1 struct EdgeCompression
2 {
3     int n;
4     vector<int> size, inside;
5     vector<array<int, 3>> edges;
6
7     EdgeCompression(const Biconnected &graph) : n(graph.cnt), size(n + 1), inside(n + 1)
8     {
9         for (int u = 1; u <= graph.n; u++) size[graph.bel[u]]++;
10        for (int id = 0; id < (int)graph.edges.size(); id++)
11        {
12            auto [u, v] = graph.edges[id];
13            u = graph.bel[u];
14            v = graph.bel[v];
15            if (u == v)
16                inside[u]++;
17            else
18                edges.push_back({u, v, id});
19        }
20    }
21};
```

## 6.2 无向边定向与强连通定向

`orient_edges(graph)` 接受 Lowlink 对象。返回长度等于逻辑边数的方向数组，第 `id` 项为该边定向后的 {起点, 终点}，顶点 1-based、边 `id` 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 `run/Run` 检查 `components==1` 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 `run`。复杂度  $O(N+M)$  时间、 $O(N+M)$  额外空间，DFS 保持递归。

```

1 // Tree edges go down; other edges go to ancestors. Handles each logical edge once.
2 inline vector<pair<int, int>> orient_edges(const Lowlink &graph)
3 {
4     vector<pair<int, int>> direction(graph.edges);
5     vector<int> seen(graph.n + 1);
6     auto dfs = [&](auto &&self, int u) -> void
7     {
8         seen[u] = 1;
9         for (auto [v, id] : graph.g[u])
10         {
11             if (direction[id].first) continue;
12             direction[id] = {u, v};
13             if (!seen[v]) self(self, v);
14         }
15     };
16     for (int u = 1; u <= graph.n; u++)
17         if (!seen[u]) dfs(dfs, u);
18     return direction;
19 }
```

## 第七章 桥树最少加边

### 7.1 连通无向图最少加边方案

先对非空连通无向图调用 `Biconnected.run()`，再调用 `bridge_augmentation(graph)`。返回原图点号构成的新增边列表，不修改原图。允许新增重边，不产生自环；不适用于非连通图。已无桥时返回空列表。桥树有  $L$  个叶子时最少加  $(L+1)/2$  条边（整数除法）。递归 DFS 顺序收集叶子，奇数时复制首叶，前后半段配对。每条桥一侧的叶子是循环连续区间，半周配对必跨过该割，故所有桥被消除；每个叶子至少需要一个新边端点，得到同样的下界。分解后本函数  $O(n+m)$  时间与空间。依赖 `biconnected.hpp`。P2860 双风格完整程序本地验证；新增方案以逐边删除连通性独立验证，尚无本库在线 AC。

```
1 // Connected, nonempty graph after run(); new parallel edges are allowed.
2 inline vector<pair<int, int>> bridge_augmentation(const Biconnected &graph)
3 {
4     assert(graph.n > 0 && graph.cnt > 0);
5     int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
6     assert(graph.cnt - bridges == 1);
7     if (graph.cnt == 1) return {};
8     auto tree = graph.bridge_forest();
9     vector<int> representative(graph.cnt + 1);
10    for (int u = 1; u <= graph.n; u++) representative[graph.bel[u]] = u;
11    vector<int> leaves;
12    auto dfs = [&](auto &&self, int u, int parent) -> void
13    {
14        if (tree[u].size() == 1) leaves.push_back(u);
15        for (auto [v, id] : tree[u])
16            if (v != parent) self(self, v, u);
17    };
18    dfs(dfs, 1, 0);
19    if (leaves.size() % 2) leaves.push_back(leaves[0]);
20    int half = leaves.size() / 2;
21    vector<pair<int, int>> added;
22    for (int i = 0; i < half; i++)
23        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
24    return added;
25 }
```

# 第八章 数据结构

## 8.1 并查集

dsu(n), 点号 0..n-1,  $n \geq 0$ 。find(x) 路径减半; same(x,y) 判断同集合; merge(x,y) 把 y 的根接到 x 的根, 已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组, 组内点号递增、组间按当前根编号排列; 可能压缩路径, 但不改变分组。fa、sz 可读, sz 仅根处有效; 不要外部改写。本版只做路径减半, 不按大小或秩合并; 单次 find 最坏  $O(n)$ 。空间  $O(n)$ , groups 另需  $O(n)$  输出与工作空间, 并执行 n 次 find。空结构可调用 groups, 不能查询顶点。

```
1 struct dsu
2 {
3     vector<int> fa, sz;
4
5     dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }
6
7     int find(int x)
8     {
9         while (x != fa[x]) x = fa[x] = fa[fa[x]];
10        return x;
11    }
12
13    bool same(int x, int y) { return find(x) == find(y); }
14
15    bool merge(int x, int y)
16    {
17        x = find(x), y = find(y);
18        if (x == y) return false;
19        sz[x] += sz[y], fa[y] = x;
20        return true;
21    }
22
23    int size(int x) { return sz[find(x)]; }
24
25    vector<vector<int>> groups()
26    {
27        vector<vector<int>> res(fa.size());
28        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
29        res.erase(
30            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
31            res.end());
32        return res;
33    }
34};
```

## 8.2 可撤销并查集

构造后 `snapshot()` 保存历史长度, `rollback(t)` 回退。禁止路径压缩; `find/merge` 为  $O(\log n)$ , 回退每次成功合并  $O(1)$ 。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }
7
8     int find(int x) const
9     {
10         while (x != fa[x]) x = fa[x];
11         return x;
12     }
13
14     int snapshot() const { return (int)his.size(); }
15
16     bool merge(int x, int y)
17     {
18         x = find(x);
19         y = find(y);
20         if (x == y) return false;
21         if (siz[x] < siz[y]) swap(x, y);
22         his.push_back({y, siz[x]});
23         fa[y] = x;
24         siz[x] += siz[y];
25         return true;
26     }
27
28     void rollback(int t)
29     {
30         assert(0 <= t && t <= snapshot());
31         while (snapshot() > t)
32         {
33             auto [y, s] = his.back();
34             his.pop_back();
35             siz[fa[y]] = s;
36             fa[y] = y;
37         }
38     }
39 };

```

## 8.3 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标  $1..n$ ，闭区间； $\text{kth}(k)$  要求所有频率非负且  $k>0$ ，超过总和返回  $n+1$ 。单次  $O(\log n)$ 。T 须能从 0 构造并支持  $+=$  与减法；可直接使用 `ModInt` 维护模数下的区间和。 $\text{kth}$  只适用于有序累计频数，不适用于取模后的区间和。

```

1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };

```

## 8.4 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护  $\text{sum}$  与加法  $\text{tag}$ ；所有中间和与长度乘积须在  $\text{long long}$  范围内。单次  $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }
7
8     void apply(int p, int l, int r, long long v)
9     {
10         sum[p] += (r - l + 1) * v;
11         tag[p] += v;
12     }
13
14     void push(int p, int l, int r)
15     {
16         int m = (l + r) / 2;
17         apply(p * 2, l, m, tag[p]);
18         apply(p * 2 + 1, m + 1, r, tag[p]);
19         tag[p] = 0;
20     }
21
22     void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }
23
24     void add(int p, int l, int r, int ql, int qr, long long v)
25     {
26         if (ql <= l && r <= qr)
27         {
28             apply(p, l, r, v);
29             return;
30         }
31         push(p, l, r);
32         int m = (l + r) / 2;
33         if (ql <= m) add(p * 2, l, m, ql, qr, v);
34         if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
35         sum[p] = sum[p * 2] + sum[p * 2 + 1];
36     }
37
38     long long query(int l, int r) { return query(1, 1, n, l, r); }
39
40     long long query(int p, int l, int r, int ql, int qr)
41     {
42         if (ql <= l && r <= qr) return sum[p];
43         push(p, l, r);
44         int m = (l + r) / 2;
45         long long ans = 0;
46         if (ql <= m) ans += query(p * 2, l, m, ql, qr);
47         if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
48         return ans;
49     }
50 };

```

## 8.5 64 位异或线性基

值类型  $U$  为 `unsigned long long`，完整支持 64 位。默认空基；`init/Init` 清空。`insert/Insert` 返回是否增秩；`contains/Contains` 判断可表示性（允许空子集，所以 0 总可表示）。`query/Query(x=0)` 返回  $x$  与所张成空间异或后的最大无符号值。`kth/Kth(k,nonempty=true)` 返回第  $k$  小的不同异或值，1-based，参数  $k$  为 `__uint128_t`，非法序号返回 `nullopt`。默认只允许非空输入子集，0 仅在 `dependent` 为真时出现；传 `false` 则包括空子集所得的 0。`minimum/Minimum` 同此约定，空输入的非空最小值不存在。`merge/Merge` 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。`rank` 是维数，`dependent` 表示存在非空零异或子集。插入、成员判断、最大值为  $O(64)$ ；合并  $O(64^2)$ 。`kth` 会在需要时 `rebuild/Rebuild` 规范化缓存，首次有效查询  $O(64^2)$ ，后续  $O(64)$ ；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```

1 struct XorBasis
2 {
3     using U = unsigned long long;
4     using Wide = __uint128_t;
5     U a[64]{};
6     vector<U> p;
7     int rank = 0;
8     bool dependent = false, dirty = false;
9
10    void init()
11    {
12        fill(a, a + 64, 0);
13        p.clear();
14        rank = 0;
15        dependent = false;
16        dirty = false;
17    }
18
19    bool insert(U x)
20    {
21        for (int i = 63; i >= 0; i--)
22            if (x >> i & 1)
23            {
24                if (!a[i])
25                {
26                    a[i] = x;
27                    rank++;
28                    dirty = true;
29                    return true;
30                }
31                x ^= a[i];
32            }
33        dependent = true;
34        return false;
35    }

```

```
37     bool contains(U x) const
38     {
39         for (int i = 63; i >= 0; i--)
40             if (x >> i & 1) x ^= a[i];
41         return x == 0;
42     }
43
44     U query(U x = 0) const
45     {
46         for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
47         return x;
48     }
49
50     void rebuild()
51     {
52         vector<U> b(a, a + 64);
53         for (int i = 0; i < 64; i++)
54             if (b[i])
55                 for (int j = i + 1; j < 64; j++)
56                     if (b[j] >> i & 1) b[j] ^= b[i];
57         p.clear();
58         for (U x : b)
59             if (x) p.push_back(x);
60         dirty = false;
61     }
```

```

63 // k is 1-based; nonempty excludes the empty input subset, not value zero.
64 optional<U> kth(Wide k, bool nonempty = true)
65 {
66     if (!k) return nullopt;
67     Wide index = k;
68     if (!nonempty || dependent) index--;
69     if (index >= (Wide(1) << rank)) return nullopt;
70     if (dirty) rebuild();
71     U answer = 0;
72     for (int i = 0; i < rank; i++)
73         if (index >> i & 1) answer ^= p[i];
74     return answer;
75 }
76
77 optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }
78
79 void merge(const XorBasis &other)
80 {
81     dependent |= other.dependent;
82     for (U x : other.a)
83         if (x) insert(x);
84 }
85 };

```

## 第九章 带修改的莫队

### 9.1 带修改的莫队

ModifiedMo( $n, m$ ) 保存离线询问,  $n > 0$ ,  $m$  为修改总数。add( $l, r, t$ ) 登记第  $t$  次修改完成后的  $[l, r]$ , 返回从 0 开始的询问编号;  $0 \leq l \leq r \leq n$ ,  $0 \leq t \leq m$ , 询问数小于 INT\_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add( $p$ )/del( $p$ ) 按下标更新当前区间统计; chg( $i, forward, l, r$ ) 将编号  $i$  的修改前进或回退, 并同步修正当前  $[l, r]$  内的贡献; ans( $id$ ) 记录结果。修改编号从 0 开始, 前进第  $i$  次修改时原时间为  $i$ , 回退时原时间为  $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约  $n^{2/3}$ , 也可显式指定正块长。 $n$ 、询问数、修改数同阶且回调  $O(1)$  时总时间  $O(n^{5/3})$ , 调度器空间  $O(q)$ 。

```
1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }
```

```

22  template <class Add, class Del, class Chg, class Ans>
23  void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24  {
25      if (q.empty()) return;
26      if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27      assert(block > 0);
28      auto ord = q;
29      auto cmp = [&](const Query &a, const Query &b)
30      {
31          int x = a.l / block, y = b.l / block;
32          if (x != y) return x < y;
33          int p = a.r / block, q = b.r / block;
34          if (p != q) return (x & 1) ? p > q : p < q;
35          return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36      };
37      sort(ord.begin(), ord.end(), cmp);
38      int l = 0, r = 0, t = 0;
39      for (auto q : ord)
40      {
41          while (t < q.t)
42          {
43              chg(t, true, l, r);
44              t++;
45          }
46          while (t > q.t)
47          {
48              t--;
49              chg(t, false, l, r);
50          }
51          while (l > q.l) add(--l);
52          while (r < q.r) add(r++);
53          while (l < q.l) del(l++);
54          while (r > q.r) del(--r);
55          ans(q.id);
56      }
57  }
58  };

```

# 第十章 带位置线性基

## 10.1 带位置限制的异或线性基

默认构造为空，init/Init 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作  $O(64)$ ，空间  $O(64)$ 。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为  $O(64n)$ 。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
1 struct PositionBasis
2 {
3     using U = unsigned long long;
4     U a[64];
5     int pos[64];
6
7     PositionBasis() { init(); }
8
9     void init()
10    {
11        fill(a, a + 64, 0);
12        fill(pos, pos + 64, -1);
13    }
```

```

15 void insert(U x, int at)
16 {
17     assert(at >= 0);
18     for (int i = 63; i >= 0; i--)
19         if (x >> i & 1)
20             {
21                 if (!a[i])
22                     {
23                         a[i] = x;
24                         pos[i] = at;
25                         return;
26                     }
27                 if (at > pos[i])
28                     {
29                         swap(a[i], x);
30                         swap(pos[i], at);
31                     }
32                 x ^= a[i];
33             }
34 }
35
36 bool contains(U x, long long lower = 0) const
37 {
38     assert(lower >= 0);
39     for (int i = 63; i >= 0; i--)
40         if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
41     return x == 0;
42 }
43
44 U query(long long lower = 0, U x = 0) const
45 {
46     assert(lower >= 0);
47     for (int i = 63; i >= 0; i--)
48         if (pos[i] >= lower) x = max(x, x ^ a[i]);
49     return x;
50 }
51
52 int rank(long long lower = 0) const
53 {
54     assert(lower >= 0);
55     int answer = 0;
56     for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
57     return answer;
58 }
59 };

```

# 第十一章 线性空间求交

## 11.1 异或线性空间求交

`basis_intersection(a,b)` 返回 `span(a)` 与 `span(b)` 的交空间的一组独立生成元，结果类型 `XorBasis`。支持完整 64 位无符号值，不修改输入，也不要求先 `rebuild`。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间  $O(64^2)$ ，额外空间  $O(64)$ 。输出按独立生成元解释：`dependent` 为 `false`，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 `result.kth(k,false)`，包含零；零空间返回空基，`rank=0`、`contains(0)=true`。

```
1 inline XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
2 {
3     using U = unsigned long long;
4     U pivot[64], from_b[64]{};
5     copy(a.a, a.a + 64, pivot);
6     XorBasis result;
7     for (U value : b.a)
8     {
9         U x = value, part = value;
10        for (int i = 63; i >= 0 && x; i--)
11            if (x >> i & 1)
12            {
13                if (!pivot[i])
14                {
15                    pivot[i] = x;
16                    from_b[i] = part;
17                    break;
18                }
19                x ^= pivot[i];
20                part ^= from_b[i];
21            }
22        if (!x && part) result.insert(part);
23    }
24    return result;
25 }
```

## 11.2 Zassenhaus：同时求和空间与交空间

`basis_sum_intersection(a,b)` 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 `dependent` 均为 `false`。将 `a` 的向量 `x` 编成 `(x,x)`，`b` 的向量 `y` 编成 `(y,0)`，用 `__uint128_t` 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间  $O(128^2)$ ，额外空间  $O(128)$ 。结果满足  $\dim(A+B)+\dim(A \cap B)=\dim(A)+\dim(B)$ ，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```

1 // Return independent generators of (sum space, intersection space).
2 inline pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a,
3                                                         const XorBasis &b)
4 {
5     using W = __uint128_t;
6     W pivot[128]{};
7     auto insert = [&](W x)
8     {
9         for (int i = 127; i >= 0 && x; i--)
10             if (x >> i & 1)
11             {
12                 if (!pivot[i])
13                 {
14                     pivot[i] = x;
15                     return;
16                 }
17                 x ^= pivot[i];
18             }
19     };
20     for (auto x : a.a) insert((W(x) << 64) | x);
21     for (auto x : b.a) insert(W(x) << 64);
22     XorBasis sum, intersection;
23     for (int i = 0; i < 128; i++)
24         if (pivot[i])
25         {
26             if (i < 64)
27                 intersection.insert((unsigned long long)pivot[i]);
28             else
29                 sum.insert((unsigned long long)(pivot[i] >> 64));
30         }
31     return {sum, intersection};
32 }
```

# 第十二章 线段树常用修改

## 12.1 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素  $x$  改为  $mul \cdot x + add$ ；乘法用  $(mul,0)$ ，加法用  $(1,add)$ ，赋值用  $(0,value)$ 。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT\_MAX$ ，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记  $(m,b)$  后应用新标记  $(a,c)$ ，复合为  $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树  $O(n)$ ，单次  $O(\log n)$ ，空间  $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```
1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
```

```

43 void apply(int p, int len, ll a, ll b)
44 {
45     t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46     t[p].mul = t[p].mul * a % mod;
47     t[p].add = (t[p].add * a % mod + b) % mod;
48 }
49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

# 第十三章 网络流

## 13.1 最大流与方案

Dinic(n), add(u,v,c) 返回偶数原边编号; flow(s,t,limit) 返回本次新增流; used(id) 查询原边净流, cut(s) 返回残量可达点集。允许重边、自环, 容量非负,  $s \neq t$ ; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般  $O(V^2E)$ , 递归增广深度  $O(V)$ 。总流不得超过 long long 上限。

```
1 struct Dinic
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll cap, initial;
9     };
10
11     int n;
12     vector<Edge> e;
13     vector<vector<int>> g;
14     vector<int> dep, cur;
15
16     Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}
17
18     int add(int u, int v, ll c)
19     {
20         assert(c >= 0);
21         int id = (int)e.size();
22         e.push_back({u, v, c, c});
23         e.push_back({v, u, 0, 0});
24         g[u].push_back(id);
25         g[v].push_back(id + 1);
26         return id;
27     }
28
29     bool bfs(int s, int t)
30     {
31         fill(dep.begin(), dep.end(), -1);
32         queue<int> q;
33         dep[s] = 0;
34         q.push(s);
35         while (!q.empty())
36         {
37             int u = q.front();
38             q.pop();
39             for (int id : g[u])
40                 if (e[id].cap && dep[e[id].to] < 0)
41                 {
42                     dep[e[id].to] = dep[u] + 1;
43                     q.push(e[id].to);
44                 }
```

```

45     }
46     return dep[t] >= 0;
47 }
48
49 ll dfs(int u, int t, ll f)
50 {
51     if (u == t) return f;
52     for (int &i = cur[u]; i < (int)g[u].size(); i++)
53     {
54         int id = g[u][i], v = e[id].to;
55         if (!e[id].cap || dep[v] != dep[u] + 1) continue;
56         ll d = dfs(v, t, min(f, e[id].cap));
57         if (d)
58         {
59             e[id].cap -= d;
60             e[id ^ 1].cap += d;
61             return d;
62         }
63     }
64     return 0;
65 }
66
67 // Returns additional flow; reuse only with the same source/sink.
68 ll flow(int s, int t, ll limit = LLONG_MAX)
69 {
70     assert(s != t && limit >= 0);
71     ll ans = 0;
72     while (ans < limit && bfs(s, t))
73     {
74         fill(cur.begin(), cur.end(), 0);
75         while (ans < limit)
76         {
77             ll d = dfs(s, t, limit - ans);
78             if (!d) break;
79             ans += d;
80         }
81     }
82     return ans;
83 }
84
85 ll used(int id) const { return e[id].initial - e[id].cap; }
86
87 vector<int> cut(int s)
88 {
89     bfs(s, s);
90     vector<int> ans;
91     for (int u = 1; u <= n; u++)
92         if (dep[u] >= 0) ans.push_back(u);
93     return ans;
94 }
95 };

```

## 13.2 势能最小费用最大流

$\text{add}(u, v, \text{cap}, \text{cost})$  返回边编号,  $\text{flow}(s, t, \text{limit})$  返回新增流量及 128 位费用,  $\text{used}(id)$  返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化  $O(VE)$ , 每次增广  $O(E \log V)$ ; 增广次数不能简单假设为  $O(V)$ 。另要求  $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$ , 防止总费用超过  $\text{int128}$ 。

```

1 struct MinCostFlow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>> g;
15
16    MinCostFlow(int n) : n(n), g(n + 1) {}
17
18    int add(int u, int v, ll cap, ll cost)
19    {
20        assert(cap >= 0 && cost != LLONG_MIN);
21        int id = (int)e.size();
22        e.push_back({u, v, cap, cost, cap});
23        e.push_back({v, u, 0, -cost, 0});
24        g[u].push_back(id);
25        g[v].push_back(id + 1);
26        return id;
27    }
28
29    ll used(int id) const { return e[id].initial - e[id].cap; }
30
31    // Negative costs allowed; initial residual graph must have no negative cycle.
32    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
33    {
34        assert(s != t && limit >= 0);
35        const i128 inf = i128(1) << 120;
36        vector<i128> h(n + 1, 0), d(n + 1);
37        vector<int> pre(n + 1);
38        // Super-source Bellman-Ford: feasible potentials for every vertex.
39        for (int it = 0; it < n; it++)
40        {
41            bool changed = false;
42            for (auto &a : e)
43                if (a.cap && h[a.to] > h[a.from] + a.cost)
44                {
45                    h[a.to] = h[a.from] + a.cost;
46                    changed = true;
47                }
48            if (!changed) break;
49            if (it == n - 1) throw invalid_argument("negative cycle");
50        }

```

```

51     ll f = 0;
52     i128 cost = 0;
53     while (f < limit)
54     {
55         fill(d.begin(), d.end(), inf);
56         d[s] = 0;
57         priority_queue<pair<i128, int>,
58                       vector<pair<i128, int>>,
59                       greater<pair<i128, int>>>
60         q;
61         q.push({0, s});
62         while (!q.empty())
63         {
64             auto [du, u] = q.top();
65             q.pop();
66             if (du != d[u]) continue;
67             for (int id : g[u])
68             {
69                 auto &a = e[id];
70                 i128 nd = du + a.cost + h[u] - h[a.to];
71                 if (a.cap && nd < d[a.to])
72                 {
73                     d[a.to] = nd;
74                     pre[a.to] = id;
75                     q.push({nd, a.to});
76                 }
77             }
78         }
79         if (d[t] == inf) break;
80         for (int u = 1; u <= n; u++)
81             if (d[u] != inf) h[u] += d[u];
82         ll take = limit - f;
83         for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
84         for (int u = t; u != s; u = e[pre[u]].from)
85         {
86             int id = pre[u];
87             e[id].cap -= take;
88             e[id ^ 1].cap += take;
89             cost += i128(take) * e[id].cost;
90         }
91         f += take;
92     }
93     return {f, cost};
94 }
95 };

```

### 13.3 上下界可行循环流

$\text{add}(u, v, lo, hi)$ ,  $0 \leq lo \leq hi$ ;  $\text{solve}()$  仅调用一次, 返回可行性, 仅成功后读取  $\text{used}(id)$ 。约束为每点流量守恒; 不是带源汇最大流。所有上下界及其总和须在  $\text{long long}$  范围内。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll = long long;
4     int n;
5     Dinic g;
6     vector<ll> balance, lower;
7     vector<int> ids;
8     bool solved = false, feasible = false;
9
10    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}
11
12    int add(int u, int v, ll lo, ll hi)
13    {
14        assert(!solved && 0 <= lo && lo <= hi);
15        int id = ids.size();
16        ids.push_back(g.add(u, v, hi - lo));
17        lower.push_back(lo);
18        balance[u] -= lo;
19        balance[v] += lo;
20        return id;
21    }
22
23    bool solve()
24    {
25        assert(!solved);
26        solved = true;
27        ll need = 0;
28        for (int u = 1; u <= n; u++)
29            if (balance[u] > 0)
30            {
31                g.add(n + 1, u, balance[u]);
32                need += balance[u];
33            }
34        else if (balance[u] < 0)
35            g.add(u, n + 2, -balance[u]);
36        return feasible = (g.flow(n + 1, n + 2) == need);
37    }
38
39    ll used(int id) const
40    {
41        assert(feasible);
42        return lower[id] + g.used(ids[id]);
43    }
44 };

```

# 第十四章 最大权闭合子图

## 14.1 最大权闭合子图与选点方案

`maximum_closure(weight, dependencies)` 返回 (最大权值, 选中点列表), `weight` 为 0-based `vector<long long>`, 图中点号 1..n. 依赖边 (u,v) 表示选 u 必须选 v, 允许环、自环与重边; 允许空选集, 因此答案不小于 0. 依赖 `flow.hpp` 中的 `Dinic`. 正权连源、负权连汇, 依赖边容量取正权总和加 1; 要求所有正权之和严格小于 `LLONG_MAX`. 负权先提升到 `int128` 再取负, 并截断到依赖边容量, 因此 `LLONG_MIN` 也可处理. 返回残量图源点可达的原顶点, 权值为正权总和减最小割; 超大负权边不会出现在最优割中, 截断不改变原问题最优值. 每次调用重建网络. 复杂度沿用 `Dinic`: 一般  $O(V^2E)$ , 空间  $O(V+E)$ , 增广保持递归. 返回一种最优闭合集, 不承诺最大或最小选点个数.

```
1 inline pair<long long, vector<int>>
2 maximum_closure(const vector<long long> &weight,
3                 const vector<pair<int, int>> &dependencies)
4 {
5     using I = __int128_t;
6     int n = weight.size(), s = n + 1, t = n + 2;
7     I positive = 0;
8     for (long long w : weight)
9         if (w > 0) positive += w;
10    assert(positive < LLONG_MAX);
11    long long inf = (long long)positive + 1;
12    Dinic graph(n + 2);
13    for (int u = 1; u <= n; u++)
14    {
15        long long w = weight[u - 1];
16        if (w > 0)
17            graph.add(s, u, w);
18        else if (w < 0)
19            graph.add(u, t, (long long)min(-I(w), I(inf)));
20    }
21    for (auto [u, v] : dependencies)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        graph.add(u, v, inf);
25    }
26    long long answer = (long long)positive - graph.flow(s, t);
27    vector<int> selected;
28    for (int u : graph.cut(s))
29        if (u <= n) selected.push_back(u);
30    return {answer, selected};
31 }
```

# 第十五章 图论

## 15.1 非负权最短路

`add(u,v,w)`、`run(s)`，`dis` 为距离，`pre` 为前驱，`path(t)` 返回点序列。不可达用 `LLONG_MAX`；要求所有有限最短距离严格小于该哨兵， $w \geq 0$ 。  $O((V+E)\log V)$ 。

```
1 struct Dijkstra
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Dijkstra(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v, ll w)
13    {
14        assert(w >= 0);
15        g[u].push_back({v, w});
16    }
17
18    void run(int s)
19    {
20        dis.assign(n + 1, inf);
21        pre.assign(n + 1, -1);
22        dis[s] = 0;
23        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
24        q.push({0, s});
25        while (!q.empty())
26        {
27            auto [d, u] = q.top();
28            q.pop();
29            if (d != dis[u]) continue;
30            for (auto [v, w] : g[u])
31                if (w < inf - d && d + w < dis[v])
32                {
33                    dis[v] = d + w;
34                    pre[v] = u;
35                    q.push({dis[v], v});
36                }
37        }
38    }
```

```
39
40     vector<int> path(int t) const
41     {
42         if (dis[t] == inf) return {};
43         vector<int> ans;
44         for (; t != -1; t = pre[t]) ans.push_back(t);
45         reverse(ans.begin(), ans.end());
46         return ans;
47     }
48 };
```

## 15.2 强连通分量

$\text{add}(u,v)$ 、 $\text{run}()$ 、 $\text{bel}[1..n]$  为分量编号， $\text{cnt}$  为数量；分量编号按缩点图拓扑序递增。递归 Kosaraju，按用户要求保留 DFS 写法； $O(V+E)$ ，同时存正反图。

```

1 struct SCC
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     SCC(int n) : n(n), g(n + 1), rg(n + 1) {}
8
9     void add(int u, int v)
10    {
11        g[u].push_back(v);
12        rg[v].push_back(u);
13    }
14
15    void dfs1(int u, vector<int> &vis, vector<int> &order)
16    {
17        vis[u] = 1;
18        for (int v : g[u])
19        {
20            if (!vis[v]) dfs1(v, vis, order);
21        }
22        order.push_back(u);
23    }
24
25    void dfs2(int u)
26    {
27        bel[u] = cnt;
28        for (int v : rg[u])
29        {
30            if (!bel[v]) dfs2(v);
31        }
32    }
33
34    // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
35    void run()
36    {
37        vector<int> vis(n + 1), order;
38        cnt = 0;
39        bel.assign(n + 1, 0);
40        for (int s = 1; s <= n; s++)
41        {
42            if (!vis[s]) dfs1(s, vis, order);
43        }
44        reverse(order.begin(), order.end());
45        for (int s : order)

```

```

46     {
47         if (!bel[s])
48         {
49             ++cnt;
50             dfs2(s);
51         }
52     }
53 }
54 };

```

## 15.3 2-SAT 与任意可行方案

add(x,a,y,b) 添加 (x=a) 或 (y=b), 变量 1..n; solve() 返回可满足性, 仅成功时读取 ans。依赖同文件 SCC;  $O(V+E)$ , 不保证字典序最小。

```

1 struct TwoSAT
2 {
3     int n;
4     SCC g;
5     vector<int> ans;
6
7     TwoSAT(int n) : n(n), g(2 * n) {}
8
9     int id(int x, bool value) const { return 2 * x - 1 + value; }
10
11     void add(int x, bool a, int y, bool b)
12     {
13         g.add(id(x, !a), id(y, b));
14         g.add(id(y, !b), id(x, a));
15     }
16
17     bool solve()
18     {
19         g.run();
20         ans.assign(n + 1, 0);
21         for (int x = 1; x <= n; x++)
22         {
23             if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
24             ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
25         }
26         return true;
27     }
28 };

```

## 15.4 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 1..n、1..m, 可有空侧及重边; add(u,v) 加边, solve() 返回最大匹配总数, l[u] 与 r[v] 为匹配端点, 0 表示未匹配。保留已有合法匹配, 可加边后再次 solve; 不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层, 递归 DFS 只沿最短层增广, 并使用当前弧。V=n+m, 单次 solve 最坏  $O((V+E)\sqrt{V})$ , 空间  $O(V+E)$ 。cover() 在 solve 后、下一次改图前调用, 返回 (左侧点列表, 右侧点列表), 为最小点覆盖; 其补集是最大独立集。覆盖与匹配方案均可直接输出。

```

1 struct BipartiteMatching

```

```

2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep, cur;
6
7     BipartiteMatching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
9     {
10    }
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= m);
15        g[u].push_back(v);
16    }
17
18    bool bfs()
19    {
20        queue<int> q;
21        fill(dep.begin(), dep.end(), n + 1);
22        fill(cur.begin(), cur.end(), 0);
23        for (int u = 1; u <= n; u++)
24        {
25            if (!l[u])
26            {
27                dep[u] = 0;
28                q.push(u);
29            }
30        }
31        while (!q.empty())
32        {
33            int u = q.front();
34            q.pop();
35            if (dep[u] >= dep[0]) continue;
36            for (int v : g[u])
37            {
38                int next = r[v];
39                if (dep[next] != n + 1) continue;
40                dep[next] = dep[u] + 1;
41                if (next) q.push(next);
42            }
43        }
44        return dep[0] != n + 1;
45    }
46
47    bool dfs(int u)
48    {
49        if (!u) return true;
50        for (int &i = cur[u]; i < (int)g[u].size(); i++)
51        {
52            int v = g[u][i];
53            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
54            {
55                l[u] = v;
56                r[v] = u;
57                return true;

```

```

58         }
59     }
60     dep[u] = n + 1;
61     return false;
62 }
63
64 // Retains an existing matching; edges may be added between calls.
65 int solve()
66 {
67     int answer = 0;
68     for (int u = 1; u <= n; u++) answer += (l[u] != 0);
69     while (bfs())
70     {
71         for (int u = 1; u <= n; u++)
72         {
73             if (!l[u] && dfs(u)) answer++;
74         }
75     }
76     return answer;
77 }
78
79 // Call after solve(), before adding more edges. Returns left/right vertex IDs.
80 pair<vector<int>, vector<int>> cover() const
81 {
82     vector<int> left, right;
83     for (int u = 1; u <= n; u++)
84     {
85         if (dep[u] == n + 1) left.push_back(u);
86     }
87     for (int v = 1; v <= m; v++)
88     {
89         if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
90     }
91     return {left, right};
92 }
93 };

```

## 15.5 割点与桥

无向多重图，点号  $1..n$ ；`add/Insert(u,v)` 返回逻辑无向边的 0-based 编号，自环和重边均可输入。`run/Run` 后 `cut[u]` 为割点标记，`bridge[id]` 为桥标记，`components` 为原图连通块数。`delta[u]` 是独立删除顶点  $u$  及其关联边后增加的连通块数，剩余块数为 `components+delta[u]`；孤立点 `delta=-1`，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 `low[child]>=dfn[u]` 的子树数。`n=1` 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度  $O(n+m)$ ，递归 DFS；新增边后需重新 `run`。每个删点结果都针对原图，不表示逐次删除。

```

1 struct Lowlink
2 {
3     int n, timer = 0, components = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge, delta;
6     int edges = 0;
7
8     Lowlink(int n) : n(n), g(n + 1) {}
9
10    int add(int u, int v)
11    {
12        g[u].push_back({v, edges});
13        g[v].push_back({u, edges});
14        return edges++;
15    }

```

```

17 void dfs(int u, int pe)
18 {
19     dfn[u] = low[u] = ++timer;
20     int children = 0;
21     for (auto [v, id] : g[u])
22     {
23         if (id == pe) continue;
24         if (!dfn[v])
25         {
26             ++children;
27             dfs(v, id);
28             low[u] = min(low[u], low[v]);
29             if (low[v] > dfn[u]) bridge[id] = 1;
30             if (pe != -1 && low[v] >= dfn[u])
31             {
32                 cut[u] = 1;
33                 delta[u]++;
34             }
35         }
36         else
37             low[u] = min(low[u], dfn[v]);
38     }
39     if (pe == -1 && children > 1) cut[u] = 1;
40     if (pe == -1) delta[u] = children - 1;
41 }
42
43 void run()
44 {
45     timer = 0;
46     components = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     cut = dfn;
50     delta = dfn;
51     bridge.assign(edges, 0);
52     for (int u = 1; u <= n; u++)
53         if (!dfn[u])
54         {
55             components++;
56             dfs(u, -1);
57         }
58 }
59 };

```

## 第十六章 开放时间最短路

## 16.1 顶点开放时间的单位边最短路

`release_bfs(open,source,neighbors)` 返回各点最早到达时间，不可达为 -1；点号  $0..n-1$ ,  $n>0$ , `open[v]` 为非负整数，源点初始时间为 `open[source]`。允许原地等待，每条边耗时恰为 1, `neighbors(u,visit)` 枚举所有出邻点并调用 `visit(v)`，可直接枚举网格而不建边。令  $R=\max(\text{open})$ ，要求  $R+n<\text{INT\_MAX}$  且内存可承受；时间  $O(n+m+R)$ ，空间  $O(n+R)$ 。第一次发现即可定距，只适用于单位边和固定开放时间；不能用于一般带权边、关闭时间或周期可用边。

```

1 template <class Neighbors>
2 vector<int> release_bfs(const vector<int> &open, int source, Neighbors neighbors)
3 {
4     int n = open.size();
5     assert(0 <= source && source < n);
6     int last = *max_element(open.begin(), open.end());
7     assert(last >= 0 && last < INT_MAX - n);
8     vector<vector<int>> pending(last + 1);
9     vector<int> dist(n, -1), queue;
10    queue.reserve(n);
11    dist[source] = open[source];
12    queue.push_back(source);
13    int head = 0, time = 0, waiting = 0;
14    while (head < int(queue.size()) || waiting)
15    {
16        int limit = head < int(queue.size()) ? dist[queue[head]] : INT_MAX;
17        while (time <= last && time <= limit && pending[time].empty()) time++;
18        int u;
19        if (time <= last && time <= limit)
20        {
21            u = pending[time].back();
22            pending[time].pop_back();
23            waiting--;
24        }
25        else
26            u = queue[head++];
27        neighbors(u,
28            [&](int v)
29            {
30                if (dist[v] != -1) return;
31                dist[v] = max(dist[u] + 1, open[v]);
32                if (dist[v] == dist[u] + 1)
33                    queue.push_back(v);
34                else
35                {
36                    pending[open[v]].push_back(v);
37                    waiting++;
38                }
39            });
40    }
41    return dist;
42 }

```

# 第十七章 字典序最小 2-SAT

## 17.1 递归染色与回滚求最小赋值

构造 LexTwoSAT( $n$ ), 变量号  $1..n$ 。add( $x,a,y,b$ ) 加入子句  $(x=a) \text{ OR } (y=b)$ , force( $x,v$ ) 强制变量值; solve() 成功后 answer[ $1..n$ ] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空,  $n=0$  时可满足, answer[0] 始终仅为占位。每个未确定变量先尝试 0, 递归传播所有蕴含; 冲突时仅撤销本次新增标记, 再试 1。history 是撤销记录, 不替代递归调用栈。可重复求解, 追加子句会清空旧答案; marked/history 仅为内部工作区, 失败时不可当作方案。内部  $\text{id}(x,v)=2(x-1)+v$ , 从 0 开始, 可用异或 1 取反; 不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏  $O(n(n+m))$  时间、 $O(n+m)$  空间, 递归栈  $O(n)$ ; 100000 变量测试是特定蕴含链, 不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍, 在线待补。

```
1 struct LexTwoSAT
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> marked, history, answer;
6
7     LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n) { assert(n >= 0); }
8
9     int id(int x, bool value) const
10    {
11        assert(1 <= x && x <= n);
12        return 2 * (x - 1) + value;
13    }
14
15    // (x == a) OR (y == b). Internal literal IDs are 0-based.
16    void add(int x, bool a, int y, bool b)
17    {
18        int u = id(x, a), v = id(y, b);
19        g[u ^ 1].push_back(v);
20        g[v ^ 1].push_back(u);
21        answer.clear();
22    }
23
24    void force(int x, bool value) { add(x, value, x, value); }
```

```
26 bool paint(int u)
27 {
28     if (marked[u ^ 1]) return false;
29     if (marked[u]) return true;
30     marked[u] = 1;
31     history.push_back(u);
32     for (int v : g[u])
33         if (!paint(v)) return false;
34     return true;
35 }
36
37 bool solve()
38 {
39     fill(marked.begin(), marked.end(), 0);
40     history.clear();
41     answer.clear();
42     for (int x = 1; x <= n; x++)
43     {
44         int u = id(x, false);
45         if (marked[u] || marked[u ^ 1]) continue;
46         int checkpoint = history.size();
47         if (!paint(u))
48         {
49             while ((int)history.size() > checkpoint)
50             {
51                 marked[history.back()] = 0;
52                 history.pop_back();
53             }
54             if (!paint(u ^ 1)) return false;
55         }
56     }
57     answer.assign(n + 1, 0);
58     for (int x = 1; x <= n; x++) answer[x] = marked[id(x, true)];
59     return true;
60 }
61 };
```

# 第十八章 有向欧拉路

## 18.1 有向图字典序最小欧拉路

构造 `DirectedEuler(n)`,  $n \geq 1$ , 点号  $1..n$ ; `add(u,v)` 返回从 0 开始的逻辑边编号, 支持自环与重边。`run()` 返回是否存在欧拉路; 参数 `start` 默认 0, `lexicographic` 默认 `true`。`start=0` 自动选择最小可行起点, 否则固定起点; 孤立点不影响有边部分的连通性。成功后 `vertices` 长度为  $m+1$ , `edge_ids` 长度为  $m$ , 第  $i$  条原边从 `vertices[i]` 指向 `vertices[i+1]`; 失败清空两数组。无边时返回单点路径, 自动起点为 1。可重复 `run`, `add` 会清空旧方案。默认求顶点序列字典序最小方案; 关闭排序后只保证任意方案, 邻接顺序可能保留上次排序。开启排序总时间  $O(n+m \log(m+1))$ , 否则  $O(n+m)$ , 空间  $O(n+m)$ 。DFS 递归深度可能达到  $m+1$ , 不是  $n$ 。度数检查后, 以消费边数判定是否遍历全部非孤立部分, 逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号, P7771 双风格完整程序本地验证, 在线待补。

```
1 struct DirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> in, out, next, vertices, edge_ids;
7
8     DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
9     {
10         assert(n > 0);
11     }
12
13     int add(int u, int v)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         int id = edges.size();
17         edges.push_back({u, v});
18         g[u].push_back({v, id});
19         out[u]++;
20         in[v]++;
21         vertices.clear();
22         edge_ids.clear();
23         return id;
24     }
25
26     void dfs(int u, int incoming)
27     {
28         while (next[u] < (int)g[u].size())
29         {
30             auto [v, id] = g[u][next[u]++];
31             dfs(v, id);
32         }
33         vertices.push_back(u);
34         if (incoming != -1) edge_ids.push_back(incoming);
35     }
```

```

37 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
38 bool run(int start = 0, bool lexicographic = true)
39 {
40     assert(0 <= start && start <= n);
41     vertices.clear();
42     edge_ids.clear();
43     int source = 0, sink = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         int delta = out[u] - in[u];
47         if (delta == 1 && !source)
48             source = u;
49         else if (delta == -1 && !sink)
50             sink = u;
51         else if (delta != 0)
52             return false;
53     }
54     if (bool(source) != bool(sink)) return false;
55     if (source)
56     {
57         if (start && start != source) return false;
58         start = source;
59     }
60     if (!start)
61     {
62         start = 1;
63         for (int u = 1; u <= n; u++)
64             if (out[u])
65             {
66                 start = u;
67                 break;
68             }
69     }
70     if (lexicographic)
71         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
72     fill(next.begin(), next.end(), 0);
73     dfs(start, -1);
74     if (edge_ids.size() != edges.size())
75     {
76         vertices.clear();
77         edge_ids.clear();
78         return false;
79     }
80     reverse(vertices.begin(), vertices.end());
81     reverse(edge_ids.begin(), edge_ids.end());
82     return true;
83 }
84 };

```

# 第十九章 字典序单词链

## 19.1 字典序最小单词链

`word_chain(words)` 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次; 相邻单词首尾字母相接, 单词序列字典序最小。无解返回 `nullopt`, 空输入返回有值的空序列。单词必须非空且仅含小写 a-z, 重复词视为不同编号。先对原编号按单词排序, 按此顺序加边, 再调用 `DirectedEuler.run(0,false)`; 不可让底层重新按终点排序。欧拉回路从最小出现首字母开始, 非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作, L 为总字符数; 空间 O(m), 递归栈 O(m)。依赖 `directed_euler.hpp`。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证, 在线待补。

```
1 // Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
2 inline optional<vector<int>> word_chain(const vector<string> &words)
3 {
4     int n = words.size();
5     vector<int> order(n);
6     iota(order.begin(), order.end(), 0);
7     for (const auto &word : words)
8     {
9         assert(!word.empty());
10        for (char c : word) assert('a' <= c && c <= 'z');
11    }
12    sort(order.begin(),
13         order.end(),
14         [&](int a, int b)
15         {
16             if (words[a] != words[b]) return words[a] < words[b];
17             return a < b;
18         });
19    DirectedEuler graph(26);
20    for (int id : order)
21        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
22    // Preserve word order in each adjacency list; do not sort by destination.
23    if (!graph.run(0, false)) return nullopt;
24    vector<int> result;
25    for (int id : graph.edge_ids) result.push_back(order[id]);
26    return result;
27 }
```

## 第二十章 无向欧拉路

### 20.1 无向图欧拉路与边方向

构造 `UndirectedEuler(n)`,  $n \geq 1$ , 点号  $1..n$ ; `add(u,v)` 返回从 0 开始的逻辑边号, 自环和重边均保留。`run()` 返回是否存在欧拉路, 参数 `start` 默认 0 (自动), `lexicographic` 默认 `true`。自动选最小可行起点; 指定起点须满足奇度条件且位于有边部分。成功得到  $m+1$  个 `vertices` 和  $m$  个 `edge_ids`, 失败清空两数组。第  $i$  条边方向是 `vertices[i]` 到 `vertices[i+1]`, 与 `edges[edge_ids[i]]` 比较可判断是否反向, 自环任选方向。`used` 按逻辑边号标记, 使两条半边共同只消费一次; 自环贡献度数 2。孤立点不影响有边部分; 无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案; 关闭排序只保证任意方案, 可能保留上次排序。可重复 `run`, 加边清空旧方案。排序开启时  $O(n+m \log(m+1))$ , 否则  $O(n+m)$ , 空间  $O(n+m)$ , DFS 递归深度可能达到  $m+1$ 。全小图走法、原边号与方向独立验证, P2731 和 SGU101 完整程序本地验证; 在线待补。

```
1 struct UndirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> next, used, vertices, edge_ids;
7
8     UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }
9
10    int add(int u, int v)
11    {
12        assert(1 <= u && u <= n && 1 <= v && v <= n);
13        int id = edges.size();
14        edges.push_back({u, v});
15        g[u].push_back({v, id});
16        g[v].push_back({u, id});
17        vertices.clear();
18        edge_ids.clear();
19        return id;
20    }
21
22    void dfs(int u, int incoming)
23    {
24        while (next[u] < (int)g[u].size())
25        {
26            auto [v, id] = g[u][next[u]++];
27            if (used[id]) continue;
28            used[id] = 1;
29            dfs(v, id);
30        }
31        vertices.push_back(u);
32        if (incoming != -1) edge_ids.push_back(incoming);
33    }
```

```

35 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
36 bool run(int start = 0, bool lexicographic = true)
37 {
38     assert(0 <= start && start <= n);
39     vertices.clear();
40     edge_ids.clear();
41     vector<int> odd;
42     for (int u = 1; u <= n; u++)
43         if (g[u].size() % 2) odd.push_back(u);
44     if (!odd.empty() && odd.size() != 2) return false;
45     if (!odd.empty())
46     {
47         if (start && start != odd[0] && start != odd[1]) return false;
48         if (!start) start = odd[0];
49     }
50     if (!start)
51     {
52         start = 1;
53         for (int u = 1; u <= n; u++)
54             if (!g[u].empty())
55             {
56                 start = u;
57                 break;
58             }
59     }
60     if (lexicographic)
61         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
62     fill(next.begin(), next.end(), 0);
63     used.assign(edges.size(), 0);
64     dfs(start, -1);
65     if (edge_ids.size() != edges.size())
66     {
67         vertices.clear();
68         edge_ids.clear();
69         return false;
70     }
71     reverse(vertices.begin(), vertices.end());
72     reverse(edge_ids.begin(), edge_ids.end());
73     return true;
74 }
75 };

```

## 第二十一章 混合图欧拉定向

### 21.1 欧拉回路与指定端点定向

`mixed_euler_orientation(n, edges, start, finish)` 返回可空的定向边序列。每条输入为  $\{u, v, \text{type}\}$ ,  $\text{type}=1$  固定  $u$  到  $v$ ,  $\text{type}=0$  可反转; 点号  $1..n$ ,  $n \geq 1$ 。省略两个端点或均传 0 时请求任意起点的欧拉回路; 两个正端点请求指定起终点的欧拉路径, 相等时为指定起点回路。无解返回 `nullopt`, 有解返回与原边顺序对应的端点对; 无边图可返回有值空序列。忽略无关孤立点, 但正端点须在有边部分。先任意定向可反转边, 扣除目标出入度差; 每条反转将两端差值分别改变  $-2$  和  $+2$ , 奇数差无解, 否则用容量 1 的反转边流调整。检查底层无向连通性后, 满流即得到所需方向。最大流图  $n+2$  点、至多  $m+n$  条正向边, 总时间可按 Dinic 的  $O((n+2)^2(m+n))$  上界计, 空间  $O(n+m)$ 。依赖 `flow.hpp`; 返回边交给 `DirectedEuler` 可生成连续走法, 指定端点时保持指定起点。定向不保证字典序最小。

```
1 // Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
2 // start=finish=0 requests a circuit; positive endpoints request a specified trail.
3 inline optional<vector<pair<int, int>>> mixed_euler_orientation(
4     int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
5 {
6     assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
7     assert(bool(start) == bool(finish));
8     vector<int> parent(n + 1), id;
9     vector<long long> degree(n + 1), balance(n + 1);
10    iota(parent.begin(), parent.end(), 0);
11    auto find = [&](auto &&self, int u) -> int
12    {
13        if (parent[u] != u) parent[u] = self(self, parent[u]);
14        return parent[u];
15    };
16    Dinic flow(n + 2);
17    vector<pair<int, int>> direction;
18    for (auto [u, v, type] : edges)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n);
21        assert(type == 0 || type == 1);
22        parent[find(find, u)] = find(find, v);
23        degree[u]++;
24        degree[v]++;
25        balance[u]++;
26        balance[v]--;
27        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
28        direction.push_back({u, v});
29    }
```

```

30     if (!edges.empty())
31     {
32         int root = find(find, edges[0][0]);
33         for (int u = 1; u <= n; u++)
34             if (degree[u] && find(find, u) != root) return nullopt;
35         if (start && (!degree[start] || !degree[finish])) return nullopt;
36     }
37     if (start)
38     {
39         balance[start]--;
40         balance[finish]++;
41     }
42     int source = n + 1, sink = n + 2;
43     long long need = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         if (balance[u] % 2) return nullopt;
47         if (balance[u] > 0)
48         {
49             flow.add(source, u, balance[u] / 2);
50             need += balance[u] / 2;
51         }
52         else if (balance[u] < 0)
53             flow.add(u, sink, -balance[u] / 2);
54     }
55     if (flow.flow(source, sink) != need) return nullopt;
56     for (int i = 0; i < (int)edges.size(); i++)
57         if (id[i] != -1 && flow.used(id[i]))
58             swap(direction[i].first, direction[i].second);
59     return direction;
60 }

```

## 21.2 任意起终点的混合欧拉路径

`mixed_euler_trail(n, edges)` 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 `DirectedEuler` 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```

1 // Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
2 // orders.
3 inline optional<vector<pair<int, int>>>
4 mixed_euler_trail(int n, const vector<array<int, 3>> &edges)
5 {
6     assert(n > 0);
7     vector<long long> degree(n + 1);
8     vector<int> odd;
9     for (auto [u, v, type] : edges)
10     {
11         assert(1 <= u && u <= n && 1 <= v && v <= n);
12         degree[u]++;
13         degree[v]++;
14     }
15     for (int u = 1; u <= n; u++)
16         if (degree[u] % 2) odd.push_back(u);
17     if (odd.empty()) return mixed_euler_orientation(n, edges);
18     if (odd.size() != 2) return nullopt;
19     auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
20     if (answer) return answer;
21     return mixed_euler_orientation(n, edges, odd[1], odd[0]);
22 }
```

## 第二十二章 图上异或行走

### 22.1 无向图最大异或行走

XorWalk(n) 建图，点号 1..n，边权为 unsigned long long，支持完整 64 位。add(u,v,w) 加无向边，允许重边、自环和零权；init(n) 清空图。build(root=1) 用递归 DFS 预处理 root 的连通块，query(u,v) 返回该块内 u 到 v 的最大边权异或和；任一点未被本次 build 覆盖则返回 nullopt，即使这两点在另一连通块相连。加入新边后必须重新 build；可更换根重复构建。空图可初始化，但不能 build。这里的行走允许重复点、重复边；不是最大异或简单路径。u=v 也可能通过绕环得到正值。DFS 距离 dist 给出一条树路径的异或值，每条已访问边形成 dist[u] xor dist[v] xor w 的闭合行走值，加入线性基；两点答案为 cycles.query(dist[u] xor dist[v])。build 后 vis、dist 可只读；cycles 表示环异或空间，不保留具体环或行走方案，不能将其非空子集标志解释为行走非空性。建图 O(n+m)，build 为 O(n+64m)，每次 query 为 O(64)，动态空间 O(n+m+64)。

```
1 struct XorWalk
2 {
3     using U = unsigned long long;
4     int n;
5     bool built = false;
6     vector<vector<pair<int, U>>> g;
7     vector<int> vis;
8     vector<U> dist;
9     XorBasis cycles;
10
11     XorWalk(int n) { init(n); }
12
13     void init(int size)
14     {
15         assert(size >= 0);
16         n = size;
17         g.clear();
18         g.resize(size_t(n) + 1);
19         vis.assign(size_t(n) + 1, 0);
20         dist.assign(size_t(n) + 1, 0);
21         cycles.init();
22         built = false;
23     }
```

```

25 void add(int u, int v, U w)
26 {
27     assert(1 <= u && u <= n && 1 <= v && v <= n);
28     g[u].push_back({v, w});
29     g[v].push_back({u, w});
30     built = false;
31 }
32
33 void dfs(int u)
34 {
35     vis[u] = 1;
36     for (auto [v, w] : g[u])
37     {
38         if (!vis[v])
39         {
40             dist[v] = dist[u] ^ w;
41             dfs(v);
42         }
43         else
44             cycles.insert(dist[u] ^ dist[v] ^ w);
45     }
46 }
47
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     fill(vis.begin(), vis.end(), 0);
52     fill(dist.begin(), dist.end(), 0);
53     cycles.init();
54     dfs(root);
55     built = true;
56 }
57
58 optional<U> query(int u, int v) const
59 {
60     assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
61     if (!vis[u] || !vis[v]) return nullopt;
62     return cycles.query(dist[u] ^ dist[v]);
63 }
64 };

```

## 第二十三章 树上算法

### 23.1 重链剖分与 LCA

add(u,v) 建无向树; build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为  $O(\log n)$ , path(u,v,work,edge) 分解路径; work(l,r) 仅适用于可交换聚合, 边权模式排除 LCA 位置。输入必须是连通无环树, 建树  $O(n)$ , 两遍递归 DFS; 支持改变根后重复 build, dfn 会重新编号。

```
1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     void dfs1(int u, int p)
27     {
28         fa[u] = p;
29         siz[u] = 1;
30         son[u] = 0;
31         for (int v : g[u])
32         {
33             if (v == p) continue;
34             dep[v] = dep[u] + 1;
35             dfs1(v, u);
36             siz[u] += siz[v];
37             if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
38         }
39     }
40
41     void dfs2(int u, int t)
42     {
43         top[u] = t;
44         dfn[u] = ++timer;
```

```

45     rk[timer] = u;
46     if (son[u]) dfs2(son[u], t);
47     for (int v : g[u])
48     {
49         if (v != fa[u] && v != son[u]) dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     dfs1(root, root);
60     dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int lca(int u, int v) const
65 {
66     while (top[u] != top[v])
67     {
68         if (dep[top[u]] < dep[top[v]]) swap(u, v);
69         u = fa[top[u]];
70     }
71     return dep[u] < dep[v] ? u : v;
72 }
73
74 // Commutative operations only; edge=true excludes the LCA's position.
75 template <class F> void path(int u, int v, F work, bool edge = false) const
76 {
77     while (top[u] != top[v])
78     {
79         if (dep[top[u]] < dep[top[v]]) swap(u, v);
80         work(dfn[top[u]], dfn[u]);
81         u = fa[top[u]];
82     }
83     if (dep[u] > dep[v]) swap(u, v);
84     if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
85 }
86 };

```

## 第二十四章 离线最近公共祖先

### 24.1 离线 Tarjan LCA

构造 `OfflineLCA(n)`, `add(u,v)` 加无向树边, `add_query(u,v)` 登记查询并返回从 0 开始的编号, `run(root=1)` 后读取 `answer[id]`。输入必须是非空树, 点号  $1..n$ ; 允许同点、重复查询及零个查询。`run` 可重复调用或换根, 不清空查询; 追加查询后需重新 `run`, 旧答案在重算前不可按新根使用。DFS 与并查集查找均递归。并查集按大小合并并压缩路径, `ancestor[find(u)]` 保存已合并部分对应的当前树祖先; 不能直接拿并查集代表当 LCA。每个子树完成后合并到父点并更新 `ancestor`; 查询另一端已完成时, 从其集合读取祖先。总时间  $O((n+q) \alpha(n))$ , 空间  $O(n+q)$ 。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证, 在线待补。

```
1 struct OfflineLCA
2 {
3     int n;
4     vector<vector<int>>> g;
5     vector<vector<pair<int, int>>>> queries;
6     vector<int> parent, size, ancestor, done, answer;
7
8     OfflineLCA(int n)
9         : n(n),
10          g(n + 1),
11          queries(n + 1),
12          parent(n + 1),
13          size(n + 1),
14          ancestor(n + 1),
15          done(n + 1)
16     {
17     }
18
19     void add(int u, int v)
20     {
21         assert(1 <= u && u <= n && 1 <= v && v <= n);
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int add_query(int u, int v)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n);
29         int id = answer.size();
30         answer.push_back(0);
31         queries[u].push_back({v, id});
32         queries[v].push_back({u, id});
33         return id;
34     }
```

```

36     int find(int u)
37     {
38         if (parent[u] != u) parent[u] = find(parent[u]);
39         return parent[u];
40     }
41
42     void merge(int u, int v)
43     {
44         u = find(u);
45         v = find(v);
46         if (size[u] < size[v]) swap(u, v);
47         parent[v] = u;
48         size[u] += size[v];
49     }
50
51     void dfs(int u, int p)
52     {
53         for (int v : g[u])
54             if (v != p)
55             {
56                 dfs(v, u);
57                 merge(u, v);
58                 ancestor[find(u)] = u;
59             }
60         done[u] = 1;
61         for (auto [v, id] : queries[u])
62             if (done[v]) answer[id] = ancestor[find(v)];
63     }
64
65     // Input must be a nonempty tree. Query IDs are 0-based.
66     void run(int root = 1)
67     {
68         assert(1 <= root && root <= n);
69         iota(parent.begin(), parent.end(), 0);
70         iota(ancestor.begin(), ancestor.end(), 0);
71         fill(size.begin(), size.end(), 1);
72         fill(done.begin(), done.end(), 0);
73         fill(answer.begin(), answer.end(), 0);
74         dfs(root, 0);
75     }
76 };

```

# 第二十五章 在线最近公共祖先

## 25.1 欧拉序与 ST 表 LCA

构造 EulerLCA(n), add(u,v,w=1) 加非负权无向树边, build(root=1) 预处理; lca(u,v) 返回最近公共祖先, distance(u,v) 仍返回无权路径边数; weighted\_distance(u,v) 返回 long long 带权距离。边权非负, 所有树上路径的权和须  $\leq \text{LLONG\_MAX}$ 。输入必须为非空树, 点号 1..n; 允许同点查询和重复 build 换根。只可在 build 后查询, 加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点, euler 长度为  $2n-1$ , first[u] 是从 0 开始的首次出现位置; depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA, 用 ST 表保存该顶点。预处理  $O(n \log n)$  时间和空间, 每次查询  $O(1)$ 。ST 内部下标从 0 开始; first、depth、euler、root\_dist 可直接读取; root\_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证; 上述无权题在线待补; QOJ8235 的带权应用已双 AC。

```
1 struct EulerLCA
2 {
3     using ll = long long;
4     int n;
5     bool ready = false;
6     vector<vector<int>> g, st;
7     vector<vector<ll>> cost;
8     vector<ll> root_dist;
9     vector<int> first, depth, euler, lg;
10
11     EulerLCA(int n)
12         : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
13     {
14     }
15
16     void add(int u, int v, ll w = 1)
17     {
18         assert(1 <= u && u <= n && 1 <= v && v <= n);
19         assert(w >= 0);
20         g[u].push_back(v);
21         g[v].push_back(u);
22         cost[u].push_back(w);
23         cost[v].push_back(w);
24         ready = false;
25     }
26
27     int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }
```

递归欧拉序与 RMQ (接上页同一结构体):

```

29 void dfs(int u, int parent)
30 {
31     first[u] = euler.size();
32     euler.push_back(u);
33     for (int i = 0; i < int(g[u].size()); i++)
34     {
35         int v = g[u][i];
36         if (v != parent)
37         {
38             depth[v] = depth[u] + 1;
39             root_dist[v] = root_dist[u] + cost[u][i];
40             dfs(v, u);
41             euler.push_back(u);
42         }
43     }
44 }
45
46 // Input must be a nonempty tree. Rebuilding may change the root.
47 void build(int root = 1)
48 {
49     assert(1 <= root && root <= n);
50     euler.clear();
51     depth[root] = 0;
52     root_dist[root] = 0;
53     dfs(root, 0);
54     int m = euler.size();
55     lg.assign(m + 1, 0);
56     for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
57     st.assign(lg[m] + 1, {});
58     st[0] = euler;
59     for (int k = 1; k <= lg[m]; k++)
60     {
61         st[k].resize(m - (1 << k) + 1);
62         for (int i = 0; i + (1 << k) <= m; i++)
63             st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
64     }
65     ready = true;
66 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```
68     int lca(int u, int v) const
69     {
70         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
71         int l = first[u], r = first[v];
72         if (l > r) swap(l, r);
73         int k = lg[r - l + 1];
74         return shallower(st[k][l], st[k][r - (1 << k) + 1]);
75     }
76
77     int distance(int u, int v) const
78     {
79         int z = lca(u, v);
80         return depth[u] + depth[v] - 2 * depth[z];
81     }
82
83     ll weighted_distance(int u, int v) const
84     {
85         int z = lca(u, v);
86         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
87     }
88 };
```

## 第二十六章 树上点集摘要

### 26.1 树上点集直径摘要

默认空点集,  $a=b=-1$ 、 $length=0$ 。insert( $v,distance$ ) 插入顶点, 重复插入合法; merge( $other,distance$ ) 合并两个点集摘要, 允许自合并; farthest( $v,distance$ ) 返回 optional<pair<int,long long>> (最远顶点、距离), 空集返回 nullopt, 距离并列时可返回任意最远点。distance( $u,v$ ) 回调必须精确返回同一棵固定非负权树的距离, 点号为非负整数; 所有距离  $\leq LLONG\_MAX$ 。每次插入或最远查询最多两次距离调用, 合并最多四次, 摘要只占  $O(1)$  空间。保存  $a$ 、 $b$  为点集中某对直径端点, 不保留全部点; 字段只读, 不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合, 重置用默认构造。

```
1 struct TreeDiameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if (a == -1)
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if (x >= y && x > length)
19         {
20             b = v;
21             length = x;
22         }
23         else if (y > length)
24         {
25             a = v;
26             length = y;
27         }
28     }
```

合并与最远点查询（接上页同一结构体）：

```
30     template <class Distance> void merge(TreeDiameter other, const Distance &distance)
31     {
32         if (other.a == -1) return;
33         insert(other.a, distance);
34         insert(other.b, distance);
35     }
36
37     template <class Distance>
38     optional<pair<int, ll>> farthest(int v, const Distance &distance) const
39     {
40         if (a == -1) return nullopt;
41         ll x = distance(a, v), y = distance(b, v);
42         return x >= y ? make_pair(a, x) : make_pair(b, y);
43     }
44 };
```

## 第二十七章 倍增祖先与路径最值

### 27.1 倍增 LCA、祖先与最大边权

构造 `LiftingLCA(n)`, `add(u,v,w=1)` 加边, `build(root=1)` 预处理非空无向树, 点号 `1..n`。 `jump(u,k)` 返回 `k` 级祖先, `k` 非负, 超过根返回 `0`; `lca(u,v)` 求最近公共祖先; `distance` 返回路径边数, 不是权值和; `max_edge` 返回路径最大边权, 同点返回 `nullopt`, 其他情况返回有值的 `optional<long long>`。支持负权和 `LLONG_MIN`, `LOW` 只是最大值合并的单位元, 不表示空路径。DFS 递归, 根深度为 `0`; `up[k][u]` 与 `mx[k][u]` 是 `2` 的 `k` 次方长度向上段的祖先与最大边权, `0` 号节点为哨兵。可重复 `build` 换根; 修改图后必须重建。预处理  $O(n \log n)$  时间与空间, 各查询  $O(\log n)$ 。不需要边权最值时, 可删去 `mx` 和 `max_edge` 相关部分。全小树点对/根、极值权重及 `50` 万点负权链本地验证; P3379、POJ1330 驱动只验证 LCA, 最大边权仍是本地证据。

```
1 struct LiftingLCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     int n, levels;
6     bool ready = false;
7     vector<vector<pair<int, ll>>> g;
8     vector<vector<int>> up;
9     vector<vector<ll>> mx;
10    vector<int> depth;
11
12    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
13    {
14        assert(n > 0);
15        levels = 1;
16        while ((1LL << levels) <= n) levels++;
17        up.assign(levels, vector<int>(n + 1));
18        mx.assign(levels, vector<ll>(n + 1, LOW));
19    }
20
21    void add(int u, int v, ll w = 1)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26        ready = false;
27    }
```

```

29 void dfs(int u, int parent, ll weight)
30 {
31     up[0][u] = parent;
32     mx[0][u] = weight;
33     for (int k = 1; k < levels; k++)
34     {
35         int p = up[k - 1][u];
36         up[k][u] = up[k - 1][p];
37         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
38     }
39     for (auto [v, w] : g[u])
40         if (v != parent)
41         {
42             depth[v] = depth[u] + 1;
43             dfs(v, u, w);
44         }
45 }
46
47 // Nonempty tree only. Node 0 means no ancestor.
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     depth[root] = 0;
52     dfs(root, 0, LOW);
53     ready = true;
54 }
55
56 int jump(int u, long long steps) const
57 {
58     assert(ready && 1 <= u && u <= n && steps >= 0);
59     if (steps > depth[u]) return 0;
60     for (int k = 0; k < levels; k++)
61         if (steps >> k & 1) u = up[k][u];
62     return u;
63 }

```

```

65  int lca(int u, int v) const
66  {
67      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68      if (depth[u] < depth[v]) swap(u, v);
69      u = jump(u, depth[u] - depth[v]);
70      if (u == v) return u;
71      for (int k = levels - 1; k >= 0; k--)
72          if (up[k][u] != up[k][v])
73              {
74                  u = up[k][u];
75                  v = up[k][v];
76              }
77      return up[0][u];
78  }
79
80  int distance(int u, int v) const
81  {
82      return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83  }
84
85  optional<ll> max_edge(int u, int v) const
86  {
87      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88      if (u == v) return nullopt;
89      if (depth[u] < depth[v]) swap(u, v);
90      ll answer = LOW;
91      int delta = depth[u] - depth[v];
92      for (int k = 0; k < levels; k++)
93          if (delta >> k & 1)
94              {
95                  answer = max(answer, mx[k][u]);
96                  u = up[k][u];
97              }
98      if (u == v) return answer;
99      for (int k = levels - 1; k >= 0; k--)
100          if (up[k][u] != up[k][v])
101              {
102                  answer = max(answer, max(mx[k][u], mx[k][v]));
103                  u = up[k][u];
104                  v = up[k][v];
105              }
106      return max(answer, max(mx[0][u], mx[0][v]));
107  }
108 };

```

# 第二十八章 树上路径交

## 28.1 两条树路径的交集

`path_intersection(a,b,c,d,depth,lca)` 返回路径 `a-b` 与 `c-d` 的公共路径，字段 `u`、`v` 是交集端点，`vertices` 是公共顶点数；空交返回 `{0,0,0}`，单点交的 `u=v`。点号从 1 开始，`depth` 数组与 `lca` 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能带带权根距离。支持数组或 `vector` 深度表，回调接收两个点并返回 `LCA`。将 `c`、`d` 分别投影到 `a-b` 得 `p`、`q`；投影是 `lca(a,b)`、`lca(a,x)`、`lca(b,x)` 中最深者。若 `p≠q`，`c-d` 必经过 `p-q`；若 `p=q`，检验 `p` 是否在 `c-d` 上即可区分单点交与空交。不依赖顶点编号大小。常数次 `LCA` 调用、`O(1)` 额外空间；使用 `EulerLCA` 时为 `O(1)`，`HLD/倍增` 时为 `O(log n)`。公共边数为 `max(0,vertices-1)`。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
1 struct PathIntersection
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 PathIntersection
9 path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for (int v : {lca(a, x), lca(b, x)})
16             if (depth[v] > depth[p]) p = v;
17         return p;
18     };
19     auto distance = [&](int u, int v)
20     {
21         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
22     };
23     int p = project(c), q = project(d);
24     if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
25     return {p, q, int(distance(p, q) + 1)};
26 }
```

## 第二十九章 点分治

### 29.1 点分治：树上距离点对计数

CentroidPairs( $n$ ), 点号  $1..n$ ; add( $u,v,w$ ) 建树, 允许森林但禁止环、重边、自环, 边权非负; 所有简单路径长度须能用 long long 表示。build() 预处理, 可重复调用; 改变边后须重建。count\_leq( $k$ ) 统计距离  $\leq k$  的无序不同顶点对, count\_exact( $k$ ) 统计距离恰为  $k$  的点对; 负  $k$  返回 0, 零距离只统计不同顶点, 跨连通块不配对。parent[ $c$ ] 是点分树父节点, 根为 0。all[ $c$ ] 为该层分治块到重心的距离, 含重心距离 0; part[ $c$ ] 为去掉重心后各子块的距离。预处理  $O(n \log^2 n)$ , 空间  $O(n \log n)$ , 每次计数  $O(n \log n)$ ; 固定树上多次查询可以复用排序结果。DFS 递归, 距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
1 struct CentroidPairs
2 {
3     using ll = long long;
4     int n;
5     vector<vector<pair<int, ll>>> g;
6     vector<int> siz, removed, parent;
7     vector<vector<ll>> all;
8     vector<vector<vector<ll>>> part;
9
10    CentroidPairs(int n)
11        : n(n),
12          g(n + 1),
13          siz(n + 1),
14          removed(n + 1),
15          parent(n + 1),
16          all(n + 1),
17          part(n + 1)
18    {
19    }
20
21    void add(int u, int v, ll w)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26    }
27
28    void size_dfs(int u, int p)
29    {
30        siz[u] = 1;
31        for (auto [v, w] : g[u])
32        {
33            if (v == p || removed[v]) continue;
34            size_dfs(v, u);
35            siz[u] += siz[v];
36        }
37    }
38
39    int centroid(int u, int p, int total)
40    {
41        for (auto [v, w] : g[u])
```

```

42     {
43         if (v != p && !removed[v] && siz[v] > total / 2)
44             return centroid(v, u, total);
45     }
46     return u;
47 }
48
49 void collect(int u, int p, ll d, vector<ll> &out)
50 {
51     out.push_back(d);
52     for (auto [v, w] : g[u])
53     {
54         if (v == p || removed[v]) continue;
55         assert(d <= numeric_limits<ll>::max() - w);
56         collect(v, u, d + w, out);
57     }
58 }
59
60 void decompose(int entry, int p)
61 {
62     size_dfs(entry, 0);
63     int c = centroid(entry, 0, siz[entry]);
64     removed[c] = 1;
65     parent[c] = p;
66     all[c] = {0};
67     for (auto [v, w] : g[c])
68     {
69         if (removed[v]) continue;
70         part[c].push_back({});
71         auto &d = part[c].back();
72         collect(v, c, w, d);
73         sort(d.begin(), d.end());
74         all[c].insert(all[c].end(), d.begin(), d.end());
75     }
76     sort(all[c].begin(), all[c].end());
77     for (auto [v, w] : g[c])
78     {
79         if (!removed[v]) decompose(v, c);
80     }
81 }
82
83 // Input must be a forest. Rebuild after changing edges.
84 void build()
85 {
86     fill(removed.begin(), removed.end(), 0);
87     for (int u = 1; u <= n; u++)
88     {
89         all[u].clear();
90         part[u].clear();
91     }
92     for (int u = 1; u <= n; u++)
93     {
94         if (!removed[u]) decompose(u, 0);
95     }
96 }
97

```

```

98  ll pairs(const vector<ll> &d, ll k) const
99  {
100      int l = 0, r = (int)d.size() - 1;
101      ll answer = 0;
102      while (l < r)
103      {
104          if (d[l] <= k - d[r])
105          {
106              answer += r - l;
107              l++;
108          }
109          else
110              r--;
111      }
112      return answer;
113  }
114
115  // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
116  ll count_leq(ll k) const
117  {
118      if (k < 0) return 0;
119      ll answer = 0;
120      for (int c = 1; c <= n; c++)
121      {
122          answer += pairs(all[c], k);
123          for (const auto &d : part[c]) answer -= pairs(d, k);
124      }
125      return answer;
126  }
127
128  ll count_exact(ll k) const
129  {
130      if (k < 0) return 0;
131      return count_leq(k) - (k ? count_leq(k - 1) : 0);
132  }
133 };

```

# 第三十章 树上启发式合并

## 30.1 树上启发式合并：子树众数颜色和

SubtreeColors(n), 点号 1..n, add(u,v) 建无向树; 要求图连通且无环, n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组, colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和, max\_freq[u] 为该频次; 支持负颜色与稀疏编号, 内部离散化, 和用 long long。重复 run 可更换颜色或根。重儿子信息保留, 轻儿子通过 DFS 序区间加入; 清空整棵临时子树后再重置最大值, 不尝试逐次删除时维护众数。O(n log n) 时间, O(n) 空间, 全部 DFS 递归。insert 是内部增量统计, 可按题目修改统计量, 但相应清空规则也必须调整。

```
1 struct SubtreeColors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<vector<int>> g;
6     vector<int> siz, heavy, tin, tout, order, color, value, freq;
7     vector<int> max_freq;
8     vector<long long> answer;
9
10    SubtreeColors(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
15        g[u].push_back(v);
16        g[v].push_back(u);
17    }
18
19    void prepare(int u, int p)
20    {
21        siz[u] = 1;
22        tin[u] = order.size();
23        order.push_back(u);
24        for (int v : g[u])
25        {
26            if (v == p) continue;
27            prepare(v, u);
28            siz[u] += siz[v];
29            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
30        }
31        tout[u] = (int)order.size() - 1;
32    }
33
34    void insert(int u)
35    {
36        int c = color[u];
37        freq[c]++;
38        if (freq[c] > best)
39        {
40            best = freq[c];
41            sum = value[c];
42        }
```

```

43     else if (freq[c] == best)
44         sum += value[c];
45 }
46
47 void solve(int u, int p, bool keep)
48 {
49     for (int v : g[u])
50     {
51         if (v != p && v != heavy[u]) solve(v, u, false);
52     }
53     if (heavy[u]) solve(heavy[u], u, true);
54     for (int v : g[u])
55     {
56         if (v == p || v == heavy[u]) continue;
57         for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
58     }
59     insert(u);
60     max_freq[u] = best;
61     answer[u] = sum;
62     if (!keep)
63     {
64         for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
65         best = 0;
66         sum = 0;
67     }
68 }
69
70 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
71 void run(const vector<int> &colors, int root = 1)
72 {
73     assert((int)colors.size() == n + 1);
74     siz.assign(n + 1, 0);
75     heavy = siz;
76     tin = siz;
77     tout = siz;
78     color = siz;
79     max_freq = siz;
80     answer.assign(n + 1, 0);
81     order.clear();
82     value.assign(colors.begin() + 1, colors.end());
83     sort(value.begin(), value.end());
84     value.erase(unique(value.begin(), value.end()), value.end());
85     freq.assign(value.size(), 0);
86     best = 0;
87     sum = 0;
88     if (!n) return;
89     assert(1 <= root && root <= n);
90     for (int u = 1; u <= n; u++)
91         color[u] =
92             lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
93     prepare(root, 0);
94     solve(root, 0, false);
95 }
96 };

```

# 第三十一章 虚树

## 31.1 虚树：关键点压缩与路径信息

VirtualTree( $n$ ),  $n \geq 1$ , 点号  $1..n$ ; add( $u,v,w=1$ ) 建非负权连通树, 无环和重边。prepare( $root=1$ ) 递归预处理, 可换根重建; 根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA, 返回 vertices 和 edges; 全部编号沿用原树, vertices 按 DFS 序排列, 非空时首项即虚树根。edges 为 (from,to,distance,minimum), 分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果, 单点返回单节点; 原树根不会强制加入, 需要时由调用者加入 keys。边按后代 DFS 序排列, 可逆序做自底向上的 DP。不分配或清空  $n$  个查询邻接表; 每次仅返回  $O(k)$  结果。预处理  $O(n \log n)$  时间空间, 单次  $O(k \log k + k \log n)$ 。lca、ancestor 也可直接使用, 要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法, 原树 DFS 保持递归。

```
1 struct VirtualTree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11     struct Result
12     {
13         vector<int> vertices;
14         vector<Edge> edges;
15     };
16
17     int n, log = 1, timer = 0;
18     vector<vector<pair<int, ll>>> g;
19     vector<vector<int>> up;
20     vector<vector<ll>> mn;
21     vector<int> depth, tin, tout;
22     vector<ll> dist;
23
24     VirtualTree(int n) : n(n), g(n + 1) {}
25
26     void add(int u, int v, ll w = 1)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
29         g[u].push_back({v, w});
30         g[v].push_back({u, w});
31     }
32
33     void dfs(int u, int p)
34     {
35         tin[u] = ++timer;
36         up[0][u] = p;
37         for (int j = 1; j < log; j++)
38         {
39             up[j][u] = up[j - 1][up[j - 1][u]];
```

```

40     mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
41 }
42 for (auto [v, w] : g[u])
43 {
44     if (v == p) continue;
45     assert(dist[u] <= numeric_limits<ll>::max() - w);
46     depth[v] = depth[u] + 1;
47     dist[v] = dist[u] + w;
48     mn[0][v] = w;
49     dfs(v, u);
50 }
51 tout[u] = timer;
52 }
53
54 void prepare(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     log = 1;
58     while ((1LL << log) <= n) log++;
59     up.assign(log, vector<int>(n + 1));
60     mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
61     depth.assign(n + 1, 0);
62     tin = depth;
63     tout = depth;
64     dist.assign(n + 1, 0);
65     timer = 0;
66     dfs(root, root);
67 }
68
69 bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
70
71 int lca(int u, int v) const
72 {
73     if (ancestor(u, v)) return u;
74     if (ancestor(v, u)) return v;
75     for (int j = log - 1; j >= 0; j--)
76     {
77         if (!ancestor(up[j][u], v)) u = up[j][u];
78     }
79     return up[0][u];
80 }
81
82 Edge path(int u, int v) const
83 {
84     assert(u != v && ancestor(u, v));
85     ll minimum = numeric_limits<ll>::max();
86     int x = v, steps = depth[v] - depth[u];
87     for (int j = 0; j < log; j++)
88     {
89         if (steps >> j & 1)
90         {
91             minimum = min(minimum, mn[j][x]);
92             x = up[j][x];
93         }
94     }
95     return {u, v, dist[v] - dist[u], minimum};

```

```

96     }
97
98     // Uses original vertex IDs. Empty input gives an empty result.
99     Result build(vector<int> keys) const
100     {
101         if (keys.empty()) return {};
102         for (int u : keys) assert(1 <= u && u <= n);
103         auto cmp = [&](int u, int v)
104         {
105             return tin[u] < tin[v];
106         };
107         sort(keys.begin(), keys.end(), cmp);
108         keys.erase(unique(keys.begin(), keys.end()), keys.end());
109         int k = keys.size();
110         for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
111         sort(keys.begin(), keys.end(), cmp);
112         keys.erase(unique(keys.begin(), keys.end()), keys.end());
113         Result result{keys, {}};
114         vector<int> st;
115         for (int u : keys)
116         {
117             while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
118             if (!st.empty()) result.edges.push_back(path(st.back(), u));
119             st.push_back(u);
120         }
121         return result;
122     }
123 };

```

## 第三十二章 字符串

### 32.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 inline vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j]) j = p[j - 1];
9         p[i] = j + (s[i] == s[j]);
10    }
11    return p;
12 }
```

### 32.2 KMP 匹配

`kmp_match(s,t)` 返回非空模式串 `t` 在 `s` 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 inline vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j]) j = p[j - 1];
9         j += (s[i] == t[j]);
10        if (j == (int)t.size())
11        {
12            ans.push_back(i - j + 1);
13            j = p[j - 1];
14        }
15    }
16    return ans;
17 }
```

## 32.3 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP，约定 `z[0]=n`，空串返回空数组。O(n)。

```

1 inline vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n) a[0] = n;
6     for (int i = 1, l = 0, r = 0; i < n; i++)
7     {
8         if (i < r) a[i] = min(r - i, a[i - l]);
9         while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
10        if (i + a[i] > r)
11        {
12            l = i;
13            r = i + a[i];
14        }
15    }
16    return a;
17 }
```

## 32.4 Manacher 回文半径

`manacher(s)` 返回 odd 与 even；odd[i] 的半径包含中心，even[i] 的中心在 i 前。字符串及下标为 0-based，空串合法，O(n)。

```

1 // odd[i] includes the center; even[i] is centered before i.
2 inline pair<vector<int>, vector<int>> manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for (int i = 0, l = 0, r = -1; i < n; i++)
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
10        odd[i] = k--;
11        if (i + k > r)
12        {
13            l = i - k;
14            r = i + k;
15        }
16    }
17    for (int i = 0, l = 0, r = -1; i < n; i++)
18    {
19        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
20        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
```

```

21     even[i] = k--;
22     if (i + k > r)
23     {
24         l = i - k - 1;
25         r = i + k;
26     }
27 }
28 return {odd, even};
29 }

```

## 32.5 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。 $O(n)$ 。

```

1 inline int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n) return 0;
5     while (i < n && j < n && k < n)
6     {
7         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
8         if (a == b)
9         {
10             ++k;
11             continue;
12         }
13         if (a > b)
14         {
15             i += k + 1;
16             if (i == j) ++i;
17         }
18         else
19         {
20             j += k + 1;
21             if (i == j) ++j;
22         }
23         k = 0;
24     }
25     return min(i, j);
26 }

```

## 32.6 AC 自动机出现次数

字母表 `a..z`, `add(s)` 返回终止状态, 要求模式非空; 全部插入后 `build()`, 只允许一次构建。`count(text)` 返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建  $O(26N)$ , 查询  $O(|text|+N)$ 。

```

1 struct AhoCorasick
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;
7     };

```

```

8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    int add(const string &s)
14    {
15        assert(!built && !s.empty());
16        int u = 0;
17        for (char ch : s)
18        {
19            int c = ch - 'a';
20            assert(0 <= c && c < 26);
21            if (!a[u].go[c])
22            {
23                int v = (int)a.size();
24                a[u].go[c] = v;
25                a.push_back(Node{});
26            }
27            u = a[u].go[c];
28        }
29        return u;
30    }
31
32    void build()
33    {
34        assert(!built);
35        built = true;
36        queue<int> q;
37        for (int v : a[0].go)
38            if (v) q.push(v);
39        while (!q.empty())
40        {
41            int u = q.front();
42            q.pop();
43            order.push_back(u);
44            for (int c = 0; c < 26; c++)
45            {
46                int v = a[u].go[c];
47                if (v)
48                {
49                    a[v].fail = a[a[u].fail].go[c];
50                    q.push(v);
51                }
52                else
53                    a[u].go[c] = a[a[u].fail].go[c];
54            }
55        }
56    }
57
58    vector<long long> count(const string &s) const
59    {
60        assert(built);
61        vector<long long> ans(a.size());
62        int u = 0;
63        for (char ch : s)

```

```
64     {
65         u = a[u].go[ch - 'a'];
66         ++ans[u];
67     }
68     for (int i = (int)order.size() - 1; i >= 0; i--)
69         ans[a[order[i]].fail] += ans[order[i]];
70     return ans;
71 }
72 };
```

## 32.7 倍增后缀数组与 LCP

构造 `SuffixArray(s)` 支持任意字节串, 按 `unsigned char` 的 `0..255` 数值比较, 包含内嵌零字节; 也可构造 `SuffixArray(a,alphabet)`, 整数序列要求  $0 \leq a[i] < \text{alphabet}$ ,  $1 \leq \text{alphabet} < \text{INT\_MAX}$ 。`init(a,alphabet)` 可重复重建。长度须小于 `INT_MAX`。`sa[k]` 为第  $k$  个非空后缀起点, `rk[i]` 为排名, 全部 0-based; `lcp[k]` 为 `sa[k-1]` 与 `sa[k]` 的最长公共前缀, `lcp[0]=0`。空序列合法, 三数组均为空。无需手动追加哨兵, 也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序, 倍增  $O(n \log n + \text{alphabet})$ , 空间  $O(n + \text{alphabet})$ ; 字节串 `alphabet=256`。大整数先自行保序离散化, 避免按最大原值开桶。Kasai 求 `lcp` 为  $O(n)$ 。不同起点  $x,y$  的 LCP 为 `lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])]` 的最小值; 本结构不内建 RMQ, 需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 `n-x`。覆盖倍增构造, 不替代 DC3/SA-IS。

```

1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for (unsigned char c : s) a.push_back(c);
10        init(a, 256);
11    }
12
13    SuffixArray(const vector<int> &s, int alphabet) { init(s, alphabet); }
14
15    void init(const vector<int> &s, int alphabet)
16    {
17        assert(s.size() < INT_MAX);
18        assert(1 <= alphabet && alphabet < INT_MAX);
19        int n = (int)s.size();
20        sa.resize(n);
21        rk.resize(n);
22        lcp.assign(n, 0);
23        if (!n) return;
24        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
25        int m = alphabet;
26        for (int i = 0; i < n; i++)
27        {
28            assert(0 <= s[i] && s[i] < alphabet);
29            rk[i] = s[i] + 1;
30            cnt[rk[i]]++;
31        }
32        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
33        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;

```

```

34     for (int k = 1; k < n; k *= 2)
35     {
36         int p = 0;
37         for (int i = n - k; i < n; i++) order[p++] = i;
38         for (int i : sa)
39             if (i >= k) order[p++] = i - k;
40         fill(cnt.begin(), cnt.begin() + m + 1, 0);
41         for (int x : rk) cnt[x]++;
42         for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
43         for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
44         old.swap(rk);
45         auto second = [&](int i)
46         {
47             return i < n - k ? old[i + k] : 0;
48         };
49         m = 1;
50         rk[sa[0]] = m;
51         for (int i = 1; i < n; i++)
52         {
53             int a = sa[i - 1], b = sa[i];
54             if (old[a] != old[b] || second(a) != second(b)) m++;
55             rk[b] = m;
56         }
57         if (m == n) break;
58     }
59     for (int i = 0; i < n; i++) rk[sa[i]] = i;
60     for (int i = 0, k = 0; i < n; i++)
61     {
62         if (!rk[i])
63         {
64             k = 0;
65             continue;
66         }
67         int j = sa[rk[i] - 1];
68         while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
69         lcp[rk[i]] = k;
70         if (k) --k;
71     }
72 }
73 };

```

## 32.8 后缀自动机

`extend(c)` 接收整数 0..25; `distinct()` 返回不同非空子串数, `counts()` 返回 `endpos` 次数且不修改结构, 可重复调用。状态 `a`, 初始状态 0, `link` 根为 -1; 空间  $O(26n)$ , 构建  $O(26n)$ 。

```

1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while (p != -1 && !a[p].go[c])
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if (p == -1)
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if (a[p].len + 1 == a[q].len)
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while (p != -1 && a[p].go[c] == q)
41                {
42                    a[p].go[c] = clone;
43                    p = a[p].link;
44                }
45                a[q].link = a[cur].link = clone;
46            }
47        }
48        last = cur;
49    }
50
51    vector<long long> counts() const

```

```
52 {
53     vector<int> bucket(a[last].len + 1), order(a.size());
54     vector<long long> ans(a.size());
55     for (auto &v : a) ++bucket[v.len];
56     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
57     for (int i = 0; i < (int)a.size(); i++)
58     {
59         order[--bucket[a[i].len]] = i;
60         ans[i] = a[i].occ;
61     }
62     for (int i = (int)order.size() - 1; i > 0; i--)
63         ans[a[order[i]].link] += ans[order[i]];
64     return ans;
65 }
66
67 long long distinct() const
68 {
69     long long ans = 0;
70     for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
71     return ans;
72 }
73 };
```

## 第三十三章 后缀查询与重复子串

### 33.1 后缀 LCP 与子串比较

先构造 SuffixArray，再构造 SuffixLCP(sa)。init(sa) / Init(sa) 可重新建表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀， $0 \leq x, y \leq n$ ，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表  $O(n \log n)$  时间、空间，每次查询  $O(1)$ ，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。

```
1 struct SuffixLCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>> st;
6
7     SuffixLCP(const SuffixArray &suffix) { init(suffix); }
8
9     void init(const SuffixArray &suffix)
10    {
11        n = suffix.sa.size();
12        rk = suffix.rk;
13        lg.assign(n + 1, 0);
14        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
15        st.assign(lg[n] + 1, {});
16        st[0] = suffix.lcp;
17        for (int k = 1; k <= lg[n]; k++)
18        {
19            int len = 1 << k;
20            st[k].resize(n - len + 1);
21            for (int i = 0; i <= n - len; i++)
22                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
23        }
24    }
```

```

26  int query(int x, int y) const
27  {
28      assert(0 <= x && x <= n && 0 <= y && y <= n);
29      if (x == y || x == n || y == n) return n - max(x, y);
30      int a = rk[x], b = rk[y];
31      if (a > b) swap(a, b);
32      int k = lg[b - a];
33      return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
34  }
35
36  // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
37  int compare(int l1, int r1, int l2, int r2) const
38  {
39      assert(0 <= l1 && l1 <= r1 && r1 <= n);
40      assert(0 <= l2 && l2 <= r2 && r2 <= n);
41      int a = r1 - l1, b = r2 - l2;
42      if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
43      return rk[l1] < rk[l2] ? -1 : 1;
44  }
45  };

```

## 33.2 两个前缀的最长公共后缀

`prefix_lcs(reversed,x,y)` 返回原序列 `s[0..x)` 与 `s[0..y)` 的最长公共后缀，端点满足  $0 \leq x, y \leq n$ 。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 `reversed.query(n-x,n-y)`，单次  $O(1)$ 。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```

1  // reversed must be built on the reversed original sequence.
2  inline int prefix_lcs(const SuffixLCP &reversed, int x, int y)
3  {
4      assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5      return reversed.query(reversed.n - x, reversed.n - y);
6  }

```

### 33.3 重复子串 AA 的起止位置计数

`square_counts(forward,reversed)` 返回 `(start,finish)` 两个长度 `n` 的 `long long` 数组, `start[i]/finish[i]` 为开始/结束于位置 `i` 的非空 AA 型子串个数; 结束位置是最后一个字符下标, 均 0-based。两对象必须分别对应同一原序列及其反序列, 长度相同, 允许空序列。枚举半长 `len`, 并以其整数倍 `i` 为锚点; 向左匹配截断为 `len-1`, 向右截断为 `len`, 由 LCP/LCS 得出合法起点连续区间, 差分累计。左侧不能放宽到 `len`, 否则相邻锚点会重复计数。预处理查询器后, 此函数  $O(n \log n)$  时间、 $O(n)$  额外空间。不同出现位置、不同半长分别计数; 不要求 A 为最小周期。P1117 的 AABB 拆分数为所有分界点 `finish[i]*start[i+1]` 之和, 累加类型需按题目范围选择, 函数本身只返回起止计数。

```

1 // Count nonempty squares AA by their starting and ending positions.
2 inline pair<vector<long long>, vector<long long>>
3 square_counts(const SuffixLCP &forward, const SuffixLCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for (int len = 1; len <= n / 2; len++)
9         for (int i = 0; i < n - len; i += len)
10            {
11                int j = i + len;
12                int left = min(len - 1, prefix_lcs(reversed, i, j));
13                int right = min(len, forward.query(i, j));
14                if (left + right < len) continue;
15                int l = i - left, r = i + right - len;
16                start[l]++;
17                start[r + 1]--;
18                finish[l + 2 * len - 1]++;
19                finish[r + 2 * len]--;
20            }
21     for (int i = 1; i < n; i++)
22     {
23         start[i] += start[i - 1];
24         finish[i] += finish[i - 1];
25     }
26     start.resize(n);
27     finish.resize(n);
28     return {start, finish};
29 }
```

## 第三十四章 回文树

### 34.1 回文树 PAM: 回文后缀与出现次数

`add(ch)` 追加小写英文字母, 返回最长回文后缀节点, 亦保存于 `last`。节点 0 是长度 0 根, 1 是长度 -1 根, 实际回文节点从 2 开始; `distinct()` 为不同非空回文串数。`t[u].len` 为长度, `fail` 为最长真回文后缀链接, `suffixes` 为该节点的非空回文后缀数量; 追加后的 `t[last].suffixes` 就是本位置结尾的回文串数。`total` 为当前全部回文子串的出现次数之和。`end` 为首次出现时的 0-based 右端点, 可恢复原串子段; `hits` 仅统计作为最长后缀的次数, 不能当出现次数。`occurrences()` 返回按节点编号的完整出现次数副本, 根置零,  $O(n)$ ; 重复调用以及查询后继续追加均安全。`init()` 清空重建。固定 26 字符集下建树  $O(n)$ , 空间  $O(26n)$ 。

```
1 struct PalindromicTree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    vector<Node> t;
11    vector<int> s;
12    int last = 0;
13    long long total = 0;
14
15    PalindromicTree() { init(); }
16
17    void init()
18    {
19        t.assign(2, Node{});
20        t[0].fail = 1;
21        t[1].len = -1;
22        s.assign(1, -1);
23        last = 0;
24        total = 0;
25    }
26
27    int get_fail(int u) const
28    {
29        int pos = s.size() - 1;
30        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
31        return u;
32    }
33
34    // Append a lowercase letter; return the longest palindromic suffix node.
35    int add(char ch)
36    {
37        int c = ch - 'a';
38        assert(0 <= c && c < 26);
39        s.push_back(c);
40        int u = get_fail(last);
41        if (!t[u].next[c])
```

```

42     {
43         int v = t.size();
44         t.push_back(Node{});
45         t[v].len = t[u].len + 2;
46         t[v].fail = t[get_fail(t[u].fail)].next[c];
47         t[v].suffixes = t[t[v].fail].suffixes + 1;
48         t[v].end = (int)s.size() - 2;
49         t[u].next[c] = v;
50     }
51     last = t[u].next[c];
52     t[last].hits++;
53     total += t[last].suffixes;
54     return last;
55 }
56
57 int distinct() const { return (int)t.size() - 2; }
58
59 // Copy-and-propagate: repeated calls and later appends remain valid.
60 vector<long long> occurrences() const
61 {
62     vector<long long> count(t.size());
63     for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
64     for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
65     count[0] = 0;
66     count[1] = 0;
67     return count;
68 }
69 };

```

# 第三十五章 数论

## 35.1 模运算、素性、CRT 与类欧几里德

mul/power 支持无符号 64 位正模数，乘积用 uint128。prime 使用确定性 64 位底数。inverse(a,m) 无逆元返回 -1。crt(r,m,b,n) 合并  $x \equiv r \pmod{m}$  与  $x \equiv b \pmod{n}$ ，要求  $m,n > 0$ 。从  $r=0$ 、 $m=1$  开始逐项合并；成功后  $r$  为最小非负解、 $m$  为最小正周期。余数允许完整 long long 范围。无解返回 false；有解但新周期超过 LLONG\_MAX 抛 overflow\_error。两种失败都会先将  $r$  归一化， $m$  不变；失败后不能继续当作完整解使用。递归 exgcd 和乘法中间量用 int128，单次  $O(\log \min(m,n))$ ，抄写 CRT 时只需类型别名、exgcd 和 crt。floor\_sum 为  $i=0..n-1$  的  $\text{floor}((ai+b)/m)$  之和，支持负  $a$ 、 $b$ ，返回 int128；约定  $n \leq 10^9$ 、 $m \leq 10^9$ ， $a$ 、 $b$  为有符号 64 位。exgcd 单独调用约定  $0 \leq a, b \leq 2^{63}$ ，输出 int128 系数满足  $ax+by=\text{gcd}(a,b)$ ， $(0,0)$  返回 0；有符号方程用 linear\_equation。

```
1 struct NumberTheory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7
8     static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }
9
10    static ull power(ull a, ull b, ull m)
11    {
12        assert(m);
13        ull r = 1 % m;
14        a %= m;
15        for (; b >= 1, a = mul(a, a, m))
16            if (b & 1) r = mul(r, a, m);
17        return r;
18    }
19
20    static bool prime(ull n)
21    {
22        if (n < 2) return false;
23        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
24            if (n % p == 0) return n == p;
25        ull d = n - 1;
26        int s = 0;
27        while (!(d & 1))
28        {
29            d >>= 1;
30            ++s;
31        }
32        for (ull a :
33            {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
34        {
35            if (a % n == 0) continue;
36            ull x = power(a, d, n);
37            if (x == 1 || x == n - 1) continue;
38            bool ok = false;
39            for (int j = 1; j < s; j++)
```

```

40         {
41             x = mul(x, x, n);
42             if (x == n - 1)
43             {
44                 ok = true;
45                 break;
46             }
47         }
48         if (!ok) return false;
49     }
50     return true;
51 }
52
53 static i128 exgcd(i128 a, i128 b, i128 &x, i128 &y)
54 {
55     if (!b)
56     {
57         x = 1;
58         y = 0;
59         return a;
60     }
61     i128 d = exgcd(b, a % b, y, x);
62     y -= a / b * x;
63     return d;
64 }
65
66 static ll inverse(ll a, ll m)
67 {
68     assert(m > 0);
69     a %= m;
70     if (a < 0) a += m;
71     i128 x, y;
72     if (exgcd(a, m, x, y) != 1) return -1;
73     return (x % m + m) % m;
74 }
75
76 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
77 static bool crt(ll &r, ll &m, ll b, ll n)
78 {
79     assert(m > 0 && n > 0);
80     r %= m;
81     if (r < 0) r += m;
82     b %= n;
83     if (b < 0) b += n;
84     i128 x, y;
85     ll g = (ll)exgcd(m, n, x, y);
86     i128 diff = i128(b) - r;
87     if (diff % g) return false;
88     i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
89     if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
90     r = (r + i128(m) * k) % mod;
91     m = (ll)mod;
92     return true;
93 }
94
95 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.

```

```

96 static i128 floor_sum(ll n, ll m, ll a, ll b)
97 {
98     assert(n >= 0 && m > 0);
99     i128 ans = 0;
100     auto norm = [&](ll &v)
101     {
102         ll q = v / m;
103         if (v % m < 0) --q;
104         v = (ll)(i128(v) - i128(q) * m);
105         return q;
106     };
107     ans += i128(norm(a)) * n * (n - 1) / 2;
108     ans += i128(norm(b)) * n;
109     while (true)
110     {
111         ans += i128(n) * (n - 1) / 2 * (a / m);
112         a %= m;
113         ans += i128(n) * (b / m);
114         b %= m;
115         i128 y = i128(a) * n + b;
116         if (y < m) break;
117         n = (ll)(y / m);
118         b = (ll)(y % m);
119         swap(a, m);
120     }
121     return ans;
122 }
123 };

```

## 35.2 Pollard–Rho 因数分解

`factor(n)` 返回升序质因子（含重数）， $1 \leq n \leq \text{ULLONG\_MAX}$ 。`n=1` 返回空数组；`n≥2` 时 `size()==1` 等价于 `n` 为素数，否则 `front()/back()` 分别取最小/最大质因子。依赖 `NumberTheory`。公开使用 `factor`；`rho/split` 为内部辅助，`rho` 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 `rho` 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度  $O(\log n)$ 。

```

1 struct PollardRho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     PollardRho(ull seed = 712367821) : rng(seed) {}
8
9     ull rho(ull n)
10    {
11        if (n % 2 == 0) return 2;
12        for (;;)
13        {
14            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
15            auto f = [&](ull v)
16            {
17                return (NumberTheory::mul(v, v, n) + u128(c)) % n;
18            };
19            // Retry bounded attempts; Las Vegas: only exact divisors returned.
20            for (int i = 0; i < 200000 && d == 1; i++)
21            {
22                x = f(x);
23                y = f(f(y));
24                d = gcd(x > y ? x - y : y - x, n);
25            }
26            if (1 < d && d < n) return d;
27        }
28    }
29
30    void split(ull n, vector<ull> &a)
31    {
32        if (n == 1) return;
33        if (NumberTheory::prime(n))
34        {
35            a.push_back(n);
36            return;
37        }
38        ull d = rho(n);
39        split(d, a);
40        split(n / d, a);
41    }

```

```

42
43     vector<ull> factor(ull n)
44     {
45         assert(n >= 1);
46         vector<ull> a;
47         split(n, a);
48         sort(a.begin(), a.end());
49         return a;
50     }
51 };

```

### 35.3 线性筛、欧拉函数与 Möbius 函数

构造参数  $0 \leq n < \text{INT\_MAX}$ ，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为  $0..n$  数组。 $x \geq 2$  时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。 $n \geq 1$  时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时  $O(\log x)$ 。 $n=0$  不可访问下标 1。

```

1 struct LinearSieve
2 {
3     vector<int> prime, lp, phi, mu;
4
5     LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if (n) phi[1] = mu[1] = 1;
8         for (int i = 2; i <= n; i++)
9         {
10             if (!lp[i])
11             {
12                 lp[i] = i;
13                 prime.push_back(i);
14                 phi[i] = i - 1;
15                 mu[i] = -1;
16             }
17             for (int p : prime)
18             {
19                 if (p > n / i) break;
20                 int j = i * p;
21                 lp[j] = p;
22                 if (i % p == 0)
23                 {
24                     phi[j] = phi[i] * p;
25                     mu[j] = 0;
26                     break;
27                 }
28                 phi[j] = phi[i] * (p - 1);
29                 mu[j] = -mu[i];
30             }
31         }
32     }
33 };

```

## 35.4 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且  $\geq 2$ 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘  $O(1)$ ，幂和逆  $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```

1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };

```

## 35.5 阶乘组合数

`Binomial<prime>(n=0)` 预处理  $0..n$ ，要求 `prime` 为素数且  $0 \leq n < \text{prime}$ 。`init(n)` 只扩大当前上限，较小或相同的 `n` 不改变表；`choose(n,k)` 返回  $C(n,k)$ ，`permute(n,k)` 返回  $n!/(n-k)!$ 。查询前显式预处理到 `n`，查询本身不扩表。`n < 0`、`k < 0` 或 `k > n` 返回 0， $C(0,0)=P(0,0)=1$ 。`fac`、`ifac` 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询  $O(1)$ ；从 `old` 扩到 `n` 新增  $O(n-\text{old}+\log \text{prime})$  次域运算，重分配还可能复制旧表。首次预处理  $O(n+\log \text{prime})$ ，空间  $O(n)$ 。超过模数不能使用阶乘除法：小素数用 `Lucas`，小合数用 `exLucas`。

```

1  template <int mod> struct Binomial
2  {
3      using Z = ModInt<mod>;
4      vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6      Binomial(int n = 0) { init(n); }
7
8      void init(int n)
9      {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
19
20     Z choose(int n, int k) const
21     {
22         if (k < 0 || k > n) return 0;
23         assert(n < (int)fac.size());
24         return fac[n] * ifac[k] * ifac[n - k];
25     }
26
27     Z permute(int n, int k) const
28     {
29         if (k < 0 || k > n) return 0;
30         assert(n < (int)fac.size());
31         return fac[n] * ifac[n - k];
32     }
33 };

```

## 第三十六章 二元一次不定方程

### 36.1 二元一次不定方程

`linear_equation(a,b,c,x,y)` 求  $ax+by=c$  的任意整数特解。a、b、c 支持完整 long long, x、y 必须为 int128。无解返回 false 且  $x=y=0$ ;  $a=b=c=0$  返回 true、 $x=y=0$ , 此时所有整数对都是解。其他有解情况, 令  $g=\gcd(|a|,|b|)$ , 通解为  $(x+(b/g)t, y-(a/g)t)$ 。依赖 NumberTheory::exgcd; 只需抄递归 exgcd 与本函数。时间、递归空间  $O(\log M)$ , 其中  $M=\max(|a|,|b|)+2$ ; 不承诺特解最小或为正。

```
1 inline bool
2 linear_equation(long long a, long long b, long long c, __int128_t &x, __int128_t &y)
3 {
4     using i128 = __int128_t;
5     x = 0;
6     y = 0;
7     if (!a && !b) return c == 0;
8     i128 aa = a < 0 ? -i128(a) : i128(a);
9     i128 bb = b < 0 ? -i128(b) : i128(b);
10    i128 u, v;
11    i128 g = NumberTheory::exgcd(aa, bb, u, v);
12    if (c % g) return false;
13    x = u * (c / g);
14    y = v * (c / g);
15    if (a < 0) x = -x;
16    if (b < 0) y = -y;
17    return true;
18 }
```

## 第三十七章 线性同余方程

### 37.1 线性同余方程

`linear_congruence(a,b,m)` 求  $a*x+b\equiv 0 \pmod m$ , 返回 {最小非负解, 最小正周期}, 无解返回 {-1,-1}。m 为正 long long, a、b 支持完整 long long。若  $g=\gcd(a,m)$  整除 b, 周期为  $m/g$ ; 包括  $a=0$  和  $m=1$ , 所有整数均满足时返回 {0,1}。先归一化 a, 并在 int128 中对 b 取相反数和乘法。依赖 NumberTheory::exgcd; 递归时间、空间  $O(\log(m+1))$ 。注意本接口的常数项是 +b, 若题目写  $a*x\equiv c$ , 应传 -c; c 为 LLONG\_MIN 时须避免在 64 位中取负。

```
1 inline pair<long long, long long>
2 linear_congruence(long long a, long long b, long long m)
3 {
4     assert(m > 0);
5     using i128 = __int128_t;
6     a %= m;
7     if (a < 0) a += m;
8     i128 x, y;
9     i128 g = NumberTheory::exgcd(a, m, x, y);
10    if (i128(b) % g) return {-1, -1};
11    i128 period = m / g;
12    x = x * (-i128(b) / g) % period;
13    if (x < 0) x += period;
14    return {(long long)x, (long long)period};
15 }
```

## 第三十八章 区间筛素数

### 38.1 区间筛素数

`segmented_primes(l,r)` 返回闭区间内递增的全部素数,  $0 \leq l \leq r \leq \text{INT\_MAX}$ ; 区间长度须能分配内存。独立函数, 每次重建底筛; 不依赖预存素数表。令  $W=r-l+1$ , 时间  $O((\sqrt{r+W})\log \log(r+3))$ , 空间  $O(\sqrt{r+W})$ 。倍数起点、递增变量均用 `long long`, 避免 `INT_MAX` 端点溢出; 0 和 1 不进入结果。

```
1 inline vector<int> segmented_primes(int l, int r)
2 {
3     assert(0 <= l && l <= r);
4     int limit = 0;
5     while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
6     vector<char> composite(limit + 1);
7     vector<char> removed(1LL * r - l + 1);
8     for (int p = 2; p <= limit; p++)
9         if (!composite[p])
10            {
11                for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
12                long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
13                for (long long j = start; j <= r; j += p) removed[j - l] = true;
14            }
15     vector<int> primes;
16     for (long long x = max(l, 2); x <= r; x++)
17         if (!removed[x - l]) primes.push_back(x);
18     return primes;
19 }
```

## 第三十九章 批量模逆元

### 39.1 任意数组的批量逆元

`batch_inverse(a,mod)` 返回与 `a` 同序的逆元向量。要求 `mod` 为正 `long long`，每个输入可为完整 `long long`；只要某一项与 `mod` 不互素，整个结果返回 `nullopt`，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 `move(a)` 可避免输入副本。依赖 `NumberTheory` 的 `inverse` 和 `exgcd`。仅求一次逆元，时间  $O(n+\log \text{mod})$ ，空间  $O(n)$ ；128 位中间乘法防止溢出。

```
1 inline optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
2 {
3     assert(mod > 0);
4     int n = a.size();
5     vector<long long> prefix(n + 1, 1 % mod);
6     for (int i = 0; i < n; i++)
7     {
8         a[i] %= mod;
9         if (a[i] < 0) a[i] += mod;
10        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
11    }
12    long long inverse = NumberTheory::inverse(prefix[n], mod);
13    if (inverse == -1) return nullopt;
14    for (int i = n - 1; i >= 0; i--)
15    {
16        long long value = a[i];
17        a[i] = __int128_t(inverse) * prefix[i] % mod;
18        inverse = __int128_t(inverse) * value % mod;
19    }
20    return a;
21 }
```

## 第四十章 连续整数逆元表

### 40.1 连续整数的线性逆元表

`inverse_table(n,p)` 返回长度  $n+1$  的 `int` 向量，位置  $i$  保存  $i$  模  $p$  的逆元， $1 \leq i \leq n$ 。位置 0 固定为 0，仅为占位，不表示零有逆元； $n=0$  返回单个占位项。要求  $p$  为素数、 $2 \leq p \leq \text{INT\_MAX}$  且  $0 \leq n < p$ ，不检测素性。时间和空间均  $O(n)$ ，中间乘法用 `long long`；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前  $i$  与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
1 inline vector<int> inverse_table(int n, int p)
2 {
3     assert(p >= 2 && 0 <= n && n < p);
4     vector<int> inverse(n + 1);
5     if (n) inverse[1] = 1;
6     for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
7     return inverse;
8 }
```

# 第四十一章 混合进制中国剩余定理

## 41.1 Garner: 大周期 CRT 与混合进制

`garner_digits(b,moduli)` 返回按输入顺序排列的混合进制各位; `garner(b,moduli,target)` 返回最小非负 CRT 解对 `target` 的余数。两个输入向量等长, 长度可为 0; 模数为正且两两互素, 允许模数 1, 余数支持完整 `long long` 范围。`target` 为正 `long long`, 与输入模数无需互素。输入不互素时抛 `invalid_argument`, 即使方程有解也不适用; 此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积, 中间运算使用 `int128`。k 个模数耗时  $O(k^2 + k \log M)$ , 空间  $O(k)$ ;  $M$  为最大输入模数。依赖 `NumberTheory` 的 `inverse` 和 `exgcd`。

```
1 // Pairwise coprime positive moduli. Digits use the input order.
2 inline vector<long long> garner_digits(const vector<long long> &b,
3                                       const vector<long long> &moduli)
4 {
5     using ll = long long;
6     using i128 = __int128_t;
7     int n = b.size();
8     assert(moduli.size() == b.size());
9     vector<ll> value(n), coefficient(n, 1), digit(n);
10    for (ll m : moduli) assert(m > 0);
11    for (int i = 0; i < n; i++)
12    {
13        ll m = moduli[i];
14        ll inverse = NumberTheory::inverse(coefficient[i], m);
15        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
16        i128 delta = (i128(b[i]) - value[i]) % m;
17        if (delta < 0) delta += m;
18        digit[i] = delta * inverse % m;
19        for (int j = i + 1; j < n; j++)
20        {
21            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
22            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
23        }
24    }
25    return digit;
26 }
```

### 规范解对目标数取模

```
28 // Return the least nonnegative CRT solution modulo target, not all solutions.
29 inline long long
30 garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
31 {
32     assert(target > 0);
33     auto digit = garner_digits(b, moduli);
34     long long answer = 0;
35     for (int i = (int)digit.size() - 1; i >= 0; i--)
36         answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
37     return answer;
38 }
```

## 第四十二章 乘法阶与原根

### 42.1 乘法阶与原根

构造 `PrimitiveRoot(n)`，要求  $2 \leq n \leq \text{INT\_MAX}$ ；`init(n)` 可重建。`order(a)` 返回  $a$  模  $n$  的乘法阶， $a$  不可逆时返回 0；`is_root(a)` 判断该剩余类是否为原根， $a$  可为完整 `long long`。`minimum()` 返回  $1..n-1$  内最小原根，无原根返回 -1；`all()` 返回升序的全部原根，无原根返回空向量。`mod`、`phi`、`exists` 只读；`factors` 为 `phi` 的不同素因子，辅助 `factor` 为试除分解。依赖 `NumberTheory` 的 `power`。初始化  $O(\sqrt{n})$ ，求阶  $O(\log^2 n)$ ，原根判定  $O((1+\omega(\phi))\log n)$ 。若最小原根为  $G$ ，顺序枚举最坏检查  $G$  个候选，不能假定  $G$  总是常数。枚举全部另需  $O(\phi \log \phi + c \log c)$  时间、 $O(c)$  空间， $c$  为原根数；大模数只求最小原根或阶时，不应调用 `all`。

```
1 struct PrimitiveRoot
2 {
3     int mod, phi;
4     bool exists;
5     vector<int> factors;
6
7     static vector<int> factor(int n)
8     {
9         vector<int> result;
10        for (int p = 2; p <= n / p; p++)
11            if (n % p == 0)
12            {
13                result.push_back(p);
14                while (n % p == 0) n /= p;
15            }
16        if (n > 1) result.push_back(n);
17        return result;
18    }
19
20    PrimitiveRoot(int n) { init(n); }
```

```
22 void init(int n)
23 {
24     assert(n >= 2);
25     mod = n;
26     phi = n;
27     for (int p : factor(n)) phi -= phi / p;
28     int odd = n;
29     if (odd % 2 == 0) odd /= 2;
30     exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
31     factors = factor(phi);
32 }
33
34 int order(long long a) const
35 {
36     a %= mod;
37     if (a < 0) a += mod;
38     if (gcd(a, (long long)mod) != 1) return 0;
39     int result = phi;
40     for (int p : factors)
41         while (result % p == 0 && NumberTheory::power(a, result / p, mod) == 1)
42             result /= p;
43     return result;
44 }
```

```
46     bool is_root(long long a) const
47     {
48         if (!exists) return false;
49         a %= mod;
50         if (a < 0) a += mod;
51         if (gcd(a, (long long)mod) != 1) return false;
52         for (int p : factors)
53             if (NumberTheory::power(a, phi / p, mod) == 1) return false;
54         return true;
55     }
56
57     int minimum() const
58     {
59         if (!exists) return -1;
60         for (int g = 1; g < mod; g++)
61             if (is_root(g)) return g;
62         return -1;
63     }
64
65     vector<int> all() const
66     {
67         int g = minimum();
68         vector<int> roots;
69         if (g == -1) return roots;
70         long long value = 1;
71         for (int k = 1; k <= phi; k++)
72         {
73             value = value * g % mod;
74             if (gcd(k, phi) == 1) roots.push_back(value);
75         }
76         sort(roots.begin(), roots.end());
77         return roots;
78     }
79 };
```

# 第四十三章 矩形 GCD 计数

## 43.1 矩形 GCD 计数

构造 CoprimePairs(n) 预处理 mu 前缀和,  $0 \leq n < \text{INT\_MAX}$ ,  $O(n)$  时间与空间; 依赖 LinearSieve。count(a,b) 求  $[1,a] \times [1,b]$  的互质数对, a,b 为非负 int, 要求  $\min(a,b) \leq n$ , 任一为 0 返回 0。rectangle(a,b,c,d,k) 求  $[a,b] \times [c,d]$  中  $\gcd(x,y)=k$  的数对, 要求  $1 \leq a \leq b$ ,  $1 \leq c \leq d$ ,  $k \geq 1$ ,  $\min(b/k, d/k) \leq n$ ; 闭区间容斥。返回 long long, 中间分块累加用 int128。单次前缀查询  $O(\sqrt{a} + \sqrt{b})$ , 额外空间  $O(1)$ 。sum 为 mu 前缀和, 只读。

```
1 struct CoprimePairs
2 {
3     vector<int> sum;
4
5     CoprimePairs(int n) : sum(LinearSieve(n).mu)
6     {
7         for (int i = 1; i <= n; i++) sum[i] += sum[i - 1];
8     }
9
10    long long count(int a, int b) const
11    {
12        assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
13        __int128_t answer = 0;
14        for (int l = 1, r; l <= min(a, b); l = r + 1)
15        {
16            int x = a / l;
17            int y = b / l;
18            r = min(a / x, b / y);
19            answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
20        }
21        return (long long)answer;
22    }
23
24    long long rectangle(int a, int b, int c, int d, int k) const
25    {
26        assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
27        return count(b / k, d / k) - count((a - 1) / k, d / k) -
28            count(b / k, (c - 1) / k) + count((a - 1) / k, (c - 1) / k);
29    }
30 };;
```

## 第四十四章 带权类欧几里德

### 44.1 整除和、带权和与平方和

`floor_moments(n,m,a,b)` 对  $i=0..n-1$ , 令  $y_i=\text{floor}((a*i+b)/m)$ , 返回  $\{\sum y_i, \sum i*y_i, \sum y_i^2\}$ , 元素为 `ModInt<998244353>`, 取整数用 `.v`。  $0 \leq n \leq 1000000001$ ,  $1 \leq m \leq 10^9$ ;  $a, b$  支持完整 `long long`。  $n=0$  返回全零。时间与递归空间  $O(\log(m+1))$ ; 模数固定, 499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间, 驱动传  $n+1$ , 输出索引顺序 0、2、1。

```
1 inline array<ModInt<998244353>, 3>
2 floor_moments(long long n, long long m, long long a, long long b)
3 {
4     using Z = ModInt<998244353>;
5     assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
6     if (!n) return {0, 0, 0};
7     auto normalize = [&](long long &v)
8     {
9         long long q = v / m;
10        if (v % m < 0) q--;
11        v = (__int128_t(v) - __int128_t(q) * m);
12        return Z(q);
13    };
14    Z x = normalize(a);
15    Z y = normalize(b);
16    Z count = n;
17    Z s1 = count * Z(n - 1) * Z(499122177);
18    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
19    Z f = 0, g = 0, h = 0;
20    if (a)
21    {
22        long long height = (a * (n - 1) + b) / m;
23        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
24        Z t = height;
25        f = count * t - u;
26        g = t * s1 - (w - u) * Z(499122177);
27        h = count * t * t - Z(2) * v - u;
28    }
29    return {f + x * s1 + y * count,
30            g + x * s2 + y * s1,
31            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
32            Z(2) * y * f};
33 }
```

## 第四十五章 模几何级数与约数和

### 45.1 模幂与几何级数同时求值

`power_sum(a,n,mod)` 返回  $\{a^n \bmod \text{mod}, \sum_{i=0..n-1} a^i \bmod \text{mod}\}$ 。a 为完整 unsigned long long, n 为 uint128 项数, mod 为正 unsigned long long。n=0 返回  $\{1 \bmod \text{mod}, 0\}$ ; a=0 且 n>0 时和为  $1 \bmod \text{mod}$ 。无需逆元, 合数模数也可用。时间与递归空间  $O(\log(n+1))$ , 至多 128 次折半。只抄此函数需 utility、cassert。

```
1 inline pair<unsigned long long, unsigned long long>
2 power_sum(unsigned long long a, __uint128_t n, unsigned long long mod)
3 {
4     assert(mod >= 1);
5     if (!n) return {1 % mod, 0};
6     auto [p, s] = power_sum(a, n / 2, mod);
7     unsigned long long square = __uint128_t(p) * p % mod;
8     unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
9     if (n % 2)
10         return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
11     return {square, sum};
12 }
```

## 45.2 幂的约数和

传入  $a$ 、 $b$ 、 $\text{mod}$  和可复用的 PollardRho 分解器 `solver`，返回  $a^b$  的正约数和取模。 $a$ 、 $b$ 、 $\text{mod}$  均为 unsigned long long,  $a \geq 1$ 、 $\text{mod} \geq 1$ 。 $a=1$  或  $b=0$  时结果为  $1 \% \text{mod}$ 。依赖 `power_sum` 与分解器；重数  $e$  对应  $e*b+1$  项，长度用 `uint128`。时间由分解及每个不同质因子的  $O(\log(e*b+2))$  求和组成。

```
1 inline unsigned long long divisor_sum_power(unsigned long long a,
2                                             unsigned long long b,
3                                             unsigned long long mod,
4                                             PollardRho &solver)
5 {
6     assert(a >= 1 && mod >= 1);
7     if (!b) return 1 % mod;
8     auto factors = solver.factor(a);
9     unsigned long long answer = 1 % mod;
10    for (int i = 0, j; i < (int)factors.size(); i = j)
11    {
12        j = i + 1;
13        while (j < (int)factors.size() && factors[j] == factors[i]) j++;
14        __uint128_t length = __uint128_t(j - i) * b + 1;
15        auto sum = power_sum(factors[i], length, mod).second;
16        answer = __uint128_t(answer) * sum % mod;
17    }
18    return answer;
19 }
```

# 第四十六章 单个数的欧拉函数

## 46.1 单个数的欧拉函数

`euler_phi(n)` 使用试除,  $O(\sqrt{n})$  时间、 $O(1)$  额外空间; `euler_phi(n,solver)` 复用 PollardRho, 时间由随机分解主导, 分解结果与递归占  $O(\log n)$  空间。  $1 \leq n \leq \text{ULLONG\_MAX}$ ,  $\text{phi}(1)=1$ 。大素数或平衡大半素数应选分解器入口, 不要用试除穷扫平方根; 批量求  $1..N$  用 LinearSieve。先除质因子再乘  $p-1$ , 结果及中间乘积均不超过原  $n$ 。只抄试除函数时不需要 NumberTheory/PollardRho。

```
1 inline unsigned long long euler_phi(unsigned long long n)
2 {
3     assert(n >= 1);
4     unsigned long long answer = n;
5     for (unsigned long long p = 2; p <= n / p; p++)
6         if (n % p == 0)
7             {
8                 answer = answer / p * (p - 1);
9                 while (n % p == 0) n /= p;
10            }
11     if (n > 1) answer = answer / n * (n - 1);
12     return answer;
13 }
14
15 inline unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
16 {
17     assert(n >= 1);
18     auto factors = solver.factor(n);
19     unsigned long long answer = n;
20     unsigned long long previous = 0;
21     for (auto p : factors)
22         if (p != previous)
23             {
24                 answer = answer / p * (p - 1);
25                 previous = p;
26            }
27     return answer;
28 }
```

## 第四十七章 Carmichael 函数

### 47.1 Carmichael: 模乘法群的公共指数

`carmichael(n)` 返回  $\lambda(n)$ , 即使所有与  $n$  互素的  $a$  都满足  $a$  的  $\lambda(n)$  次幂模  $n$  为 1 的最小正指数; 约定  $\lambda(1)=1$ 。要求  $1 \leq n \leq \text{INT\_MAX}$ 。试除分解后将各素数幂的指数取 lcm, 特别处理 2 的三次及以上幂。时间  $O(\sqrt{n})$ , 额外空间  $O(1)$ ; 所有中间值不超过有符号 64 位范围, 结果不超过  $n$ 。该指数只适用于可逆底数, 不能据此直接对非互素底数的幂降指数。函数不依赖原根模块。

```
1 inline int carmichael(int n)
2 {
3     assert(n >= 1);
4     long long answer = 1;
5     for (int p = 2; p <= n / p; p++)
6         if (n % p == 0)
7             {
8                 int power = 1;
9                 while (n % p == 0)
10                     {
11                         n /= p;
12                         power *= p;
13                     }
14                 int value = power / p * (p - 1);
15                 if (p == 2 && power >= 8) value /= 2;
16                 answer = answer / gcd(answer, (long long)value) * value;
17             }
18     if (n > 1) answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
19     return answer;
20 }
```

## 第四十八章 整数分拆

### 48.1 整数分拆与重数上限

`Partitions(N,mod)` 求无序正整数分拆,  $0 \leq N < \text{INT\_MAX}$ ,  $1 \leq \text{mod} \leq \text{INT\_MAX}$ , `p[n]` 为不限制重数的分拆数; `p[0]=1%mod`。预处理  $O(N\sqrt{N})$  时间、 $O(N)$  空间; 模数不要求素数。`limited(n,k)` 限制每个正整数最多出现  $k$  次,  $0 \leq n \leq N$ ,  $0 \leq k \leq \text{INT\_MAX}$ ;  $k=0$  时仅空分拆存在,  $k=1$  为互异分拆。单次  $O(1+\sqrt{(n/(k+1))})$ , 额外空间  $O(1)$ 。`p/mod` 只读; 中间五边形数与累加采用 `long long`。

```
1 struct Partitions
2 {
3     int mod;
4     vector<int> p;
5
6     Partitions(int n, int mod) : mod(mod), p(n + 1)
7     {
8         assert(n >= 0 && mod >= 1);
9         p[0] = 1 % mod;
10        for (int i = 1; i <= n; i++)
11        {
12            long long value = 0;
13            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
14            {
15                long long a = j * (3 * j - 1) / 2;
16                long long b = j * (3 * j + 1) / 2;
17                int sign = j % 2 ? 1 : -1;
18                value += 1LL * sign * p[i - a];
19                if (b <= i) value += 1LL * sign * p[i - b];
20            }
21            p[i] = (value % mod + mod) % mod;
22        }
23    }
24
25    int limited(int n, int k) const
26    {
27        assert(0 <= n && n < (int)p.size() && k >= 0);
28        long long step = 1LL + k;
29        long long value = p[n];
30        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
31        {
32            long long a = step * (j * (3 * j - 1) / 2);
33            long long b = step * (j * (3 * j + 1) / 2);
34            int sign = j % 2 ? -1 : 1;
35            value += 1LL * sign * p[n - a];
36            if (b <= n) value += 1LL * sign * p[n - b];
37        }
38        return (value % mod + mod) % mod;
39    }
40};
```

## 第四十九章 素数模数组合数

### 49.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回  $C(n,k)$  模 p; n、k 为完整 unsigned long long 范围,  $k > n$  返回 0,  $C(0,0)=1$ 。init(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且  $p \geq 2$ 。预处理时间  $O(p + \log p)$ , 动态版空间  $O(p)$ ; 单次查询  $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常  $p \leq 10^6$ ; 大素数不能因查询下标小就忽略这里整段  $O(p)$  的预处理。fac、ifac 下标为  $0..p-1$ , 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```
1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p) { init(p); }
7
8     void init(int p)
9     {
10         assert(p >= 2);
11         mod = p;
12         fac.resize(p);
13         ifac.resize(p);
14         fac[0] = 1;
15         for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
16         long long a = fac[p - 1], r = 1;
17         for (int e = p - 2; e; e >>= 1)
18         {
19             if (e & 1) r = r * a % p;
20             a = a * a % p;
21         }
22         ifac[p - 1] = r;
23         for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
24     }
```

```
26 int choose(unsigned long long n, unsigned long long k) const
27 {
28     if (k > n) return 0;
29     long long answer = 1;
30     while (n || k)
31     {
32         int a = n % mod, b = k % mod;
33         if (b > a) return 0;
34         answer = answer * fac[a] % mod;
35         answer = answer * ifac[b] % mod;
36         answer = answer * ifac[a - b] % mod;
37         n /= mod;
38         k /= mod;
39     }
40     return answer;
41 }
42 };
```

# 第五十章 合数模数组合数

## 50.1 合数模数组合数 (扩展 Lucas)

ExLucas(mod) 预处理固定模数, choose(n,k) 返回  $C(n,k)$  模 mod; n、k 为 unsigned long long,  $k > n$  返回 0, mod=1 返回 0。适合  $\text{mod} \leq 10^6$  的合数或素数模数, 不能用于  $10^9$  级素数的整段预处理。分解 mod 为互素素数幂  $q = p^e$ , 预处理每段内不含 p 因子的前缀积; unit(n,f) 去除 n! 中全部 p 因子, valuation 统计指数。分子分母指数之差不少于 e 时该项为零, 否则只对互素单位部分求逆, 最后 CRT 合并。预处理  $O(\sqrt{\text{mod}} + \sum q)$ , 空间  $O(\sum (q+1))$ ; 每次查询  $O(\sum (\log_p(n+1) \cdot \log(n+1) + \log q))$ , 求和遍历不同素因子。所有乘法用 long long; 同一对象可重复查询, factors、fac 与辅助函数视为内部数据。

```
1 struct ExLucas
2 {
3     using ull = unsigned long long;
4     using ll = long long;
5
6     struct Factor
7     {
8         int p, q, exponent, start, crt;
9     };
10
11     int mod;
12     vector<int> fac;
13     vector<Factor> factors;
14
15     static ll power(ll a, ull b, int m)
16     {
17         ll result = 1;
18         while (b)
19         {
20             if (b & 1) result = result * a % m;
21             a = a * a % m;
22             b >>= 1;
23         }
24         return result;
25     }
26
27     ExLucas(int m) : mod(m)
28     {
29         assert(m >= 1);
30         int x = m;
31         for (int p = 2; p <= x / p; p++)
32         {
33             if (x % p) continue;
34             int q = 1, e = 0;
35             do
36             {
37                 x /= p;
38                 q *= p;
39                 e++;
40             } while (x % p == 0);
41             add_factor(p, q, e);
```

```

42     }
43     if (x > 1) add_factor(x, x, 1);
44 }
45
46 void add_factor(int p, int q, int e)
47 {
48     int start = fac.size();
49     fac.push_back(1);
50     for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
51     fac.push_back(fac.back());
52     int phi = q - q / p;
53     int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
54     factors.push_back({p, q, e, start, crt});
55 }
56
57 static ull valuation(ull n, int p)
58 {
59     ull result = 0;
60     while (n)
61     {
62         n /= p;
63         result += n;
64     }
65     return result;
66 }
67
68 ll unit(ull n, const Factor &f) const
69 {
70     ll result = 1;
71     while (n)
72     {
73         result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
74         result = result * fac[f.start + n % f.q] % f.q;
75         n /= f.p;
76     }
77     return result;
78 }

```

```

80 int choose(ull n, ull k) const
81 {
82     if (k > n || mod == 1) return 0;
83     ll answer = 0;
84     for (const auto &f : factors)
85     {
86         ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
87         if (e >= ull(f.exponent)) continue;
88         ll denominator = unit(k, f) * unit(n - k, f) % f.q;
89         ll residue =
90             unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
91         residue = residue * power(f.p, e, f.q) % f.q;
92         answer = (answer + residue * f.crt) % mod;
93     }
94     return answer;
95 }
96 };

```

# 第五十一章 二次剩余

## 51.1 素数模平方根 (Cipolla)

`mod_sqrt(n,p)` 返回 `vector<long long>`, 按升序列出全部不同根; 无解返回空向量, 零或模数 2 返回单根。要求  $p$  为素数,  $2 \leq p \leq \text{INT\_MAX}$ ,  $n$  支持完整 `long long` 范围并在入口归一化。先用 Euler 判别;  $p \equiv 3 \pmod{4}$  时直接快速幂, 其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器; 随机性影响寻找非剩余的尝试次数, 不影响结果, 通常期望  $O(\log p)$  时间、 $O(1)$  辅助空间。乘法用 `long long`, 扩域实部的三因子乘积中间先取模; 指数中的  $p+1$  先转 `long long`, 避免 `INT_MAX` 溢出。此版本不处理合数模数及 64 位大素数, 也不用于浮点开方。

```
1 inline vector<long long> mod_sqrt(long long n, int p)
2 {
3     using ll = long long;
4     assert(p >= 2);
5     n = (n % p + p) % p;
6     if (n == 0 || p == 2) return {n};
7     auto power = [p](ll a, ll b)
8     {
9         ll result = 1;
10        while (b)
11        {
12            if (b & 1) result = result * a % p;
13            a = a * a % p;
14            b >>= 1;
15        }
16        return result;
17    };
18    if (power(n, (p - 1) / 2) != 1) return {};
19    ll answer;
20    if (p % 4 == 3)
21        answer = power(n, (ll(p) + 1) / 4);
22    else
23    {
24        static mt19937_64 rng(712367);
25        ll a, w;
26        do
27        {
28            a = rng() % p;
29            w = (a * a % p - n + p) % p;
30        } while (power(w, (p - 1) / 2) != p - 1);
31        using P = pair<ll, ll>;
32        auto mul = [p, w](P x, P y)
33        {
34            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
35            ll imag = (x.first * y.second + x.second * y.first) % p;
36            return P{real, imag};
37        };
38        P result{1, 0}, base{a, 1};
39        ll exponent = (ll(p) + 1) / 2;
40        while (exponent)
41        {
```

```
42         if (exponent & 1) result = mul(result, base);
43         base = mul(base, base);
44         exponent >>= 1;
45     }
46     answer = result.first;
47 }
48 ll other = p - answer;
49 if (answer > other) swap(answer, other);
50 return {answer, other};
51 }
```

## 第五十二章 素数模高次剩余

### 52.1 素数模高次剩余：全部根的紧凑表示

`solve(a,k,p,g)` 求  $x^k \equiv a \pmod{p}$ ;  $p$  必须是 2 到  $1e12$  的素数,  $g$  是模  $p$  原根,  $k$  为正 unsigned long long,  $a$  支持完整 signed long long。无解返回 `nullopt`; 否则得到 `first`、`ratio`、`count`, 全部解为  $\text{first} \cdot \text{ratio}^i \pmod{p}$ ,  $0 \leq i < \text{count}$ , 无重复; `first` 不保证最小。 $a \equiv 0$  时仅有根 0, 返回  $\{0, 1, 1\}$ 。枚举时每次乘 `ratio`, 乘法先转 `int128`; 无需展开全部根即可读取解数。给定原根后, 期望  $O(\sqrt{p})$  时间与空间, 依赖 `DiscreteLog`、`linear_congruence` 及模快速幂。不验证  $p$  的素性或  $g$  的阶; 合数模数不在此接口范围内。

```
1 struct KthResidue
2 {
3     using ll = long long;
4     ll first, ratio, count;
5
6     // Prime p <= 1e12; g is a primitive root of p; k > 0.
7     // All roots: first * ratio^i mod p, 0 <= i < count.
8     static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
9     {
10         assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
11         a %= p;
12         if (a < 0) a += p;
13         if (a == 0) return KthResidue{0, 1, 1};
14         ll logarithm = DiscreteLog::solve(g, a, p);
15         if (logarithm < 0) return nullopt;
16         auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
17         if (exponent < 0) return nullopt;
18         ll first = NumberTheory::power(g, exponent, p);
19         ll ratio = NumberTheory::power(g, period, p);
20         return KthResidue{first, ratio, (p - 1) / period};
21     }
22 };
```

## 第五十三章 素数幂模高次剩余

### 53.1 素数幂模高次剩余：含非互素被开方数

`solve(a,k,p,e,g)` 求  $x^k \equiv a \pmod{p^e}$ 。p 是素数， $e \geq 1$ ， $p^e \leq 1e12$ ，k 为正 unsigned long long；a 为 signed long long。奇素数须提供模  $p^e$  的原根 g；p=2 时不使用 g。无解返回 `nullopt`，否则 `size()` 返回解数，`get(i,j,lift)` 返回指定根，范围是 `count[0]`、`count[1]`、`lifts`，全部组合无重复且无遗漏。字段 `first`、`ratio[2]`、`step` 描述乘法根族与加法提升；`step*lifts=p^e`。原根与素性由调用者保证。期望  $O(\sqrt{p^e})$  时间和空间；单次 `get` 为  $O(\log(p^e))$ ，不强制展开全部根。依赖离散对数、线性同余及模幂。

```
1 struct PrimePowerRoots
2 {
3     using ll = long long;
4     ll first = 0, step = 1, lifts = 1;
5     array<ll, 2> ratio{1, 1}, count{1, 1};
6
7     ll size() const { return count[0] * count[1] * lifts; }
8
9     ll get(ll i, ll j, ll lift) const
10    {
11        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
12        assert(lift >= 0 && lift < lifts);
13        ll x = (__int128)first * NumberTheory::power(ratio[0], i, step) % step;
14        x = (__int128)x * NumberTheory::power(ratio[1], j, step) % step;
15        return x + lift * step;
16    }
17
18    // p prime, 1 <= e, p^e <= 1e12, k > 0.
19    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.
```

接上页同一结构体（第三页继续同一函数）:

```
20 static optional<PrimePowerRoots>
21 solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
22 {
23     assert(p >= 2 && p <= 10000000000000LL && e >= 1 && e <= 39 && k > 0);
24     array<ll, 40> power{1};
25     for (int i = 1; i <= e; i++)
26     {
27         assert(power[i - 1] <= 10000000000000LL / p);
28         power[i] = power[i - 1] * p;
29     }
30     ll mod = power[e];
31     a %= mod;
32     if (a < 0) a += mod;
33     PrimePowerRoots answer;
34     if (a == 0)
35     {
36         answer.step = power[(e - 1) / k + 1];
37         answer.lifts = mod / answer.step;
38         return answer;
39     }
40     int t = 0;
41     while (a % p == 0)
42     {
43         a /= p;
44         ++t;
45     }
46     if (t % k) return nullopt;
```

接上页同一结构体（第三页继续同一函数）：

```

47     ll scale = power[t / k], q = power[e - t];
48     answer.first = 1;
49     answer.step = scale * q;
50     answer.lifts = mod / answer.step;
51     auto cyclic = [&](ll base, ll target, ll order, int slot)
52     {
53         ll logarithm = DiscreteLog::solve(base, target, q);
54         if (logarithm < 0) return false;
55         auto [x, period] = linear_congruence(k % order, -logarithm, order);
56         if (x < 0) return false;
57         answer.first = (__int128)answer.first * NumberTheory::power(base, x, q) % q;
58         answer.ratio[slot] = NumberTheory::power(base, period, q);
59         answer.count[slot] = order / period;
60         return true;
61     };
62     if (p != 2)
63     {
64         assert(g >= 1 && g < mod);
65         if (!cyclic(g % q, a, q / p * (p - 1), 0)) return nullopt;
66     }
67     else if (q >= 4)
68     {
69         int sign = a % 4 == 3;
70         auto [x, period] = linear_congruence(k % 2, -sign, 2);
71         if (x < 0) return nullopt;
72         answer.first = x ? q - 1 : 1;
73         answer.ratio[0] = period == 1 ? q - 1 : 1;
74         answer.count[0] = 2 / period;
75         if (q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1)) return nullopt;
76     }
77     answer.first *= scale;
78     return answer;
79 }
80 };

```

## 第五十四章 合数求根的准备

### 54.1 合数求根准备：分解模数与素数幂原根

`root_factors(M, rho)` 为  $1 \leq M \leq 1e12$  生成完整的  $(p, e, g)$  列表，可直接传入 `CompositeRoots`。rho 是可复用的 `PollardRho` 对象。按素数递增返回； $p=2$  时  $g=0$ ，其余  $g$  为模  $p^e$  的最小正原根。先分解  $M$ ，再分解各个欧拉函数值，逐个检查原根候选的阶。 $M=1$  返回空列表。分解使用随机重试；原根搜索耗时取决于候选数，不包含 BSGS。

```
1 inline vector<tuple<long long, int, long long>> root_factors(long long mod,
2                                                                PollardRho &rho)
3 {
4     assert(mod >= 1 && mod <= 1000000000000LL);
5     auto primes = rho.factor(mod);
6     vector<tuple<long long, int, long long>> answer;
7     for (int i = 0; i < (int)primes.size();)
8     {
9         long long p = primes[i], power = 1;
10        int e = 0;
11        while (i < (int)primes.size() && primes[i] == (unsigned long long)p)
12        {
13            power *= p;
14            ++e;
15            ++i;
16        }
17        long long g = 0;
18        if (p != 2)
19        {
20            long long phi = power / p * (p - 1);
21            auto factors = rho.factor(phi);
22            factors.erase(unique(factors.begin(), factors.end()), factors.end());
23            auto primitive = [&](long long candidate)
24            {
25                if (candidate % p == 0) return false;
26                for (auto q : factors)
27                    if (NumberTheory::power(candidate, phi / q, power) == 1)
28                        return false;
29                return true;
30            };
31            g = 2;
32            while (!primitive(g)) ++g;
33        }
34        answer.push_back({p, e, g});
35    }
36    return answer;
37 }
```

## 第五十五章 合数模高次剩余

### 55.1 一般合数模高次剩余：CRT 组合与按索引取根

`solve(a,k,factors)` 求  $x^k \equiv a \pmod{M}$ , `factors` 是完整且素数互异的  $(p,e,g)$  列表,  $M$  为所有  $p^e$  的乘积,  $M \leq 1e12$ ,  $k$  为正 `unsigned long long`,  $a$  为 `signed long long`。奇素数的  $g$  是模  $p^e$  原根;  $p=2$  时填 0。无解返回 `nullopt`, 否则 `mod=M`, `total` 为解数, `get(index)` 在  $0 \leq \text{index} < \text{total}$  上取出每个根恰好一次, 不按数值排序。空因子列表表示模 1, 唯一标准代表为 0。依赖 `PrimePowerRoots` 和模逆元; 列表接口不做分解; 重载 `solve(a,k,M,rho)` 自动分解并寻找原根,  $\rho$  为 PollardRho。重复同模数宜复用因子列表。预处理的主要成本是各素数幂的 BSGS; `get` 为  $O(F \log M)$ ,  $F$  为不同素因子数。无需展开全部根, `parts` 和 `weight` 保存局部根族及 CRT 系数。

```
1 struct CompositeRoots
2 {
3     using ll = long long;
4     ll mod = 1, total = 1;
5     vector<PrimePowerRoots> parts;
6     vector<ll> weight;
7
8     ll get(ll index) const
9     {
10         assert(index >= 0 && index < total);
11         ll answer = 0;
12         for (int t = 0; t < (int)parts.size(); t++)
13         {
14             const auto &p = parts[t];
15             ll local = index % p.size();
16             index /= p.size();
17             ll lift = local % p.lifts;
18             local /= p.lifts;
19             ll j = local % p.count[1], i = local / p.count[1];
20             ll root = p.get(i, j, lift);
21             answer = (answer + (__int128)root * weight[t]) % mod;
22         }
23         return answer;
24     }
25
26     // Factors are distinct primes with positive exponents: (p, e, g).
27     // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.
```

局部求根与 CRT 系数（接上页同一结构体）：

```

28     static optional<CompositeRoots>
29     solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
30     {
31         assert(k > 0);
32         CompositeRoots answer;
33         for (auto [p, e, g] : factors)
34         {
35             auto roots = PrimePowerRoots::solve(a, k, p, e, g);
36             if (!roots) return nullopt;
37             ll m = roots->step * roots->lifts;
38             assert(gcd(answer.mod, m) == 1);
39             assert(answer.mod <= 1000000000000LL / m);
40             answer.mod *= m;
41             answer.total += roots->size();
42             answer.parts.push_back(*roots);
43         }
44         for (const auto &p : answer.parts)
45         {
46             ll m = p.step * p.lifts, rest = answer.mod / m;
47             ll inverse = NumberTheory::inverse(rest % m, m);
48             answer.weight.push_back((__int128)rest * inverse % answer.mod);
49         }
50         return answer;
51     }
52
53     static optional<CompositeRoots>
54     solve(ll a, unsigned long long k, ll mod, PollardRho &rho)
55     {
56         return solve(a, k, root_factors(mod, rho));
57     }
58 };

```

# 第五十六章 多项式插值

## 56.1 拉格朗日插值与连续点优化

先声明 Lagrange。init(xs,ys,p) 使用任意横坐标, xs、ys 为等长 0-based vector<long long>; consecutive(ys,p) 专用于横坐标 0..n-1。两者均覆盖旧数据。要求 p 为素数,  $2 \leq p \leq \text{INT\_MAX}$ , 点数  $n \leq p$ , 任意横坐标取模后两两不同; 不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值, 允许 k、xs、ys 为完整 long long 范围, 内部先归一化; 空数据约定零多项式。通用预处理  $O(n^2 + n \log p)$ , 连续点预处理  $O(n + \log p)$ , 每次查询  $O(n)$ , 空间  $O(n)$ 。查询利用前缀积与滚动后缀积, 无需求逆, 因此 k 与样本点模 p 相等时也正确。若求  $1^d + \dots + k^d$ , 要求  $0 \leq d \leq p-2$ , 先计算 0..d+1 的前缀和作为 ys, 再用 consecutive; 用快速幂生成这些样本另花  $O(d \log(d+1))$ 。仅处理素数域中的精确插值, 不适用于浮点曲线拟合或重复节点的导数约束。

```
1 struct Lagrange
2 {
3     using ll = long long;
4     int n, mod;
5     vector<ll> x, w;
6
7     ll norm(ll a) const
8     {
9         a %= mod;
10        if (a < 0) a += mod;
11        return a;
12    }
13
14    ll power(ll a, int b) const
15    {
16        ll result = 1;
17        while (b)
18        {
19            if (b & 1) result = result * a % mod;
20            a = a * a % mod;
21            b >>= 1;
22        }
23        return result;
24    }
25
26    void init(const vector<ll> &xs, const vector<ll> &ys, int p)
27    {
28        n = ys.size();
29        mod = p;
30        assert(p >= 2 && n <= p && xs.size() == ys.size());
31        x.resize(n);
32        w.resize(n);
33        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
34        for (int i = 0; i < n; i++)
35        {
36            ll denominator = 1;
37            for (int j = 0; j < n; j++)
38                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
39            assert(denominator != 0);
```

```

40     w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
41 }
42 }
43
44 void consecutive(const vector<ll> &ys, int p)
45 {
46     n = ys.size();
47     mod = p;
48     assert(p >= 2 && n <= p);
49     x.resize(n);
50     w.resize(n);
51     if (!n) return;
52     vector<ll> invfac(n, 1);
53     ll factorial = 1;
54     for (int i = 1; i < n; i++) factorial = factorial * i % mod;
55     invfac[n - 1] = power(factorial, mod - 2);
56     for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
57     for (int i = 0; i < n; i++)
58     {
59         x[i] = i;
60         w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
61         if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
62     }
63 }
64
65 ll query(ll k) const
66 {
67     k = norm(k);
68     vector<ll> pre(n + 1, 1);
69     for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
70     ll answer = 0, suffix = 1;
71     for (int i = n - 1; i >= 0; i--)
72     {
73         answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
74         suffix = suffix * norm(k - x[i]) % mod;
75     }
76     return answer;
77 }
78 };

```

## 第五十七章 参数化 NTT 卷积

### 57.1 可指定素数与原根的 NTT 卷积

声明 `NttConvolution<mod, primitive>`, `primitive` 默认 3。mod 必须为 `int` 范围内的素数, `primitive` 是模 `mod` 原根; 不在接口内验证素性或阶。`max_size` 是 `mod-1` 的最大二次幂因子。`ntt(a,invert)` 原地变换, 长度须为正二次幂且不超过 `max_size`; `invert=true` 为逆变换。`multiply(a,b)` 返回普通卷积, 任一为空则为空; 非空时长度和减一不超过 `max_size`。元素类型 `Z`, 数组类型 `Poly`。 $O(n \log n)$  时间、 $O(n)$  空间。998244353 和 167772161 均可用原根 3。

```
1 template <int mod, int primitive = 3> struct NttConvolution
2 {
3     static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
4     static constexpr int max_size = (mod - 1) & -(mod - 1);
5     using Z = ModInt<mod>;
6     using Poly = vector<Z>;
7
8     // mod is prime, primitive is a primitive root modulo mod.
9     static void ntt(Poly &a, bool invert = false)
10    {
11        int n = a.size();
12        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
13        for (int i = 1, j = 0; i < n; i++)
14        {
15            int bit = n >> 1;
16            for (; j & bit; bit >>= 1) j ^= bit;
17            j ^= bit;
18            if (i < j) swap(a[i], a[j]);
19        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```

20     for (int half = 1; half < n; half *= 2)
21     {
22         int len = half * 2;
23         Z step = Z(primitive).pow((mod - 1) / len);
24         if (invert) step = step.inv();
25         for (int i = 0; i < n; i += len)
26         {
27             Z w = 1;
28             for (int j = 0; j < half; j++)
29             {
30                 Z u = a[i + j], v = a[i + j + half] * w;
31                 a[i + j] = u + v;
32                 a[i + j + half] = u - v;
33                 w = w * step;
34             }
35         }
36     }
37     if (invert)
38     {
39         Z inverse = Z(n).inv();
40         for (auto &x : a) x = x * inverse;
41     }
42 }
43
44 static Poly multiply(Poly a, Poly b)
45 {
46     if (a.empty() || b.empty()) return {};
47     assert(a.size() + b.size() - 1 <= max_size);
48     int size = a.size() + b.size() - 1, n = 1;
49     while (n < size) n *= 2;
50     a.resize(n);
51     b.resize(n);
52     ntt(a);
53     ntt(b);
54     for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
55     ntt(a, true);
56     a.resize(size);
57     return a;
58 }
59 };

```

# 第五十八章 多项式平移

## 58.1 多项式系数平移

`polynomial_shift<mod=998244353,primitive=3>(f,c)` 返回  $f(x+c)$  的升幂系数，结果长度始终与  $f$  相同，保留末尾零；空输入返回空。参数  $c$  使用 `ModInt<mod>`。依赖参数化 NTT， $\text{mod}$  为素数且  $\text{primitive}$  为原根；非空长度  $n$  要求  $2n-1 \leq \text{NTT::max\_size}$ ，因此阶乘均可逆。逆序的  $f[i]!$  与  $c^i/i!$  卷积后提取对应项， $O(n \log n)$  时间、 $O(n)$  空间。 $c=0$  合法。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
3 {
4     using N = NttConvolution<mod, primitive>;
5     using Z = typename N::Z;
6     using Poly = typename N::Poly;
7     if (f.empty()) return {};
8     assert(f.size() <= (N::max_size + 1ULL) / 2);
9     int n = f.size();
10    Poly fact(n), inv(n), a(n), b(n);
11    fact[0] = 1;
12    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
13    inv[n - 1] = fact[n - 1].inv();
14    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
15    Z power = 1;
16    for (int i = 0; i < n; i++)
17    {
18        a[n - 1 - i] = f[i] * fact[i];
19        b[i] = power * inv[i];
20        power = power * c;
21    }
22    auto product = N::multiply(a, b);
23    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
24    return a;
25 }
```

# 第五十九章 等比点求值

## 59.1 Chirp Z：等比点求值

`chirp_z(f,a,r,m)` 返回  $f(a \cdot r^i)$ ,  $i=0..m-1$ ;  $f$  为升幂系数数组,  $a, r$  为 `ModInt<mod>`。  $m \geq 0$ ,  $f.size() \leq INT\_MAX$ ; 空  $f$  表示零多项式,  $m=0$  返回空。约定  $r^0=1$ , 因此  $r=0$  时首项为  $f(a)$ 、其余为  $f(0)$ ;  $a=0$  或  $r=1$  也单独处理。普通分支依赖 NTT 素数及原根, 非空长度  $n$  要求  $n+m-1 \leq NTT::max\_size$ 。利用三角数指数与循环卷积, 避免平方指数的整数溢出;  $O((n+m) \log(n+m))$  时间、 $O(n+m)$  空间, 特殊分支  $O(n+m)$ 。默认模板参数 `mod=998244353`, `primitive=3`。输出顺序允许重复点。

```
1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>>
3 chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
4 {
5     using N = NttConvolution<mod, primitive>;
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     assert(m >= 0 && f.size() <= INT_MAX);
9     if (m == 0 || f.empty()) return Poly(m);
10    if (a.v == 0 || r.v == 0 || r.v == 1)
11    {
12        Poly answer(m, f[0]);
13        Z value = 0;
14        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
15        answer[0] = value;
16        if (r.v == 1) fill(answer.begin(), answer.end(), value);
17        return answer;
18    }
19    assert(f.size() + m - 1 <= N::max_size);
20    int n = f.size(), size = 1;
21    while (size < n + m - 1) size *= 2;
22    Poly x(size), y(size), inverse(max(n, m));
23    Z power = 1, triangle = 1, ri = r.inv();
24    for (auto &v : inverse)
25    {
26        v = triangle;
27        triangle = triangle * power;
28        power = power * ri;
29    }
```

构造循环卷积并取点值（接上页同一函数）：

```
30     power = 1;
31     for (int j = 0; j < n; j++)
32     {
33         x[n - 1 - j] = f[j] * power * inverse[j];
34         power = power * a;
35     }
36     power = 1;
37     triangle = 1;
38     for (int j = 0; j < n + m - 1; j++)
39     {
40         y[j] = triangle;
41         triangle = triangle * power;
42         power = power * r;
43     }
44     N::ntt(x);
45     N::ntt(y);
46     for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
47     N::ntt(x, true);
48     Poly answer(m);
49     for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
50     return answer;
51 }
```

## 第六十章 斯特林数整行计算

### 60.1 第二类斯特林数：同一行前缀

`stirling_second_row<mod,primitive>(n,m)` 返回  $S(n,0..m)$ ，默认 `mod=998244353`、`primitive=3`。模数必须是素数，`primitive` 是原根。`n` 支持完整 `unsigned long long`；`2m+1` 不能超过 NTT 的 `max_size`。 $S(n,k)$  为无标号非空集合划分数；`n=0` 时仅第零项为 1，`k>n` 为零。一次卷积配合逆阶乘， $O(m \log m + m \log \text{mod})$  时间、 $O(m)$  空间。依赖 `NttConvolution`；模 167772161 可直接使用模板参数 `<167772161>`。

```
1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(m >= 0 && m <= (N::max_size - 1) / 2);
6     using Z = typename N::Z;
7     typename N::Poly a(m + 1), b(m + 1);
8     b[0] = 1;
9     for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
10    b[m] = b[m].inv();
11    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
12    for (int i = 0; i <= m; i++)
13    {
14        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
15        a[i] = value * b[i];
16        if (i % 2) b[i] = Z(0) - b[i];
17    }
18    auto answer = N::multiply(a, b);
19    answer.resize(m + 1);
20    return answer;
21 }
```

## 60.2 无符号第一类斯特林数：同一行

`stirling_first_row<mod,primitive>(n)` 返回无符号  $c(n,0..n)$ ，默认模 998244353、原根 3。 $c(n,k)$  为  $n$  元排列恰有  $k$  个循环的个数。 $n \geq 0$ ， $n+1 \leq \text{NTT 的 max\_size}$ ； $n=0$  返回  $\{1\}$ 。倍增配合多项式平移计算  $x(x+1)\cdots(x+n-1)$ ， $O(n \log n)$  时间、 $O(n)$  空间。带符号版本再乘  $(-1)^{(n-k)}$ 。依赖 `NttConvolution`；模 167772161 可用 `<167772161>`。

```

1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_first_row(int n)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(n >= 0 && n < N::max_size);
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     Poly fact(n + 1), inv(n + 1);
9     fact[0] = 1;
10    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
11    inv[n] = fact[n].inv();
12    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
13    function<Poly(int)> solve = [&](int len) -> Poly
14    {
15        if (len == 0) return {1};
16        int m = len / 2;
17        Poly f = solve(m);
18        Poly a(m + 1), b(m + 1);
19        Z power = 1;
20        for (int i = 0; i <= m; i++)
21        {
22            a[m - i] = f[i] * fact[i];
23            b[i] = power * inv[i];
24            power = power * m;
25        }
26        auto c = N::multiply(a, b);
27        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
28        f = N::multiply(f, a);
29        if (len % 2)
30        {
31            f.push_back(0);
32            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
33            f[0] = f[0] * (len - 1);
34        }
35        return f;
36    };
37    return solve(n);
38 }
```

# 第六十一章 集合变换与卷积

## 61.1 按位 OR、AND、XOR 卷积

SetConvolution<mod=998244353>; Z 为 ModInt<mod>, Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且  $\leq \text{INT\_MAX}$ , 表示固定二进制位数的全集, 长度 1 合法。transform(a,op,invert=false) 原地变换, op 只能为 |、&、^; | 为子集和, & 为超集和, ^ 为 Walsh 变换。multiply(a,b,op) 返回  $\sum a[i]b[j]$  (满足  $i \text{ op } j = s$ ), a、b 长度必须相同。mod 可为  $\geq 2$  的 int 范围合数; XOR 要求 mod 为奇数, 逆 2 用  $(\text{mod}+1)/2$ , 不使用费马逆元。 $O(N \log N)$  时间, 原地变换  $O(1)$  额外空间, 卷积  $O(N)$  空间。

```
1 template <int mod = 998244353> struct SetConvolution
2 {
3     static_assert(mod >= 2);
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void transform(Poly &a, char op, bool invert = false)
8     {
9         assert(!a.empty() && a.size() <= INT_MAX);
10        int n = a.size();
11        assert((n & (n - 1)) == 0);
12        assert(op == '|' || op == '&' || op == '^');
13        assert(op != '^' || mod % 2 == 1);
14        Z half = (mod + 1LL) / 2;
15        for (int len = 1; len < n; len *= 2)
16            for (int i = 0; i < n; i += len * 2)
17                for (int j = 0; j < len; j++)
18                {
19                    Z &u = a[i + j], &v = a[i + j + len];
20                    if (op == '|')
21                        v = invert ? v - u : v + u;
22                    else if (op == '&')
23                        u = invert ? u - v : u + v;
24                    else
25                    {
26                        Z x = u, y = v;
27                        u = x + y;
28                        v = x - y;
29                        if (invert)
30                        {
31                            u = u * half;
32                            v = v * half;
33                        }
34                    }
35                }
36    }
```

点乘并逆变换（接上页同一结构体）：

```
38     static Poly multiply(Poly a, Poly b, char op)
39     {
40         assert(a.size() == b.size());
41         transform(a, op);
42         transform(b, op);
43         for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
44         transform(a, op, true);
45         return a;
46     }
47 };
```

## 第六十二章 不相交子集卷积

### 62.1 不相交子集卷积

`subset_convolution<mod=998244353>(a,b)` 返回  $c[s] = \sum a[t]b[s \text{ xor } t]$ , 求和枚举  $t \subseteq s$ , 等价于  $i|j=s$  且  $i \& j=0$ 。依赖 `SetConvolution`, 只使用 OR 变换; `mod` 为  $\geq 2$  的 `int` 范围整数, 允许偶数与合数。两个数组长度相同、非空,  $N=2^m \leq \text{INT\_MAX}$ ; 长度 1 的结果为  $a[0]b[0]$ 。按 `popcount` 分层, 层间普通卷积再逆 OR 变换, 保留总层数等于目标集合大小的项。 $O(m^2 2^m)$  时间、 $O(m 2^m)$  空间; 两组分层数组约占  $8(m+1)2^m$  字节, 另有线性辅助数组。 $m=20$  时仅分层数组约 168 MiB, 使用前核对内存。

```
1 // c[s] = sum_{t subset s} a[t] * b[s xor t].
2 template <int mod = 998244353>
3 vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
4                                         const vector<ModInt<mod>> &b)
5 {
6     using F = SetConvolution<mod>;
7     using Poly = typename F::Poly;
8     assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
9     int n = a.size(), m = 0;
10    assert((n & (n - 1)) == 0);
11    while ((1 << m) < n) m++;
12    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
13    vector<int> rank(n);
14    for (int s = 0; s < n; s++)
15    {
16        rank[s] = __builtin_popcount(unsigned(s));
17        f[rank[s]][s] = a[s];
18        g[rank[s]][s] = b[s];
19    }
20    for (int k = 0; k <= m; k++)
21    {
22        F::transform(f[k], '|');
23        F::transform(g[k], '|');
24    }
25    Poly answer(n), h(n);
26    for (int k = 0; k <= m; k++)
27    {
28        fill(h.begin(), h.end(), 0);
29        for (int i = 0; i <= k; i++)
30            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
31        F::transform(h, '|', true);
32        for (int s = 0; s < n; s++)
33            if (rank[s] == k) answer[s] = h[s];
34    }
35    return answer;
36 }
```

# 第六十三章 多项式

## 63.1 NTT、形式幂级数、FWT 与递推

依赖 number\_theory.hpp。模数固定 998244353，乘法结果长度  $\leq 2^{23}$ ；inverse 要求  $a[0] \neq 0$ ，log 要求  $a[0]=1$ ，exp 要求  $a[0]=0$ ，截断长度  $n>0$ 。朴素逐层 NTT 实现的 FPS 为  $O(n \log n)$ 。FWT 的 op 为  $\wedge$ 、 $|$ 、 $\&$ ；独立且支持其他模数的接口见 SetConvolution。BM 返回  $s[n]=\sum c[i-1]s[n-i]$  的系数；recurrence 用  $O(k^2 \log n)$  求第  $n$  项。BM 对有限样本给出的递推不自动保证未知后续项成立。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void ntt(Poly &a, bool invert = false)
8     {
9         int n = (int)a.size();
10        assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
11        for (int i = 1, j = 0; i < n; i++)
12        {
13            int bit = n >> 1;
14            for (; j & bit; bit >>= 1) j ^= bit;
15            j ^= bit;
16            if (i < j) swap(a[i], a[j]);
17        }
18        for (int len = 2; len <= n; len *= 2)
19        {
20            Z step = Z(3).pow((mod - 1) / len);
21            if (invert) step = step.inv();
22            for (int i = 0; i < n; i += len)
23            {
24                Z w = 1;
25                for (int j = 0; j < len / 2; j++, w = w * step)
26                {
27                    Z u = a[i + j], v = a[i + j + len / 2] * w;
28                    a[i + j] = u + v;
29                    a[i + j + len / 2] = u - v;
30                }
31            }
32        }
33        if (invert)
34        {
35            Z inv = Z(n).inv();
36            for (auto &x : a) x = x * inv;
37        }
38    }
39
40    static Poly multiply(Poly a, Poly b)
41    {
42        if (a.empty() || b.empty()) return {};
43        int size = (int)a.size() + (int)b.size() - 1, n = 1;
```

```

44     assert(size <= (1 << 23));
45     while (n < size) n *= 2;
46     a.resize(n);
47     b.resize(n);
48     ntt(a);
49     ntt(b);
50     for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
51     ntt(a, true);
52     a.resize(size);
53     return a;
54 }
55
56 static Poly inverse(const Poly &a, int n)
57 {
58     assert(n > 0 && !a.empty() && a[0].v);
59     Poly b{a[0].inv()};
60     while ((int)b.size() < n)
61     {
62         int m = min(n, 2 * (int)b.size());
63         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
64         auto t = multiply(f, b);
65         t.resize(m);
66         for (auto &x : t) x = Z(0) - x;
67         t[0] = t[0] + 2;
68         b = multiply(b, t);
69         b.resize(m);
70     }
71     return b;
72 }
73
74 static Poly derivative(const Poly &a)
75 {
76     Poly b;
77     for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
78     return b;
79 }
80
81 static Poly integral(const Poly &a)
82 {
83     Poly b(a.size() + 1);
84     vector<Z> inv(a.size() + 1);
85     if (!a.empty()) inv[1] = 1;
86     for (int i = 2; i <= (int)a.size(); i++)
87         inv[i] = Z(mod - mod / i) * inv[mod % i];
88     for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
89     return b;
90 }
91
92 static Poly log(const Poly &a, int n)
93 {
94     assert(n > 0 && !a.empty() && a[0].v == 1);
95     auto b = multiply(derivative(a), inverse(a, n));
96     b.resize(n - 1);
97     return integral(b);
98 }
99

```

```

100 static Poly exp(const Poly &a, int n)
101 {
102     assert(n > 0 && (a.empty() || a[0].v == 0));
103     Poly b{1};
104     while ((int)b.size() < n)
105     {
106         int m = min(n, 2 * (int)b.size());
107         auto t = log(b, m);
108         for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
109         t[0] = t[0] + 1;
110         b = multiply(b, t);
111         b.resize(m);
112     }
113     return b;
114 }
115
116 static void fwt(Poly &a, char op, bool invert = false)
117 {
118     int n = (int)a.size();
119     assert(n > 0 && (n & (n - 1)) == 0);
120     const Z inv2 = (mod + 1) / 2;
121     for (int len = 1; len < n; len *= 2)
122         for (int i = 0; i < n; i += len * 2)
123             for (int j = 0; j < len; j++)
124             {
125                 Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
126                 if (op == '^')
127                 {
128                     u = x + y;
129                     v = x - y;
130                     if (invert)
131                     {
132                         u = u * inv2;
133                         v = v * inv2;
134                     }
135                 }
136                 else if (op == '|')
137                     v = invert ? y - x : y + x;
138                 else
139                 {
140                     assert(op == '&');
141                     u = invert ? x - y : x + y;
142                 }
143             }
144 }
145
146 // c: s[n]=sum c[i-1]*s[n-i], i=1..L.
147 static Poly berlekamp_massey(const Poly &s)
148 {
149     Poly c{1}, b{1};
150     int l = 0, m = 1;
151     Z last = 1;
152     for (int n = 0; n < (int)s.size(); n++)
153     {
154         Z d = s[n];
155         for (int i = 1; i <= l; i++) d = d + c[i] * s[n - i];

```

```

156         if (!d.v)
157         {
158             ++m;
159             continue;
160         }
161         auto old = c;
162         Z coef = d / last;
163         if (c.size() < b.size() + m) c.resize(b.size() + m);
164         for (int j = 0; j < (int)b.size(); j++) c[j + m] = c[j + m] - coef * b[j];
165         if (2 * l <= n)
166         {
167             l = n + 1 - l;
168             b = old;
169             last = d;
170             m = 1;
171         }
172         else
173             ++m;
174     }
175     c.erase(c.begin());
176     for (auto &x : c) x = Z(0) - x;
177     return c;
178 }
179
180 static Z recurrence(Poly init, const Poly &c, unsigned long long n)
181 {
182     int k = (int)c.size();
183     if (n < init.size()) return init[n];
184     if (!k) return 0;
185     assert((int)init.size() >= k);
186     auto combine = [&](const Poly &a, const Poly &b)
187     {
188         Poly t(2 * k - 1);
189         for (int i = 0; i < k; i++)
190             for (int j = 0; j < k; j++) t[i + j] = t[i + j] + a[i] * b[j];
191         for (int i = 2 * k - 2; i >= k; i--)
192             for (int j = 1; j <= k; j++) t[i - j] = t[i - j] + t[i] * c[j - 1];
193         t.resize(k);
194         return t;
195     };
196     Poly a(k), r(k);
197     r[0] = 1;
198     if (k == 1)
199         a[0] = c[0];
200     else
201         a[1] = 1;
202     for (; n >= 1, a = combine(a, a))
203         if (n & 1) r = combine(r, a);
204     Z ans = 0;
205     for (int i = 0; i < k; i++) ans = ans + r[i] * init[i];
206     return ans;
207 }
208 };

```

# 第六十四章 代数与数论进阶

## 64.1 模高斯消元、行列式、矩阵树定理

素数模数。solve(a,n) 输入增广矩阵，返回 consistent、rank、特解 particular 和零空间基 kernel。determinant(empty)=1。spanning\_trees(n,edges) 使用 0-based 无向图，允许重边，忽略自环；返回生成树数模 p。高斯与行列式  $O(n^3)$ ，矩阵幂  $O(n^3 \log e)$ 。

```
1 template <int mod> struct LinearAlgebra
2 {
3     using Z = ModInt<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for (const auto &v : a) assert((int)v.size() == n + 1);
20        for (int col = 0; col < n && row < m; col++)
21        {
22            int p = row;
23            while (p < m && !a[p][col].v) ++p;
24            if (p == m) continue;
25            swap(a[p], a[row]);
26            Z inv = a[row][col].inv();
27            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
28            for (int i = 0; i < m; i++)
29                if (i != row && a[i][col].v)
30                {
31                    Z f = a[i][col];
32                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
33                }
34            where[col] = row++;
35        }
36        for (int i = row; i < m; i++)
37            if (a[i][n].v) return {false, row, {}, {}};
38        Solution ans{true, row, vector<Z>(n), {}};
39        for (int j = 0; j < n; j++)
40            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
41        for (int j = 0; j < n; j++)
42            if (where[j] == -1)
43            {
44                vector<Z> v(n);
```

```

45         v[j] = 1;
46         for (int k = 0; k < n; k++)
47             if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
48         ans.kernel.push_back(v);
49     }
50     return ans;
51 }
52
53 static Z determinant(Matrix a)
54 {
55     int n = (int)a.size();
56     Z ans = 1;
57     for (int i = 0; i < n; i++)
58     {
59         assert((int)a[i].size() == n);
60         int p = i;
61         while (p < n && !a[p][i].v) ++p;
62         if (p == n) return 0;
63         if (p != i)
64         {
65             swap(a[p], a[i]);
66             ans = Z(0) - ans;
67         }
68         ans = ans * a[i][i];
69         Z inv = a[i][i].inv();
70         for (int j = i + 1; j < n; j++)
71         {
72             Z f = a[j][i] * inv;
73             for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
74         }
75     }
76     return ans;
77 }
78
79 static Matrix multiply(const Matrix &a, const Matrix &b)
80 {
81     assert(!a.empty() && !b.empty());
82     int n = a.size(), m = b[0].size(), k = b.size();
83     assert((int)a[0].size() == k);
84     Matrix c(n, vector<Z>(m));
85     for (int i = 0; i < n; i++)
86         for (int t = 0; t < k; t++)
87             for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
88     return c;
89 }
90
91 static Matrix power(Matrix a, unsigned long long e)
92 {
93     int n = a.size();
94     assert(n > 0 && (int)a[0].size() == n);
95     Matrix r(n, vector<Z>(n));
96     for (int i = 0; i < n; i++) r[i][i] = 1;
97     for (; e >= 1, a = multiply(a, a))
98         if (e & 1) r = multiply(r, a);
99     return r;
100 }

```

```
101
102 // Undirected multigraph, vertices 0..n-1; loops ignored.
103 static Z spanning_trees(int n, const vector<pair<int, int>> &edges)
104 {
105     assert(n > 0);
106     Matrix lap(n, vector<Z>(n));
107     for (auto [u, v] : edges)
108         if (u != v)
109             {
110                 lap[u][u] = lap[u][u] + 1;
111                 lap[v][v] = lap[v][v] + 1;
112                 lap[u][v] = lap[u][v] - 1;
113                 lap[v][u] = lap[v][u] - 1;
114             }
115     lap.pop_back();
116     for (auto &row : lap) row.pop_back();
117     return determinant(lap);
118 }
119 };
```

## 64.2 杜教筛

构造预筛上限  $1 \leq B < \text{INT\_MAX}$ ，内存须可用； $\text{mertens}(n)$  返回  $\mu$  的前缀和， $\text{totient\_sum}(n)$  返回  $\varphi$  的前缀和 ( $\text{int128}$ )。约定  $0 \leq n \leq 10^{11}$ ， $n=0$  返回 0。按整除分块递归并写入  $\text{mmu}/\text{mphi}$  缓存；重复查询复用结果，清空缓存可重算，不可修改  $\text{pmu}/\text{pphi}/\text{limit}$ 。单次最大参数  $N$  的常用均摊分析为  $O(B+N/\sqrt{B})$ （哈希按期望常数计算）， $B$  约  $N^{2/3}$  时为  $O(N^{2/3})$ ；小  $B$  不构成大输入性能保证。多次不同  $N$  可能新增缓存，空间随实际缓存增长；递归深度  $O(\log N)$ 。

```

1 struct DuJiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        LinearSieve s(limit);
14        for (int i = 1; i <= limit; i++)
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll mertens(ll n)
22    {
23        if (n <= limit) return pmu[n];
24        if (mmu.count(n)) return mmu[n];
25        ll ans = 1;
26        for (ll l = 2, r; l <= n; l = r + 1)
27        {
28            r = n / (n / l);
29            ans -= (r - l + 1) * mertens(n / l);
30        }
31        return mmu[n] = ans;
32    }
33
34    I totient_sum(ll n)
35    {
36        if (n <= limit) return pphi[n];
37        if (mphi.count(n)) return mphi[n];
38        I ans = I(n) * (n + 1) / 2;
39        for (ll l = 2, r; l <= n; l = r + 1)
40        {
41            r = n / (n / l);
42            ans -= I(r - l + 1) * totient_sum(n / l);
43        }
44        return mphi[n] = ans;
45    }
46 };

```

## 64.3 扩展 BSGS

依赖 number\_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小  $x \geq 0$  使  $a^x \equiv b \pmod{m}$ ，无解返回 -1； $1 \leq m \leq 10^{12}$ ，a、b 可为完整 long long 范围，入口归一化，约定  $0^0=1$ 。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间  $O(\sqrt{m} + \log m)$ ，空间  $O(\sqrt{m})$ ，哈希表先 reserve；基于 unordered\_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```

1 struct DiscreteLog
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
6     static ll solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if (m == 1 || b == 1) return 0;
12        if (a == 1) return -1;
13        ll offset = 0, k = 1;
14        for (ll g; (g = gcd(a, m)) > 1;)
15        {
16            if (b == k) return offset;
17            if (b % g) return -1;
18            b /= g;
19            m /= g;
20            k = (__int128)k * (a / g) % m;
21            ++offset;
22        }
23        ll target = (__int128)b * NumberTheory::inverse(k, m) % m;
24        ll step = sqrtl(m) + 1;
25        unordered_map<ll, ll> baby;
26        baby.reserve(step);
27        ll cur = 1 % m;
28        for (ll j = 0; j < step; j++)
29        {
30            if (!baby.count(cur)) baby[cur] = j;
31            cur = (__int128)cur * a % m;
32        }
33        ll inv = NumberTheory::inverse(NumberTheory::power(a, step, m), m);
34        cur = target;
35        for (ll i = 0; i <= step; i++)
36        {
37            auto it = baby.find(cur);
38            if (it != baby.end()) return offset + i * step + it->second;
39            cur = (__int128)cur * inv % m;
40        }
41        return -1;
42    }
43 };

```

## 第六十五章 任意模数行列式

### 65.1 任意正模数行列式：欧几里得消元

`determinant_mod(a,mod)` 返回  $\det(a)$  的最小非负余数。 $a$  为 `long long` 方阵，元素支持完整 signed 64 位； $1 \leq \text{mod} \leq \text{LLONG\_MAX}$ ，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回  $1 \% \text{mod}$ ， $\text{mod}=1$  时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 `int128`。 $O(n^3 \log \text{mod})$  时间、 $O(n^2)$  空间；素数模通常优先用已有高斯行列式。

```
1 inline long long determinant_mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for (auto &row : a)
6     {
7         assert((int)row.size() == n);
8         for (auto &x : row)
9         {
10             x %= mod;
11             if (x < 0) x += mod;
12         }
13     }
14     long long answer = 1 % mod;
15     for (int i = 0; i < n; i++)
16     {
17         for (int j = i + 1; j < n; j++)
18             while (a[j][i])
19             {
20                 long long quotient = a[i][i] / a[j][i];
21                 for (int k = i; k < n; k++)
22                 {
23                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
24                     value %= mod;
25                     if (value < 0) value += mod;
26                     a[i][k] = (long long)value;
27                 }
28                 swap(a[i], a[j]);
29                 answer = answer ? mod - answer : 0;
30             }
31         if (a[i][i] == 0) return 0;
32         answer = (__int128)answer * a[i][i] % mod;
33     }
34     return answer;
35 }
```

# 第六十六章 带权生成树计数

## 66.1 带权矩阵树：无向树与内外向树形图

声明 `MatrixTree<mod>`，调用 `count(n, edges, root, kind)` 返回所有合法树的边权乘积之和，`mod` 必须是素数。`Edge=(u,v,Z)`，点号 `0..n-1`， $n \geq 1$ ；`Kind::undirected` 为无向生成树，`toward_root` 为每点沿边能到根，`away_from_root` 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 `root` 决定答案。依赖 `LinearAlgebra` 的素数模行列式， $O(n^3+m)$  时间、 $O(n^2)$  额外空间。

```
1 template <int mod> struct MatrixTree
2 {
3     using A = LinearAlgebra<mod>;
4     using Z = typename A::Z;
5     using Edge = tuple<int, int, Z>;
6     enum class Kind
7     {
8         undirected,
9         toward_root,
10        away_from_root
11    };
12
13    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
14    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
15    {
16        assert(n >= 1 && root >= 0 && root < n);
17        typename A::Matrix lap(n - 1, vector<Z>(n - 1));
18        auto add = [&](int u, int v, Z w)
19        {
20            if (u == root) return;
21            int i = u - (u > root), j = v - (v > root);
22            lap[i][i] = lap[i][i] + w;
23            if (v != root) lap[i][j] = lap[i][j] - w;
24        };
25        for (auto [u, v, w] : edges)
26        {
27            assert(u >= 0 && u < n && v >= 0 && v < n);
28            if (u == v) continue;
29            if (kind == Kind::away_from_root) swap(u, v);
30            add(u, v, w);
31            if (kind == Kind::undirected) add(v, u, w);
32        }
33        return A::determinant(lap);
34    }
35};
```

# 第六十七章 合数模生成树计数

## 67.1 任意正模数带权矩阵树

`count(n, edges, root, kind, mod)` 返回树的边权乘积之和对 `mod` 的最小非负余数。`mod` 为任意正 signed 64 位数，边权支持完整 signed 64 位。`Kind` 的 `undirected`、`toward_root`、`away_from_root` 分别表示无向、内向和外向树；点号 `0..n-1`。自环忽略，重边按不同边计数，单点返回 `1%mod`，`mod=1` 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 `determinant_mod`； $O(n^3 \log \text{mod} + m)$  时间、 $O(n^2)$  额外空间。不使用模逆元。已知模数是素数时一般优先使用 `MatrixTree<mod>`。

```
1 struct MatrixTreeMod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if (u == root) return;
19            int i = u - (u > root), j = v - (v > root);
20            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
21            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
22        };
23        for (auto [u, v, w] : edges)
24        {
25            assert(u >= 0 && u < n && v >= 0 && v < n);
26            if (u == v) continue;
27            w %= mod;
28            if (w < 0) w += mod;
29            if (kind == Kind::away_from_root) swap(u, v);
30            add(u, v, w);
31            if (kind == Kind::undirected) add(v, u, w);
32        }
33        return determinant_mod(move(lap), mod);
34    }
35};
```

# 第六十八章 计算几何

## 68.1 整数精确二维几何

坐标绝对值  $\leq 10^{12}$ ，点数  $\leq 10^6$ 。叉积、点积、线段相交、凸包、点在多边形、点在凸包、旋转卡壳直径。凸包逆时针，不保留共线内点，不重复首点。contains 返回 0 外部、1 边界、2 内部。convex\_contains/diameter2 输入必须为本模板 strict hull。PolarLess 要求排除零向量。

```
1 struct IntegerGeometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }
12
13         bool operator==(Point b) const { return x == b.x && y == b.y; }
14     };
15
16     static I cross(Point a, Point b, Point c)
17     {
18         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
19     }
20
21     static I dot(Point a, Point b, Point c)
22     {
23         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
24     }
25
26     static I dist2(Point a, Point b) { return dot(a, b, b); }
27
28     static int sign(I x) { return (x > 0) - (x < 0); }
29
30     static bool on_segment(Point p, Point a, Point b)
31     {
32         return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
33     }
34
35     static bool intersect(Point a, Point b, Point c, Point d)
36     {
37         int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
38         int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
39         if (!x && on_segment(c, a, b)) return true;
40         if (!y && on_segment(d, a, b)) return true;
41         if (!z && on_segment(a, c, d)) return true;
42         if (!w && on_segment(b, c, d)) return true;
43         return x * y < 0 && z * w < 0;
44     }
```

```

45
46 static vector<Point> hull(vector<Point> p)
47 {
48     sort(p.begin(), p.end());
49     p.erase(unique(p.begin(), p.end()), p.end());
50     int n = (int)p.size();
51     if (n < 3) return p;
52     vector<Point> h;
53     for (Point v : p)
54     {
55         while (h.size() > 1 && cross(h[h.size() - 2], h.back(), v) <= 0)
56             h.pop_back();
57         h.push_back(v);
58     }
59     int low = (int)h.size();
60     for (int i = n - 2; i >= 0; i--)
61     {
62         while ((int)h.size() > low && cross(h[h.size() - 2], h.back(), p[i]) <= 0)
63             h.pop_back();
64         h.push_back(p[i]);
65     }
66     h.pop_back();
67     return h; // CCW, no repeated endpoint, no collinear interior points.
68 }
69
70 static I area2(const vector<Point> &p)
71 {
72     I ans = 0;
73     int n = (int)p.size();
74     for (int i = 0; i < n; i++)
75         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
76     return ans;
77 }
78
79 // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
80 static int contains(const vector<Point> &p, Point q)
81 {
82     int winding = 0, n = (int)p.size();
83     for (int i = 0; i < n; i++)
84     {
85         Point a = p[i], b = p[(i + 1) % n];
86         if (on_segment(q, a, b)) return 1;
87         I c = cross(a, b, q);
88         if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
89         if (a.y > q.y && b.y <= q.y && c < 0) --winding;
90     }
91     return winding ? 2 : 0;
92 }
93
94 // Input must be the strict CCW hull above. O(log n).
95 static int convex_contains(const vector<Point> &p, Point q)
96 {
97     int n = (int)p.size();
98     if (n < 3) return contains(p, q);
99     I a = cross(p[0], p[1], q), b = cross(p[0], p[n - 1], q);
100     if (a < 0 || b > 0) return 0;

```

```

101     if (!a) return on_segment(q, p[0], p[1]) ? 1 : 0;
102     if (!b) return on_segment(q, p[0], p[n - 1]) ? 1 : 0;
103     int l = 1, r = n - 1;
104     while (l + 1 < r)
105     {
106         int m = (l + r) / 2;
107         if (cross(p[0], p[m], q) >= 0)
108             l = m;
109         else
110             r = m;
111     }
112     I c = cross(p[l], p[r], q);
113     return c < 0 ? 0 : c == 0 ? 1 : 2;
114 }
115
116 static I diameter2(const vector<Point> &p)
117 {
118     int n = (int)p.size();
119     if (n < 2) return 0;
120     if (n == 2) return dist2(p[0], p[1]);
121     I ans = 0;
122     int j = 1;
123     for (int i = 0; i < n; i++)
124     {
125         int k = (i + 1) % n;
126         while (cross(p[i], p[k], p[(j + 1) % n]) > cross(p[i], p[k], p[j]))
127             j = (j + 1) % n;
128         ans = max({ans, dist2(p[i], p[j]), dist2(p[k], p[j])});
129     }
130     return ans;
131 }
132
133 struct PolarLess
134 {
135     static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
136
137     bool operator()(Point a, Point b) const
138     {
139         if (half(a) != half(b)) return half(a) < half(b);
140         I c = cross({0, 0}, a, b);
141         if (c) return c > 0;
142         return dist2({0, 0}, a) < dist2({0, 0}, b);
143     }
144 };
145 };

```

## 68.2 浮点点线圆构造

long double, 显式 eps 默认  $10^{-12}$ 。要求有限输入及适当尺度；接近相切时按容差分类，不能提供任意实数输入的精确拓扑保证。Result 区分 none/one/two/infinite/degenerate。圆圆交点仅在圆心和正半径完全相同时返回 infinite；零半径圆按点处理，不把近同心圆当作重合。圆相关交点容差按局部距离和半径缩放，不使用固定的单位长度下限。零长度线返回 degenerate；投影与点到线段距离将零线段视为点。overlap 返回交叠面积，小圆弓用级数避免三角函数相减消失。坐标缩放和误差预算仍须按题面复核。

```

1 struct RealGeometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     RealGeometry(R eps = 1e-12L) : eps(eps) { assert(eps > 0); }
8
9     struct Point
10    {
11        R x = 0, y = 0;
12
13        Point operator+(Point b) const { return {x + b.x, y + b.y}; }
14
15        Point operator-(Point b) const { return {x - b.x, y - b.y}; }
16
17        Point operator*(R k) const { return {x * k, y * k}; }
18
19        Point operator/(R k) const { return {x / k, y / k}; }
20    };
21
22    struct Circle
23    {
24        Point o;
25        R r;
26    };
27    enum class Kind
28    {
29        none,
30        one,
31        two,
32        infinite,
33        degenerate
34    };
35
36    struct Result
37    {
38        Kind kind;
39        vector<Point> p;
40    };
41
42    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
43
44    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
45
46    static R norm(Point a) { return hypotl(a.x, a.y); }
47
48    static Point perp(Point a) { return {-a.y, a.x}; }
49
50    Point projection(Point p, Point a, Point b) const
51    {
52        Point v = b - a;
53        R d = dot(v, v);
54        if (d == 0) return a;
55        return a + v * (dot(p - a, v) / d);
56    }

```

```

57
58 R segment_distance(Point p, Point a, Point b) const
59 {
60     Point v = b - a;
61     R d = dot(v, v);
62     if (d == 0) return norm(p - a);
63     R t = clamp(dot(p - a, v) / d, R(0), R(1));
64     return norm(p - (a + v * t));
65 }
66
67 Result line_intersection(Point a, Point b, Point c, Point d) const
68 {
69     Point u = b - a, v = d - c;
70     R nu = norm(u), nv = norm(v);
71     if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
72     R det = cross(u, v);
73     if (fabsl(det) <= eps * nu * nv)
74         return {fabsl(cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
75                 ? Kind::infinite
76                 : Kind::none,
77                 {}};
78     return {Kind::one, {a + u * (cross(c - a, v) / det)}};
79 }
80
81 Result line_circle(Point a, Point b, Circle c) const
82 {
83     assert(c.r >= 0);
84     Point v = b - a;
85     R len = norm(v);
86     if (len == 0) return {Kind::degenerate, {}};
87     Point h = projection(c.o, a, b);
88     R d = norm(h - c.o), tol = eps * max(d, c.r);
89     if (d > c.r + tol) return {Kind::none, {}};
90     if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
91     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
92     Point w = v * (t / len);
93     return {Kind::two, {h - w, h + w}};
94 }
95
96 Result circles(Circle a, Circle b) const
97 {
98     assert(a.r >= 0 && b.r >= 0);
99     Point v = b.o - a.o;
100    R d = norm(v);
101    R tol = eps * max({d, a.r, b.r});
102    // Infinite intersections require identical positive-radius circles.
103    if (d == 0)
104    {
105        if (a.r != b.r) return {Kind::none, {}};
106        if (a.r == 0) return {Kind::one, {a.o}};
107        return {Kind::infinite, {}};
108    }
109    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
110    if (a.r == 0 || b.r == 0)
111    {
112        R radius = max(a.r, b.r);

```

```

113         if (fabsl(d - radius) > tol) return {Kind::none, {}};
114         return {Kind::one, {a.r == 0 ? a.o : b.o}};
115     }
116     if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
117     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
118     Point h = a.o + v * (x / d);
119     // Near-concentric circles must not be mistaken for internal tangency.
120     if (fabsl(x) > a.r + tol) return {Kind::none, {}};
121     if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
122     R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
123     Point w = perp(v) * (y / d);
124     return {Kind::two, {h - w, h + w}};
125 }
126
127 R overlap(Circle a, Circle b) const
128 {
129     assert(a.r >= 0 && b.r >= 0);
130     R d = norm(a.o - b.o), pi = acosl(-1.L);
131     if (d >= a.r + b.r) return 0;
132     if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
133     R difference = (a.r - b.r) * (a.r + b.r);
134     R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
135     R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
136     auto segment = [](R angle)
137     {
138         if (angle < 1e-3L)
139         {
140             R z = angle * angle;
141             return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
142         }
143         return angle - sinl(2 * angle) / 2;
144     };
145     return a.r * a.r * segment(x) + b.r * b.r * segment(y);
146 }
147 };

```

# 第六十九章 优化与可持久化

## 69.1 Max-plus 矩阵幂

$a[i][j]$  为转移收益, 不存在的边为  $neg$ ;  $power(k)$  求恰好  $k$  步的最优转移, 单位矩阵对角为 0。所有有限中间值绝对值小于  $2^{119}$ 。  $O(n^3 \log k)$ ; 2025 成都 B 的转移类型, 具体题目的建模仍需调用者完成。

```
1 struct MaxPlusMatrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if (identity)
11             for (int i = 0; i < n; i++) a[i][i] = 0;
12     }
13
14     MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
15     {
16         assert(n == b.n);
17         MaxPlusMatrix c(n);
18         for (int i = 0; i < n; i++)
19             for (int k = 0; k < n; k++)
20                 if (a[i][k] != neg)
21                     for (int j = 0; j < n; j++)
22                         if (b.a[k][j] != neg)
23                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
24         return c;
25     }
26
27     MaxPlusMatrix power(unsigned long long e) const
28     {
29         MaxPlusMatrix b = *this, r(n, true);
30         for (; e >= 1, b = b * b)
31             if (e & 1) r = r * b;
32         return r;
33     }
34 };
```

## 69.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标,  $add(\{k,b,id\})$  插入直线,  $query(x)$  返回最小值与最小编号, 要求  $x$  在预登记坐标内且至少有一条线。整数求值用  $int128$ ,  $k$ 、 $b$ 、 $x$  为有符号 64 位且中间值可表示。单次  $O(\log n)$ 。

```
1 struct LiChao
2 {
3     using ll = long long;
4     using I = __int128_t;
```

```

5
6 struct Line
7 {
8     ll k = 0, b = 0;
9     int id = -1;
10
11     I value(ll x) const { return I(k) * x + b; }
12 };
13
14 struct Node
15 {
16     Line line;
17     int left = -1, right = -1;
18 };
19
20 vector<ll> xs;
21 vector<Node> a{Node{}};
22
23 LiChao(vector<ll> x) : xs(x)
24 {
25     sort(xs.begin(), xs.end());
26     xs.erase(unique(xs.begin(), xs.end()), xs.end());
27     assert(!xs.empty());
28 }
29
30 bool better(Line u, Line v, ll x) const
31 {
32     return v.id == -1 ||
33         (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
34 }
35
36 void add(Line line)
37 {
38     assert(line.id >= 0);
39     add(0, 0, (int)xs.size() - 1, line);
40 }
41
42 void add(int p, int l, int r, Line line)
43 {
44     int m = (l + r) / 2;
45     if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
46     if (l == r || line.id == -1) return;
47     bool left = better(line, a[p].line, xs[l]);
48     bool right = better(line, a[p].line, xs[r]);
49     if (!left && !right) return;
50     int next = left ? a[p].left : a[p].right;
51     if (next == -1)
52     {
53         next = (int)a.size();
54         a.push_back(Node{});
55         if (left)
56             a[p].left = next;
57         else
58             a[p].right = next;
59     }
60     if (left)

```

```

61         add(next, l, m, line);
62     else
63         add(next, m + 1, r, line);
64 }
65
66 pair<I, int> query(ll x) const
67 {
68     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
69     assert(idx < (int)xs.size() && xs[idx] == x);
70     int p = 0, l = 0, r = (int)xs.size() - 1;
71     Line best;
72     while (p != -1)
73     {
74         if (better(a[p].line, best, x)) best = a[p].line;
75         if (l == r) break;
76         int m = (l + r) / 2;
77         if (idx <= m)
78         {
79             p = a[p].left;
80             r = m;
81         }
82         else
83         {
84             p = a[p].right;
85             l = m + 1;
86         }
87     }
88     assert(best.id != -1);
89     return {best.value(x), best.id};
90 }
91 };

```

### 69.3 主席树：静态区间第 k 小

构造传入 0-based 数组， $\text{kth}(l, r, k)$  的区间使用 1-based 闭区间；要求  $1 \leq k \leq r - l + 1$ 。离散化后每个前缀一个版本，单次  $O(\log n)$ ，总空间  $O(n \log n)$ 。

```

1 struct PersistentKth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};
9     vector<int> root{0};
10    vector<long long> vals;
11
12    PersistentKth(const vector<long long> &a) : vals(a)
13    {
14        sort(vals.begin(), vals.end());
15        vals.erase(unique(vals.begin(), vals.end()), vals.end());
16        for (long long x : a)
17        {
18            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();

```

```

19         root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20     }
21 }
22
23 int add(int old, int l, int r, int x)
24 {
25     int p = t.size();
26     Node copy = t[old];
27     t.push_back(copy);
28     ++t[p].sum;
29     if (l < r)
30     {
31         int m = (l + r) / 2;
32         if (x <= m)
33         {
34             int v = add(t[old].l, l, m, x);
35             t[p].l = v;
36         }
37         else
38         {
39             int v = add(t[old].r, m + 1, r, x);
40             t[p].r = v;
41         }
42     }
43     return p;
44 }
45
46 long long kth(int l, int r, int k) const
47 {
48     assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49     int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50     while (L < R)
51     {
52         int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53         if (k <= c)
54         {
55             u = t[u].l;
56             v = t[v].l;
57             R = m;
58         }
59         else
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };

```

## 第七十章 可持久化数组

### 70.1 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0), a 是非空 0-based 初值数组; 第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置, 创建并返回新版本编号; copy(version) 以  $O(1)$  时间复制版本; query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点, 旧版本不变; build 为内部函数, 不可覆盖历史节点。建树  $O(n)$ , 修改及查询  $O(\log n)$ , 初始  $2n-1$  个节点, 每次修改至多  $\text{ceil}(\log_2(n))+1$  个; 总空间  $O(n + \text{修改数} * \log n + \text{版本数})$ 。changes 仅用于预留节点池, 低估后可扩容, 所有外部引用应使用下标, 不保存 vector 元素引用。节点下标用 int, 节点总量须可表示。

```
1 struct PersistentArray
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root;
14
15     PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
16     {
17         assert(n > 0 && changes >= 0);
18         int levels = 1;
19         for (int x = n - 1; x; x >>= 1) levels++;
20         t.reserve(2ULL * n + 1ULL * changes * levels + 1);
21         root.push_back(build(1, n, a));
22     }
23
24     int build(int l, int r, const vector<ll> &a)
25     {
26         int p = t.size();
27         t.push_back(Node{});
28         if (l == r)
29             t[p].value = a[l - 1];
30         else
31         {
32             int m = (l + r) / 2;
33             int left = build(l, m, a);
34             int right = build(m + 1, r, a);
35             t[p].l = left;
36             t[p].r = right;
37         }
38         return p;
39     }
40
41     int modify(int old, int l, int r, int pos, ll value)
```

```

42     {
43         int p = t.size();
44         Node copy = t[old];
45         t.push_back(copy);
46         if (l == r)
47             t[p].value = value;
48         else
49         {
50             int m = (l + r) / 2;
51             if (pos <= m)
52             {
53                 int child = modify(t[old].l, l, m, pos, value);
54                 t[p].l = child;
55             }
56             else
57             {
58                 int child = modify(t[old].r, m + 1, r, pos, value);
59                 t[p].r = child;
60             }
61         }
62         return p;
63     }
64
65     // Create a new version from any existing version. Positions are 1-based.
66     int set(int version, int pos, ll value)
67     {
68         assert(0 <= version && version < (int)root.size());
69         assert(1 <= pos && pos <= n);
70         int p = modify(root[version], 1, n, pos, value);
71         root.push_back(p);
72         return (int)root.size() - 1;
73     }
74
75     int copy(int version)
76     {
77         assert(0 <= version && version < (int)root.size());
78         root.push_back(root[version]);
79         return (int)root.size() - 1;
80     }
81
82     ll query(int version, int pos) const
83     {
84         assert(0 <= version && version < (int)root.size());
85         assert(1 <= pos && pos <= n);
86         int p = root[version], l = 1, r = n;
87         while (l < r)
88         {
89             int m = (l + r) / 2;
90             if (pos <= m)
91             {
92                 p = t[p].l;
93                 r = m;

```

```
94         }
95         else
96         {
97             p = t[p].r;
98             l = m + 1;
99         }
100     }
101     return t[p].value;
102 }
103 };
```

# 第七十一章 可持久化区间操作

## 71.1 可持久化区间加与版本拼接

PersistentRange(n) 建立长度  $n$  的全零版本 0,  $n \geq 1$ ; 使用隐式节点, 不预建整棵树。add( $v, l, r, x$ ) 在版本  $v$  的  $[l, r)$  加  $x$ ; splice( $dst, src, l, r$ ) 将  $dst$  的  $[l, r)$  替换为  $src$  同一位置的值, 其他位置保留  $dst$ ; 二者均返回新版本编号。query( $v, l, r$ ) 返回区间和, const 且不分配节点。下标 0-based、区间半开, 允许空区间; 空修改及零增量仍创建版本但复用根, 整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先, 用 offset 修正跨版本拼接; 每次修改/拼接/查询  $O(\log n)$ , 版本别名  $O(1)$ , 总节点  $O(q \log n)$ 。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内; 节点总数和版本数须小于 INT\_MAX。t、root 可读以统计空间或查看根, 不要外部改写。

```
1 struct PersistentRange
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit PersistentRange(int size) : n(size) { assert(n > 0); }
16
17     int clone(int p)
18     {
19         Node node = t[p];
20         t.push_back(node);
21         return int(t.size()) - 1;
22     }
23
24     void pull(int p, int len)
25     {
26         t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
27     }
```

区间加与版本拼接（接上页同一结构体）：

```

29     int change(int p, int l, int r, int ql, int qr, ll value)
30     {
31         if (qr <= l || r <= ql) return p;
32         int u = clone(p);
33         if (ql <= l && r <= qr)
34         {
35             t[u].lazy += value;
36             t[u].sum += value * (r - l);
37             return u;
38         }
39         int m = l + (r - l) / 2;
40         int a = change(t[p].left, l, m, ql, qr, value);
41         int b = change(t[p].right, m, r, ql, qr, value);
42         t[u].left = a;
43         t[u].right = b;
44         pull(u, r - l);
45         return u;
46     }
47
48     int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
49     {
50         if (qr <= l || r <= ql) return p;
51         if (ql <= l && r <= qr)
52         {
53             if (offset == 0) return q;
54             int u = clone(q);
55             t[u].lazy += offset;
56             t[u].sum += offset * (r - l);
57             return u;
58         }
59         int u = clone(p), m = l + (r - l) / 2;
60         offset += t[q].lazy - t[p].lazy;
61         int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
62         int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
63         t[u].left = a;
64         t[u].right = b;
65         pull(u, r - l);
66         return u;
67     }

```

只读查询与对外接口（接上页同一结构体）：

```

69  ll get(int p, int l, int r, int ql, int qr, ll carry) const
70  {
71      if (qr <= l || r <= ql) return 0;
72      if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
73      carry += t[p].lazy;
74      int m = l + (r - l) / 2;
75      return get(t[p].left, l, m, ql, qr, carry) +
76             get(t[p].right, m, r, ql, qr, carry);
77  }
78
79  void check(int version, int l, int r) const
80  {
81      assert(0 <= version && version < int(root.size()));
82      assert(0 <= l && l <= r && r <= n);
83  }
84
85  int add(int version, int l, int r, ll value)
86  {
87      check(version, l, r);
88      int p = l == r || value == 0 ? root[version]
89              : change(root[version], 0, n, l, r, value);
90      root.push_back(p);
91      return int(root.size()) - 1;
92  }
93
94  int splice(int destination, int source, int l, int r)
95  {
96      check(destination, l, r);
97      check(source, l, r);
98      int p = l == r ? root[destination]
99              : replace(root[destination], root[source], 0, n, l, r, 0);
100     root.push_back(p);
101     return int(root.size()) - 1;
102 }
103
104 ll query(int version, int l, int r) const
105 {
106     check(version, l, r);
107     return l == r ? 0 : get(root[version], 0, n, l, r, 0);
108 }
109 };

```

# 第七十二章 树上路径第 k 小

## 72.1 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次； $1 \leq k \leq$  路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用  $\text{root}[u] + \text{root}[v] - \text{root}[\text{lca}] - \text{root}[\text{parent}(\text{lca})]$ ，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理  $O(n \log n)$ ，每次查询  $O(\log n)$ ，空间  $O(n \log n)$ 。递归深度最坏  $O(n)$ ，保持递归写法。

```
1 struct TreePathKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, lg;
11    vector<vector<int>> g, up;
12    vector<int> root, dep;
13    vector<ll> value, vals;
14    vector<Node> t;
15
16    TreePathKth(const vector<ll> &a)
17        : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
18    {
19        assert(n > 0);
20        lg = 1;
21        while ((1LL << lg) <= n) lg++;
22        up.assign(lg, vector<int>(n + 1));
23    }
24
25    void add(int u, int v)
26    {
27        g[u].push_back(v);
28        g[v].push_back(u);
29    }
30
31    int insert(int old, int l, int r, int x)
32    {
33        int p = t.size();
34        Node copy = t[old];
35        t.push_back(copy);
36        t[p].sum++;
37        if (l == r) return p;
38        int m = (l + r) / 2;
39        if (x <= m)
40        {
41            int child = insert(t[old].l, l, m, x);
```

```

42         t[p].l = child;
43     }
44     else
45     {
46         int child = insert(t[old].r, m + 1, r, x);
47         t[p].r = child;
48     }
49     return p;
50 }
51
52 void dfs(int u, int p)
53 {
54     up[0][u] = p;
55     for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
56     int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
57     root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
58     for (int v : g[u])
59     {
60         if (v == p) continue;
61         dep[v] = dep[u] + 1;
62         dfs(v, u);
63     }
64 }
65
66 void build(int r = 1)
67 {
68     assert(1 <= r && r <= n);
69     vals = value;
70     sort(vals.begin(), vals.end());
71     vals.erase(unique(vals.begin(), vals.end()), vals.end());
72     t.assign(1, Node{});
73     dep[r] = 0;
74     dfs(r, 0);
75 }
76
77 int lca(int u, int v) const
78 {
79     if (dep[u] < dep[v]) swap(u, v);
80     int diff = dep[u] - dep[v];
81     for (int j = 0; j < lg; j++)
82         if (diff >> j & 1) u = up[j][u];
83     if (u == v) return u;
84     for (int j = lg - 1; j >= 0; j--)
85         if (up[j][u] != up[j][v])
86         {
87             u = up[j][u];
88             v = up[j][v];
89         }
90     return up[0][u];
91 }
92
93 ll kth(int u, int v, int k) const
94 {
95     int z = lca(u, v);
96     assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
97     int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];

```

```
98     int l = 0, r = (int)vals.size() - 1;
99     while (l < r)
100     {
101         int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
102         int m = (l + r) / 2;
103         if (k <= count)
104         {
105             a = t[a].l;
106             b = t[b].l;
107             c = t[c].l;
108             d = t[d].l;
109             r = m;
110         }
111         else
112         {
113             k -= count;
114             a = t[a].r;
115             b = t[b].r;
116             c = t[c].r;
117             d = t[d].r;
118             l = m + 1;
119         }
120     }
121     return vals[l];
122 }
123 };
```

# 第七十三章 动态区间顺序统计

## 73.1 动态区间第 k 小 (树状数组套权值线段树)

DynamicKth(a,future) 传入非空 0-based 初值, future 包含所有将来可能赋入的新值; 内部自动合并初值并离散化。set(pos,value) 单点赋值, kth(l,r,k) 查询闭区间第 k 小, 下标均为 1-based;  $1 \leq k \leq r-l+1$ 。允许重复值与完整 long long 权值; 新值必须已在离散化集合内, 操作可按原顺序执行, 不保留历史版本。每个树状数组节点对应一棵可变权值线段树; 删除旧值后插入新值。空节点回收到 free\_head 链, 重复修改不会按操作次数累计泄漏节点。设 D 为不同值数,  $S=n+\text{future.size()}$ , 预处理  $O(S \log S+n \log n \log(D+1))$ , 单次修改及查询  $O(\log n \log(D+1))$ 。节点数上界  $B=(\text{ceil}(\log_2(D))+1) \cdot \sum \text{lowbit}(i)$ , 求和  $i=1..n$ ; 空间  $O(S+n+B)$ 。update、modify、id 和节点池均为内部接口; 不能手动减去不存在的值。

```
1 struct DynamicKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, free_head = 0;
11    vector<int> bit, rank;
12    vector<ll> vals;
13    vector<Node> t{Node{}};
14
15    DynamicKth(const vector<ll> &a, vector<ll> future = {})
16        : n(a.size()), bit(n + 1), rank(n + 1)
17    {
18        assert(n > 0);
19        vals = future;
20        vals.insert(vals.end(), a.begin(), a.end());
21        sort(vals.begin(), vals.end());
22        vals.erase(unique(vals.begin(), vals.end()), vals.end());
23        for (int i = 1; i <= n; i++)
24        {
25            rank[i] = id(a[i - 1]);
26            modify(i, rank[i], 1);
27        }
28    }
29
30    int id(ll value) const
31    {
32        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
33        assert(p < (int)vals.size() && vals[p] == value);
34        return p;
35    }
36
37    int new_node()
38    {
39        if (!free_head)
```

```

40     {
41         t.push_back(Node{});
42         return (int)t.size() - 1;
43     }
44     int p = free_head;
45     free_head = t[p].l;
46     t[p] = Node{};
47     return p;
48 }
49
50 int update(int p, int l, int r, int x, int delta)
51 {
52     if (!p) p = new_node();
53     t[p].sum += delta;
54     if (l < r)
55     {
56         int m = (l + r) / 2;
57         if (x <= m)
58         {
59             int child = update(t[p].l, l, m, x, delta);
60             t[p].l = child;
61         }
62         else
63         {
64             int child = update(t[p].r, m + 1, r, x, delta);
65             t[p].r = child;
66         }
67     }
68     if (!t[p].sum)
69     {
70         t[p].l = free_head;
71         free_head = p;
72         return 0;
73     }
74     return p;
75 }
76
77 void modify(int pos, int x, int delta)
78 {
79     for (int i = pos; i <= n; i += i & -i)
80         bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
81 }

```

```

83 void set(int pos, ll value)
84 {
85     assert(1 <= pos && pos <= n);
86     int x = id(value);
87     if (rank[pos] == x) return;
88     modify(pos, rank[pos], -1);
89     modify(pos, x, 1);
90     rank[pos] = x;
91 }
92
93 ll kth(int l, int r, int k) const
94 {
95     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
96     vector<int> a, b;
97     for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
98     for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
99     int L = 0, R = (int)vals.size() - 1;
100     while (L < R)
101     {
102         int count = 0;
103         for (int p : a) count += t[t[p].l].sum;
104         for (int p : b) count -= t[t[p].l].sum;
105         bool left = k <= count;
106         if (!left) k -= count;
107         for (int &p : a) p = left ? t[p].l : t[p].r;
108         for (int &p : b) p = left ? t[p].l : t[p].r;
109         int m = (L + R) / 2;
110         if (left)
111             R = m;
112         else
113             L = m + 1;
114     }
115     return vals[L];
116 }
117 };

```

## 第七十四章 区间不同数统计

### 74.1 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数， $1 \leq l \leq r \leq n$ 。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 0..n-1 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 0..l-1 的频数和。每个元素只复制一条节点路径，预处理  $O(n \log n)$ ，查询  $O(\log n)$ ，空间  $O(n \log n)$ 。动态版预留  $n \cdot (\text{ceil}(\log_2(n))+1)+1$  个节点，避免扩容峰值；root 和节点池为内部数据。

```
1 struct PersistentDistinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n;
9     vector<Node> t{Node{}};
10    vector<int> root;
11
12    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
13    {
14        assert(n > 0);
15        int height = 1;
16        while ((1LL << (height - 1)) < n) height++;
17        t.reserve(size_t(n) * height + 1);
18        vector<long long> vals = a;
19        sort(vals.begin(), vals.end());
20        vals.erase(unique(vals.begin(), vals.end()), vals.end());
21        vector<int> last(vals.size());
22        for (int i = 1; i <= n; i++)
23        {
24            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
25            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
26            last[x] = i;
27        }
28    }
29
30    int insert(int old, int l, int r, int x)
31    {
32        int p = t.size();
33        Node copy = t[old];
34        t.push_back(copy);
35        t[p].sum++;
36        if (l < r)
37        {
38            int m = (l + r) / 2;
39            if (x <= m)
40            {
41                int child = insert(t[old].l, l, m, x);
42                t[p].l = child;
```

```
43         }
44         else
45         {
46             int child = insert(t[old].r, m + 1, r, x);
47             t[p].r = child;
48         }
49     }
50     return p;
51 }
52
53 int query(int l, int r) const
54 {
55     assert(1 <= l && l <= r && r <= n);
56     int u = root[l - 1], v = root[r];
57     int L = 0, R = n - 1, answer = 0;
58     while (L < R)
59     {
60         int m = (L + R) / 2;
61         if (l - 1 <= m)
62         {
63             u = t[u].l;
64             v = t[v].l;
65             R = m;
66         }
67         else
68         {
69             answer += t[t[v].l].sum - t[t[u].l].sum;
70             u = t[u].r;
71             v = t[v].r;
72             L = m + 1;
73         }
74     }
75     return answer + t[v].sum - t[u].sum;
76 }
77 };
```

# 第七十五章 图论进阶

## 75.1 矩形二分图最小权匹配

`solve(cost)` 输入  $n \times m$  完整权值矩阵,  $0 \leq n \leq m$ ; 每行匹配不同列, 返回最小费用及列号 (0-based)。允许负费用, 使用 int128 势能;  $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代, 当前接口仅表示完整二分图。

```
1 struct Hungarian
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if (!n) return {0, {}};
10        int m = cost[0].size();
11        assert(n <= m);
12        vector<I> u(n + 1), v(m + 1);
13        vector<int> p(m + 1), way(m + 1);
14        const I inf = I(1) << 120;
15        for (int i = 1; i <= n; i++)
16        {
17            assert((int)cost[i - 1].size() == m);
18            p[0] = i;
19            int j0 = 0;
20            vector<I> dis(m + 1, inf);
21            vector<bool> used(m + 1);
22            do
23            {
24                used[j0] = true;
25                int i0 = p[j0], j1 = 0;
26                I delta = inf;
27                for (int j = 1; j <= m; j++)
28                    if (!used[j])
29                    {
30                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
31                        if (cur < dis[j])
32                        {
33                            dis[j] = cur;
34                            way[j] = j0;
35                        }
36                        if (dis[j] < delta)
37                        {
38                            delta = dis[j];
39                            j1 = j;
40                        }
41                    }
42                for (int j = 0; j <= m; j++)
43                    if (used[j])
44                    {
45                        u[p[j]] += delta;
```

```

46         v[j] -= delta;
47     }
48     else
49         dis[j] -= delta;
50     j0 = j1;
51 } while (p[j0]);
52 do
53 {
54     int j1 = way[j0];
55     p[j0] = p[j1];
56     j0 = j1;
57 } while (j0);
58 }
59 vector<int> ans(n);
60 for (int j = 1; j <= m; j++)
61     if (p[j]) ans[p[j] - 1] = j - 1;
62 return {-v[0], ans};
63 }
64 };

```

## 75.2 有向最小树形图

`solve(n, root, edges)`, 点号  $0..n-1$ ,  $n \geq 1$ ,  $0 \leq \text{root} < n$ ; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边和自环 (自环不选),  $n=1$  的费用为 0; 费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于  $2^{120}$ , 类型为 `int128`。每个非根点选最小入边; 若成环则缩点, 跨环入边减去原终点已选入边费用, 以计算替代代价。 $O(VE)$ , `edges` 按值传入, 原图保持不变。仅返回费用, 不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验, 暂无专门 OJ AC。

```

1 struct Arborescence
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while (true)
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for (auto a : e)
21                 if (a.u != a.v && a.w < in[a.v])

```

```
22         {
23             in[a.v] = a.w;
24             pre[a.v] = a.u;
25         }
26     in[root] = 0;
27     for (I x : in)
28     {
29         if (x == inf) return nullopt;
30         ans += x;
31     }
```

判环与缩点（接上页同一函数）：

```
32     int cnt = 0;
33     for (int i = 0; i < n; i++)
34     {
35         int u = i;
36         while (vis[u] != i && id[u] == -1 && u != root)
37         {
38             vis[u] = i;
39             u = pre[u];
40         }
41         if (u != root && id[u] == -1)
42         {
43             id[u] = cnt;
44             for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
45             ++cnt;
46         }
47     }
48     if (!cnt) return ans;
49     for (int i = 0; i < n; i++)
50         if (id[i] == -1) id[i] = cnt++;
51     for (auto &a : e)
52     {
53         int v = a.v;
54         a.u = id[a.u];
55         a.v = id[a.v];
56         if (a.u != a.v) a.w -= in[v];
57     }
58     root = id[root];
59     n = cnt;
60 }
61 }
62 };
```

## 75.3 无向全局最小割

`solve(w)`,  $n \geq 2$ ;  $w$  为  $n \times n$  对称非负权矩阵, 重边先累加, 主对角置零。返回 {割权, 一侧顶点列表}, 编号  $0..n-1$ ; 两侧非空, 不要求原图连通, 不连通时割权为 0。并列最优割返回其中一种, 不承诺点数或字典序。矩阵按值传入, 原图不变。 $O(n^3)$  时间、 $O(n^2)$  空间; 无向边总权 (每条只计一次) 严格小于  $2^{120}$ , 内部 `int128`。负权不适用。已逐条核验返回点集在原图上的跨割费用, 暂无专门 OJ AC。

```

1 struct StoerWagner
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for (int i = 0; i < n; i++) group[i] = {i};
14        I best = I(1) << 120;
15        vector<int> side;
16        while (active.size() > 1)
17        {
18            vector<I> dis(n);
19            vector<bool> used(n);
20            int prev = -1;
21            for (int step = 0; step < (int)active.size(); step++)
22            {
23                int sel = -1;
24                for (int v : active)
25                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;

```

本轮割、合并与继续扩张（接上页同一函数）：

```
26         if (step + 1 == (int)active.size())
27         {
28             if (dis[sel] < best)
29             {
30                 best = dis[sel];
31                 side = group[sel];
32             }
33             for (int v : active)
34                 if (v != prev && v != sel)
35                     w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
36             group[prev].insert(
37                 group[prev].end(), group[sel].begin(), group[sel].end());
38             active.erase(find(active.begin(), active.end(), sel));
39             break;
40         }
41         used[sel] = true;
42         prev = sel;
43         for (int v : active)
44             if (!used[v]) dis[v] += w[sel][v];
45     }
46 }
47 return {best, side};
48 }
49 };
```

# 第七十六章 最小割树

## 76.1 Gomory–Hu 最小割树

gomory\_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边;  $n \geq 1$ , 原图点号 0..n-1, 输入为无向边 tuple 列表, 重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG\_MAX。两点最小割是树路径最小边权; 删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边,  $n=1$  返回空树。运行 n-1 次 Dinic, 一般上界  $O(n^3m+n^2)$ , 额外空间  $O(n+m)$ , DFS 增广保持递归。P4897 输入 n 表示 n+1 个点, 驱动传 n+1。双版本 P4897 已 AC, 最长单点分别 252、185 ms。

```
1 inline vector<tuple<int, int, long long>>
2 gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
3 {
4     assert(n >= 1);
5     vector<int> parent(n);
6     vector<long long> value(n);
7     for (int s = 1; s < n; s++)
8     {
9         int t = parent[s];
10        Dinic flow(n);
11        for (auto [u, v, w] : edges)
12            if (u != v)
13            {
14                flow.add(u + 1, v + 1, w);
15                flow.add(v + 1, u + 1, w);
16            }
17        long long cut = flow.flow(s + 1, t + 1);
18        vector<bool> side(n);
19        for (int u : flow.cut(s + 1)) side[u - 1] = true;
20        for (int v = 1; v < n; v++)
21            if (v != s && parent[v] == t && side[v]) parent[v] = s;
22        if (side[parent[t]])
23        {
24            parent[s] = parent[t];
25            parent[t] = s;
26            value[s] = value[t];
27            value[t] = cut;
28        }
29        else
30            value[s] = cut;
31    }
32    vector<tuple<int, int, long long>> tree;
33    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
34    return tree;
35 }
```

# 第七十七章 最小割树查询

## 77.1 最小割树全点对查询

`cut_tree_values(n, tree)` 对合法的连通无环加权树计算全点对路径最小边权；点号  $0..n-1$ 、 $n \geq 1$ 、恰有  $n-1$  条边，边权为非负 `long long`。输入可直接使用 `gomory_hu` 的返回值。结果 `ans[u][v]` 在  $u \neq v$  时为两点最小割；对角线为 `LLONG_MAX` 哨兵，不能解释成同点最小割。 $n=1$  返回单个哨兵。 $O(n^2)$  时间与空间，递归 DFS 深度  $O(n)$ ，查询  $O(1)$ 。只需要标准库，不依赖流模板；不是任意图的查询函数。

```
1 inline vector<vector<long long>>
2 cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
3 {
4     assert(n >= 1 && tree.size() == (size_t)n - 1);
5     vector<vector<pair<int, long long>>> g(n);
6     for (auto [u, v, w] : tree)
7     {
8         assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
9         g[u].push_back({v, w});
10        g[v].push_back({u, w});
11    }
12    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
13    for (int s = 0; s < n; s++)
14    {
15        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
16        {
17            answer[s][u] = value;
18            for (auto [v, w] : g[u])
19                if (v != parent) self(self, v, u, min(value, w));
20        };
21        dfs(dfs, s, -1, LLONG_MAX);
22    }
23    return answer;
24 }
```

# 第七十八章 带权二分图匹配

## 78.1 二分图最大权匹配（缺边与可选匹配）

WeightedMatching(n,m), 两侧编号 1..n、1..m。add(x,y,w) 加边，重边保留最大权；边权支持完整 long long 范围，内部势能与答案为 int128。solve() 要求全部左点匹配不同右点，无解返回 nullopt；n=m 时即最大权完美匹配。solve(true) 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；l[x]、r[y] 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的  $-2^{120}$  哨兵，仅用 add 修改；不把缺边当零权边。允许空侧。全匹配  $O(n^2m)$ ，可选模式增加 n 个虚拟零权右点，为  $O(n^2(m+n))$ ；空间  $O(nm+n+m)$ 。按最短松弛量扩展交错树：每次至少访问一个新右点，最多 n 次后增广；无有限松弛量即无解。

```
1 struct WeightedMatching
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n, m;
6     vector<vector<I>> w;
7     vector<int> l, r;
8
9     WeightedMatching(int n, int m)
10         : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
11     {
12     }
13
14     void add(int x, int y, long long weight)
15     {
16         assert(1 <= x && x <= n && 1 <= y && y <= m);
17         w[x][y] = max(w[x][y], I(weight));
18     }
19
20     optional<I> solve(bool allow_unmatched = false)
21     {
22         fill(l.begin(), l.end(), 0);
23         fill(r.begin(), r.end(), 0);
24         if (!allow_unmatched && n > m) return nullopt;
25         int cols = m + (allow_unmatched ? n : 0);
26         vector<I> u(n + 1), v(cols + 1);
27         vector<int> p(cols + 1), way(cols + 1);
28         for (int i = 1; i <= n; i++)
29         {
30             p[0] = i;
31             int j0 = 0;
32             vector<I> dis(cols + 1, inf);
33             vector<bool> used(cols + 1);
34             do
35             {
36                 used[j0] = true;
37                 int x = p[j0], j1 = 0;
38                 I delta = inf;
39                 for (int j = 1; j <= cols; j++)
40                 {
41                     if (used[j]) continue;
```

```

42         I weight = j <= m ? w[x][j] : 0;
43         if (weight != -inf)
44         {
45             I cur = -weight - u[x] - v[j];
46             if (cur < dis[j])
47             {
48                 dis[j] = cur;
49                 way[j] = j0;
50             }
51         }
52         if (dis[j] < delta)
53         {
54             delta = dis[j];
55             j1 = j;
56         }
57     }
58     if (delta == inf) return nullopt;
59     for (int j = 0; j <= cols; j++)
60     {
61         if (used[j])
62         {
63             u[p[j]] += delta;
64             v[j] -= delta;
65         }
66         else if (dis[j] != inf)
67             dis[j] -= delta;
68     }
69     j0 = j1;
70     } while (p[j0]);
71     do
72     {
73         int j1 = way[j0];
74         p[j0] = p[j1];
75         j0 = j1;
76     } while (j0);
77 }
78 I answer = 0;
79 for (int j = 1; j <= m; j++)
80 {
81     if (!p[j]) continue;
82     l[p[j]] = j;
83     r[j] = p[j];
84     answer += w[p[j]][j];
85 }
86 return answer;
87 }
88 };

```

# 第七十九章 精确有界半平面交

## 79.1 整数半平面的有界多边形交

`bounded_polygon(lines)` 求  $a*x+b*y \leq c$  的交集闭包顶点,  $\text{Line}=\{a,b,c\}$ , 三个整数系数绝对值均  $\leq 10^9$ 。要求交集有界, 空集也允许; 只返回正面积多边形, 点或线段交集返回空向量。无界输入不在接口范围内。 $\text{Point}=\{x,y,d\}$  表示  $(x/d,y/d)$ ,  $d>0$ , 分数不必约分; 按逆时针返回, 不重复首点, 相邻重复点已去除。排序与所有构造、判定使用整数和 `int128`, 不使用 `epsilon`。 $O(n \log n)$  时间、 $O(n)$  空间。浮点转换与面积计算需单独处理。

```
1 struct IntegerHalfplanes
2 {
3     using I = __int128_t;
4     using ll = long long;
5
6     struct Line
7     {
8         ll a, b, c;
9     };
10
11     struct Point
12     {
13         ll x, y, d;
14     };
15
16     static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }
17
18     static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }
19
20     static Point meet(Line u, Line v)
21     {
22         I d = cross(u, v);
23         assert(d != 0);
24         I x = I(u.c) * v.b - I(u.b) * v.c;
25         I y = I(u.a) * v.c - I(u.c) * v.a;
26         if (d < 0)
27         {
28             d = -d;
29             x = -x;
30             y = -y;
31         }
32         return {(ll)x, (ll)y, (ll)d};
33     }
```

接上页同一结构体与函数：

```

35     static I value(Line u, Point p)
36     {
37         return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38     }
39
40     static bool same(Point p, Point q)
41     {
42         return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43     }
44
45     static vector<Point> bounded_polygon(vector<Line> lines)
46     {
47         vector<Line> h;
48         for (auto u : lines)
49         {
50             assert(-10000000000 <= u.a && u.a <= 10000000000);
51             assert(-10000000000 <= u.b && u.b <= 10000000000);
52             assert(-10000000000 <= u.c && u.c <= 10000000000);
53             if (!u.a && !u.b)
54             {
55                 if (u.c < 0) return {};
56             }
57             else
58                 h.push_back(u);
59         }
60         sort(h.begin(),
61              h.end(),
62              [](Line u, Line v)
63              {
64                  if (half(u) != half(v)) return half(u) < half(v);
65                  I d = cross(u, v);
66                  if (d) return d > 0;
67                  ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                  ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                  return I(u.c) * y < I(v.c) * x;
70              });

```

接上页同一结构体与函数：

```

71     deque<Line> q;
72     for (int i = 0; i < (int)h.size(); i++)
73     {
74         Line u = h[i];
75         if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
76         while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
77             q.pop_back();
78         while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
79         if (!q.empty() && cross(q.back(), u) == 0) return {};
80         q.push_back(u);
81     }
82     while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
83         q.pop_back();
84     while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
85     if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
86     vector<Point> answer;
87     for (int i = 0; i < (int)q.size(); i++)
88     {
89         Point p = meet(q[i], q[(i + 1) % q.size()]);
90         if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
91     }
92     if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
93     if (answer.size() < 3) return {};
94     return answer;
95 }
96 };

```

# 第八十章 圆与多边形面积

## 80.1 圆与简单多边形相交面积

`area(points,circle)` 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。`edge(a,b,r)` 返回圆心在 origin 时有向边 `a` 到 `b` 的面积贡献，`r` 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和， $O(n)$  时间、 $O(1)$  额外空间。依赖浮点几何的 `Point/Circle` 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
1 struct CirclePolygon
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     // Signed contribution of edge a -> b; circle centered at the origin.
9     static R edge(Point a, Point b, R r)
10    {
11        if (r == 0) return 0;
12        Point v = b - a;
13        R length = G::norm(v);
14        if (length == 0) return 0;
15        Point u = v / length;
16        R h = G::cross(u, a);
17        Point cuts[4] = {a};
18        int count = 1;
19        if (fabs(h) <= r)
20        {
21            R delta = sqrtl(max(R(0), (r - fabs(h)) * (r + fabs(h))));
22            R t = -G::dot(a, u);
23            Point foot = G::perp(u) * h;
24            for (R s : {-delta, delta})
25                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
26        }
27        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```

28     R answer = 0;
29     for (int i = 1; i < count; i++)
30     {
31         Point p = cuts[i - 1], q = cuts[i];
32         R cross = G::cross(p, q);
33         if (G::norm((p + q) / 2) <= r)
34             answer += cross / 2;
35         else
36             answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
37     }
38     return answer;
39 }
40
41 // Simple polygon, either orientation; duplicate consecutive vertices allowed.
42 static R area(const vector<Point> &p, Circle c)
43 {
44     assert(c.r >= 0);
45     if (p.size() < 3 || c.r == 0) return 0;
46     R answer = 0, error = 0;
47     for (int i = 0; i < (int)p.size(); i++)
48     {
49         R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
50         R y = value - error;
51         R sum = answer + y;
52         error = (sum - answer) - y;
53         answer = sum;
54     }
55     return fabsl(answer);
56 }
57 };

```

# 第八十一章 最小覆盖圆

## 81.1 随机增量最小覆盖圆

`solve(points,rng,eps)` 返回 `optional<Circle>`, `Circle={o,r}`; 空集返回 `nullopt`, 单点或重复同一点返回半径零。`rng` 是调用者提供的 `mt19937_64` 随机数引擎。期望  $O(n)$  时间, 最坏  $O(n^3)$ ,  $O(n)$  空间; 随机化不可省略。`eps` 默认 32 倍 long double 机器精度, 用于相对半径的包含判断。`diameter(a,b)` 求直径圆; `through(a,b,c)` 求外接圆, 共线时返回最远点对的直径圆。依赖浮点几何的 `Point/Circle` 与向量运算。输入及中间运算须有限且尺度适当; 接近共线时圆心可能不稳定, 无法恢复读入前的精度。

```
1 struct EnclosingCircle
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     static Circle diameter(Point a, Point b)
9     {
10         Point o = a + (b - a) / 2;
11         return {o, max(G::norm(o - a), G::norm(o - b))};
12     }
13
14     static Circle through(Point a, Point b, Point c)
15     {
16         Point u = b - a, v = c - a;
17         R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
18         if (scale == 0) return {a, 0};
19         u = u / scale;
20         v = v / scale;
21         R det = 2 * G::cross(u, v);
22         if (det == 0)
23         {
24             Circle best = diameter(a, b);
25             for (Circle q : {diameter(a, c), diameter(b, c)})
26                 if (q.r > best.r) best = q;
27             return best;
28         }
29         R x = G::dot(u, u), y = G::dot(v, v);
30         Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
31         Point o = a + offset * scale;
32         return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
33     }
34 }
```

随机增量主过程（接上页同一结构体）：

```

35     static optional<Circle>
36     solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
37     {
38         assert(eps >= 0 && eps < 1);
39         if (p.empty()) return nullopt;
40         shuffle(p.begin(), p.end(), rng);
41         Circle c{p[0], 0};
42         auto outside = [&](Point q)
43         {
44             return G::norm(q - c.o) > c.r * (1 + eps);
45         };
46         for (int i = 1; i < (int)p.size(); i++)
47             if (outside(p[i]))
48             {
49                 c = {p[i], 0};
50                 for (int j = 0; j < i; j++)
51                     if (outside(p[j]))
52                     {
53                         c = diameter(p[i], p[j]);
54                         for (int k = 0; k < j; k++)
55                             if (outside(p[k])) c = through(p[i], p[j], p[k]);
56                     }
57             }
58         return c;
59     }
60 };

```

## 第八十二章 圆的切线

### 82.1 两圆公切线与过点切线

`solve(a,b,eps)` 返回 `Result`, 包含 `infinite` 和 `lines`; `infinite` 为真表示无穷多条, 否则 `lines` 给出全部切线。每条 `Tangent` 保存切点 `a`、`b`, 单位法向量 `normal`, 以及 `side`。`a`、`b` 对应输入两圆, 直线方程  $\text{dot}(\text{normal}, x-a)=0$ ; 切点相同时仍有效。`side=1` 外公切线、`-1` 内公切线、`0` 两个不同点。半径零表示点, `from_point(p,c,eps)` 等价于第一圆取点, 圆上切点读 `b`。输入有限, 坐标及半径绝对值  $\leq 1e100$ , 半径非负。`eps` 默认  $1e-12$ , 用于相对圆心距的近相切判断; 近相切可能合并为一条, 浮点坐标仍有舍入误差。严格同心等半径时 `infinite=true`, 包括同一点; 同心不等半径返回空。 $O(1)$ ; 依赖浮点几何的 `Point/Circle`、点运算及 `norm/perp`。

```
1 struct CircleTangents
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     struct Tangent
9     {
10         Point a, b, normal;
11         int side; // 1 outer, -1 inner, 0 two distinct point circles.
12     };
13
14     struct Result
15     {
16         bool infinite = false;
17         vector<Tangent> lines;
18     };
19
20     // Line: dot(normal, x - a) = 0. Radius zero represents a point.
21     static Result solve(Circle a, Circle b, R eps = 1e-12L)
22     {
23         assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
24         Point v = b.o - a.o;
25         R d = G::norm(v);
26         if (d == 0) return {a.r == b.r, {}};
27         Point u = v / d;
28         if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```

29     Result answer;
30     for (int side : {1, -1})
31     {
32         if (side == -1 && (a.r == 0 || b.r == 0)) continue;
33         R gap = a.r - side * b.r;
34         if (fabs1(gap) > d * (1 + eps)) continue;
35         bool touch = fabs1(gap) >= d * (1 - eps);
36         R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
37         R s = sqrt1(max(R(0), (1 - c) * (1 + c)));
38         for (int sign : {-1, 1})
39         {
40             Point normal = u * c + G::perp(u) * (sign * s);
41             Point p = a.o + normal * a.r;
42             Point q = b.o + normal * (side * b.r);
43             answer.lines.push_back({p, q, normal, side});
44             if (touch) break;
45         }
46     }
47     return answer;
48 }
49
50 static Result from_point(Point p, Circle c, R eps = 1e-12L)
51 {
52     return solve({p, 0}, c, eps);
53 }
54 };

```

# 第八十三章 浮点最近点对

## 83.1 浮点平面最近点对

`solve(points)` 返回 `long double` 最短距离 (不是平方), 少于两个点返回 `nullopt`, 重复点返回零。Point={x,y}; 输入须有限, 坐标绝对值  $\leq 1e100$ 。递归分治, 按纵坐标归并,  $O(n \log n)$  时间、 $O(n)$  额外空间。用 `hypotl` 求距离, 不先平方; 排序不使用 `epsilon`。浮点运算仍有舍入误差, 输入前已丢失的精度无法恢复。

```
1 struct ClosestPair
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x, y;
8     };
9
10    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
11    static optional<R> solve(vector<Point> p)
12    {
13        int n = p.size();
14        if (n < 2) return nullopt;
15        sort(p.begin(),
16             p.end(),
17             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
18        for (int i = 1; i < n; i++)
19            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
20        auto by_y = [](Point a, Point b)
21        {
22            return tie(a.y, a.x) < tie(b.y, b.x);
23        };
24        auto distance = [](Point a, Point b)
25        {
26            return hypotl(a.x - b.x, a.y - b.y);
27        };
28        vector<Point> tmp(n);
29        function<R(int, int)> dfs = [&](int l, int r) -> R
30        {
31            if (r - l <= 3)
32            {
33                R best = numeric_limits<R>::infinity();
34                for (int i = l; i < r; i++)
35                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
36                sort(p.begin() + l, p.begin() + r, by_y);
37                return best;
38            }
39        }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
39     int m = (l + r) / 2;
40     R x = p[m].x;
41     R best = min(dfs(l, m), dfs(m, r));
42     merge(p.begin() + l,
43           p.begin() + m,
44           p.begin() + m,
45           p.begin() + r,
46           tmp.begin() + l,
47           by_y);
48     copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
49     vector<Point> strip;
50     for (int i = l; i < r; i++)
51         if (fabs1(p[i].x - x) <= best)
52         {
53             for (int j = (int)strip.size() - 1; j >= 0; j--)
54             {
55                 if (p[i].y - strip[j].y > best) break;
56                 best = min(best, distance(p[i], strip[j]));
57             }
58             strip.push_back(p[i]);
59         }
60     return best;
61 };
62 return dfs(0, n);
63 }
64 };
```

# 第八十四章 精确几何进阶

## 84.1 整数最近点对与 Minkowski 和

`closest_pair(points)` 返回最短距离平方，少于两个点为 `nullopt`，重复点返回 0， $O(n \log n)$ ，全程精确整数。  
`minkowski` 输入 strict CCW 凸包，返回不重复首点的凸包；非退化  $O(n+m)$ ，退化时  $O((n+m)\log(n+m))$ 。坐标及坐标和须满足整数几何范围。

```
1 struct GeometryExtra
2 {
3     using G = IntegerGeometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance,  $O(n \log n)$ .
8     static optional<I> closest_pair(vector<P> p)
9     {
10         int n = p.size();
11         if (n < 2) return nullopt;
12         sort(p.begin(), p.end());
13         for (int i = 1; i < n; i++)
14             if (p[i] == p[i - 1]) return I(0);
15         vector<P> tmp(n);
16         auto by_y = [](P a, P b)
17         {
18             return tie(a.y, a.x) < tie(b.y, b.x);
19         };
20         function<I(int, int)> solve = [&](int l, int r) -> I
21         {
22             if (r - l <= 3)
23             {
24                 I best = I(1) << 120;
25                 for (int i = l; i < r; i++)
26                     for (int j = i + 1; j < r; j++)
27                         best = min(best, G::dist2(p[i], p[j]));
28                 sort(p.begin() + l, p.begin() + r, by_y);
29                 return best;
30             }
31             int m = (l + r) / 2;
32             long long x = p[m].x;
33             I best = min(solve(l, m), solve(m, r));
34             merge(p.begin() + l,
35                  p.begin() + m,
36                  p.begin() + m,
37                  p.begin() + r,
38                  tmp.begin() + l,
39                  by_y);
40             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
41             vector<P> strip;
42             for (int i = l; i < r; i++)
43                 if ((I(p[i].x) - x) * (I(p[i].x) - x) < best)
44                     {
```

```

45         for (int j = (int)strip.size() - 1; j >= 0; j--)
46         {
47             I dy = I(p[i].y) - strip[j].y;
48             if (dy * dy >= best) break;
49             best = min(best, G::dist2(p[i], strip[j]));
50         }
51         strip.push_back(p[i]);
52     }
53     return best;
54 };
55 return solve(0, n);
56 }
57
58 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
59 static vector<P> minkowski(vector<P> a, vector<P> b)
60 {
61     if (a.empty() || b.empty()) return {};
62     auto plus = [](P u, P v)
63     {
64         return P{u.x + v.x, u.y + v.y};
65     };
66     if (a.size() < 3 || b.size() < 3)
67     {
68         vector<P> p;
69         for (P u : a)
70             for (P v : b) p.push_back(plus(u, v));
71         return G::hull(p);
72     }
73     auto start = [](vector<P> &p)
74     {
75         auto it =
76             min_element(p.begin(),
77                         p.end(),
78                         [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
79         rotate(p.begin(), it, p.end());
80     };
81     start(a);
82     start(b);
83     int n = a.size(), m = b.size(), i = 0, j = 0;
84     vector<P> result;
85     while (i < n || j < m)
86     {
87         result.push_back(plus(a[i % n], b[j % m]));
88         if (i == n)
89         {
90             ++j;
91             continue;
92         }
93         if (j == m)
94         {
95             ++i;
96             continue;
97         }

```

```
98         P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
99         P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
100         I c = G::cross({0, 0}, u, v);
101         if (c >= 0) ++i;
102         if (c <= 0) ++j;
103     }
104     return result;
105 }
106 };
```

## 84.2 整数三维共线、共面与方向

坐标绝对值  $\leq 10^9$ ；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on\_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```

1 struct IntegerGeometry3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
27
28     static I orient(Point a, Point b, Point c, Point d)
29     {
30         return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
31     }
32
33     static bool collinear(Point a, Point b, Point c)
34     {
35         auto v = cross(diff(b, a), diff(c, a));
36         return !v.x && !v.y && !v.z;
37     }
38
39     static bool on_segment(Point p, Point a, Point b)
40     {
41         return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
42     }
43 };

```

# 第八十五章 凸包支撑点查询

## 85.1 支撑点查询恢复下凸包

静态 `build(left,right,oracle)` 返回从左到右的严格下凸包顶点, `Point` 为  $\{x,y\}$ 。有限非空整数点集无需显式给出; 回调 `oracle(a,b)` 必须精确返回使  $a*x+b*y$  最大的可行点, 允许任意最优解。 `left/right` 必须分别为最小/最大横坐标中纵坐标最低的点; 全体同横坐标时二者必须相同。所有坐标绝对值不超过  $10^{18}$ , 方向差使用 `long long`, 叉积先提升至 `int128`; 回调内部也须保证数值精确且无溢出。对弦  $a \rightarrow b$  查询方向  $(b.y-a.y, a.x-b.x)$ , 严格位于弦下方则递归分治, 否则确认边; 输出时删除共线中间点。若分治找到  $K$  个严格改善支撑点, 查询次数为  $2K+1$  (同横坐标时为 0), 额外时间、空间和最坏递归深度为  $O(K+1)$ 。  $K$  包括可能返回的共线非顶点, 不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合: 点为  $(|S|, |N(S)|)$ , 左点赋权  $a$ 、右点赋权  $b$ , 左  $\rightarrow$  邻点依赖; 下凸包方向满足  $a \geq 0, b < 0$ , 最优闭包的右点恰为邻集。端点为  $(0,0)$  和  $(\text{左点数}, \text{全部左点的邻点数})$ , 有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍, 尚不代表关联赛题整题通过。

```
1 struct SupportHull
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Point
7     {
8         ll x, y;
9     };
10
11     static I cross(Point a, Point b, Point c)
12     {
13         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
14     }
```

```

16 // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
17 // left/right: lowest points at the minimum/maximum x.
18 template <class Oracle>
19 static vector<Point> build(Point left, Point right, Oracle &&oracle)
20 {
21     assert(left.x <= right.x);
22     vector<Point> hull{left};
23     auto dfs = [&](auto &&self, Point a, Point b) -> void
24     {
25         Point c = oracle(b.y - a.y, a.x - b.x);
26         I turn = cross(a, b, c);
27         assert(turn <= 0);
28         if (turn < 0)
29         {
30             assert(a.x < c.x && c.x < b.x);
31             self(self, a, c);
32             self(self, c, b);
33         }
34         else
35         {
36             while (hull.size() >= 2 &&
37                 cross(hull[hull.size() - 2], hull.back(), b) == 0)
38                 hull.pop_back();
39             hull.push_back(b);
40         }
41     };
42     if (left.x < right.x)
43         dfs(dfs, left, right);
44     else
45         assert(left.y == right.y);
46     return hull;
47 }
48 };

```

# 第八十六章 动态树

## 86.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make\_root/find\_root 维护与查询当前表示树根。每次操作均摊  $O(\log n)$ ,  $O(n)$  空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1) {}
16
17    bool is_root(int x) const
18    {
19        int f = a[x].fa;
20        return a[f].ch[0] != x && a[f].ch[1] != x;
21    }
22
23    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }
24
25    void reverse(int x)
26    {
27        if (!x) return;
28        swap(a[x].ch[0], a[x].ch[1]);
29        a[x].rev = !a[x].rev;
30    }
31
32    void push(int x)
33    {
34        if (!a[x].rev) return;
35        reverse(a[x].ch[0]);
36        reverse(a[x].ch[1]);
37        a[x].rev = false;
38    }
39
40    void rotate(int x)
41    {
42        int y = a[x].fa, z = a[y].fa;
```

```

43     int k = a[y].ch[1] == x;
44     int w = a[x].ch[k ^ 1];
45     if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46     a[x].fa = z;
47     a[y].ch[k] = w;
48     if (w) a[w].fa = y;
49     a[x].ch[k ^ 1] = y;
50     a[y].fa = x;
51     pull(y);
52     pull(x);
53 }
54
55 void splay(int x)
56 {
57     int top = 0, y = x;
58     st[top++] = y;
59     while (!is_root(y))
60     {
61         y = a[y].fa;
62         st[top++] = y;
63     }
64     while (top) push(st[--top]);
65     while (!is_root(x))
66     {
67         int y = a[x].fa, z = a[y].fa;
68         if (!is_root(y))
69         {
70             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71             rotate(same ? y : x);
72         }
73         rotate(x);
74     }
75 }
76
77 void access(int x)
78 {
79     int last = 0;
80     for (int y = x; y; y = a[y].fa)
81     {
82         splay(y);
83         // The old right child's fa remains y: it becomes a path-parent.
84         a[y].ch[1] = last;
85         if (last) a[last].fa = y;
86         pull(y);
87         last = y;
88     }
89     splay(x);
90 }
91
92 void make_root(int x)
93 {
94     access(x);
95     reverse(x);
96 }
97
98 int find_root(int x)

```

```

99     {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }

110
111     bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }
112
113     bool link(int x, int y)
114     {
115         make_root(x);
116         if (find_root(y) == x) return false;
117         a[x].fa = y;
118         return true;
119     }
120
121     bool cut(int x, int y)
122     {
123         make_root(x);
124         access(y);
125         if (a[y].ch[0] != x || a[x].ch[1]) return false;
126         a[y].ch[0] = 0;
127         a[x].fa = 0;
128         pull(y);
129         return true;
130     }
131
132     void set(int x, U value)
133     {
134         access(x);
135         a[x].val = value;
136         pull(x);
137     }
138
139     // x and y must be connected. This is a path aggregate, not a subtree sum.
140     U query(int x, int y)
141     {
142         make_root(x);
143         access(y);
144         return a[y].sum;
145     }
146 };

```

# 第八十七章 随机平衡树

## 87.1 FHQ Treap: 可重集合与顺序统计

`insert(x)` 插入一个副本, `erase(x)` 删除一个副本并返回是否存在; `rank(x)` 为严格小于 `x` 的元素数加一, `x` 不必存在; `kth(k)` 要求  $1 \leq k \leq \text{size}()$ 。 `prev/next` 为严格前驱/后继, 不存在时返回 `nullopt`。支持重复值、`LLONG_MIN` 和 `LLONG_MAX`, 分裂不使用 `x-1`。随机优先级下单次期望  $O(\log n)$ , 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```
1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}
17
18    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }
19
20    int node(ll x)
21    {
22        int p = a.size();
23        a.push_back({0, 0, 1, x, rng()});
24        return p;
25    }
26
27    // left: val < x, or val <= x when equal is true. No x-1 overflow.
28    void split(int p, ll x, int &l, int &r, bool equal = false)
29    {
30        if (!p)
31        {
32            l = 0;
33            r = 0;
34            return;
35        }
36        if (a[p].val < x || (equal && a[p].val == x))
37        {
38            l = p;
39            split(a[p].r, x, a[p].r, r, equal);
40        }
41        else
42        {
43            r = p;
```

```

44         split(a[p].l, x, l, a[p].l, equal);
45     }
46     pull(p);
47 }
48
49 int merge(int l, int r)
50 {
51     if (!l || !r) return l ? l : r;
52     if (a[l].pri > a[r].pri)
53     {
54         a[l].r = merge(a[l].r, r);
55         pull(l);
56         return l;
57     }
58     a[r].l = merge(l, a[r].l);
59     pull(r);
60     return r;
61 }
62
63 void insert(ll x)
64 {
65     int p = node(x), l, r;
66     split(root, x, l, r);
67     root = merge(merge(l, p), r);
68 }
69
70 bool erase(ll x)
71 {
72     int l, mid, r;
73     split(root, x, l, mid);
74     split(mid, x, mid, r, true);
75     bool found = mid != 0;
76     if (mid) mid = merge(a[mid].l, a[mid].r);
77     root = merge(l, merge(mid, r));
78     return found;
79 }
80
81 int size() const { return a[root].siz; }
82
83 int less(ll x, bool equal = false) const
84 {
85     int p = root, ans = 0;
86     while (p)
87     {
88         if (a[p].val < x || (equal && a[p].val == x))
89         {
90             ans += a[a[p].l].siz + 1;
91             p = a[p].r;
92         }
93         else
94             p = a[p].l;
95     }
96     return ans;
97 }
98
99 int rank(ll x) const { return less(x) + 1; }

```

```

100
101 ll kth(int k) const
102 {
103     assert(1 <= k && k <= size());
104     int p = root;
105     while (true)
106     {
107         int left = a[p].l].siz;
108         if (k == left + 1) return a[p].val;
109         if (k <= left)
110             p = a[p].l;
111         else
112         {
113             k -= left + 1;
114             p = a[p].r;
115         }
116     }
117 }
118
119 optional<ll> prev(ll x) const
120 {
121     int k = less(x);
122     if (!k) return nullopt;
123     return kth(k);
124 }
125
126 optional<ll> next(ll x) const
127 {
128     int k = less(x, true);
129     if (k == size()) return nullopt;
130     return kth(k + 1);
131 }
132 };

```

## 87.2 隐式 Treap：序列翻转、加值与区间和

insert(k,value) 在前 k 个元素后插入， $0 \leq k \leq \text{size}()$ ；erase(l,r)、reverse(l,r)、add(l,r,value)、query(l,r) 均使用 1-based 非空闭区间。values() 返回当前完整序列， $O(n)$ ，使用递归中序遍历。其余操作在随机优先级下期望  $O(\log n)$ ；翻转与加法懒标记同时维护。所有值、和、标记及  $\text{value} \times \text{长度}$  中间结果必须落在 long long 范围内。删除不回收节点，内存按累计插入数计算。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false;
11    };
12
13    vector<Node> a{Node{}};

```

```

14  mt19937_64 rng;
15  int root = 0;
16
17  SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}
18
19  int size() const { return a[root].siz; }
20
21  void pull(int p)
22  {
23      a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
24      a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
25  }
26
27  void apply_add(int p, ll x)
28  {
29      if (!p) return;
30      a[p].val += x;
31      a[p].sum += x * a[p].siz;
32      a[p].tag += x;
33  }
34
35  void apply_reverse(int p)
36  {
37      if (!p) return;
38      swap(a[p].l, a[p].r);
39      a[p].rev = !a[p].rev;
40  }
41
42  void push(int p)
43  {
44      if (a[p].rev)
45      {
46          apply_reverse(a[p].l);
47          apply_reverse(a[p].r);
48          a[p].rev = false;
49      }
50      if (a[p].tag)
51      {
52          apply_add(a[p].l, a[p].tag);
53          apply_add(a[p].r, a[p].tag);
54          a[p].tag = 0;
55      }
56  }
57
58  // Split into the first k elements and the rest.
59  void split(int p, int k, int &l, int &r)
60  {
61      if (!p)
62      {
63          l = 0;
64          r = 0;
65          return;
66      }
67      push(p);
68      int left = a[a[p].l].siz;
69      if (k > left)

```

```

70     {
71         l = p;
72         split(a[p].r, k - left - 1, a[p].r, r);
73     }
74     else
75     {
76         r = p;
77         split(a[p].l, k, l, a[p].l);
78     }
79     pull(p);
80 }
81
82 int merge(int l, int r)
83 {
84     if (!l || !r) return l ? l : r;
85     if (a[l].pri > a[r].pri)
86     {
87         push(l);
88         a[l].r = merge(a[l].r, r);
89         pull(l);
90         return l;
91     }
92     push(r);
93     a[r].l = merge(l, a[r].l);
94     pull(r);
95     return r;
96 }
97
98 // Insert after the first k elements, 0 <= k <= size().
99 void insert(int k, ll value)
100 {
101     assert(0 <= k && k <= size());
102     int p = a.size(), l, r;
103     a.push_back({0, 0, 1, value, value, 0, rng(), false});
104     split(root, k, l, r);
105     root = merge(merge(l, p), r);
106 }
107
108 // All range operations use 1-based inclusive intervals.
109 void erase(int l, int r)
110 {
111     assert(1 <= l && l <= r && r <= size());
112     int x, y, z;
113     split(root, r, y, z);
114     split(y, l - 1, x, y);
115     root = merge(x, z);
116 }
117
118 void reverse(int l, int r)
119 {
120     assert(1 <= l && l <= r && r <= size());
121     int x, y, z;
122     split(root, r, y, z);
123     split(y, l - 1, x, y);
124     apply_reverse(y);
125     root = merge(merge(x, y), z);

```

```

126     }
127
128     void add(int l, int r, ll value)
129     {
130         assert(1 <= l && l <= r && r <= size());
131         int x, y, z;
132         split(root, r, y, z);
133         split(y, l - 1, x, y);
134         apply_add(y, value);
135         root = merge(merge(x, y), z);
136     }
137
138     ll query(int l, int r)
139     {
140         assert(1 <= l && l <= r && r <= size());
141         int x, y, z;
142         split(root, r, y, z);
143         split(y, l - 1, x, y);
144         ll ans = a[y].sum;
145         root = merge(merge(x, y), z);
146         return ans;
147     }
148
149     void collect(int p, vector<ll> &out)
150     {
151         if (!p) return;
152         push(p);
153         collect(a[p].l, out);
154         out.push_back(a[p].val);
155         collect(a[p].r, out);
156     }
157
158     vector<ll> values()
159     {
160         vector<ll> answer;
161         collect(root, answer);
162         return answer;
163     }
164 };

```

# 第八十八章 伸展树

## 88.1 Splay: 可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义, 严格前驱后继不存在返回 nullopt。重复键计数存储; 失败查询也伸展最后访问节点, 单次均摊  $O(\log n)$ 。不回收节点, 空间按累计新建节点数计算。支持 long long 全范围; 不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const { return a[root].siz; }
15
16     void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }
17
18     int node(ll value, int parent)
19     {
20         int x = a.size();
21         a.push_back({{0, 0}, parent, 1, 1, value});
22         return x;
23     }
24
25     void rotate(int x)
26     {
27         int y = a[x].fa, z = a[y].fa;
28         int k = a[y].ch[1] == x;
29         int w = a[x].ch[k ^ 1];
30         if (z)
31             a[z].ch[a[z].ch[1] == y] = x;
32         else
33             root = x;
34         a[x].fa = z;
35         a[y].ch[k] = w;
36         if (w) a[w].fa = y;
37         a[x].ch[k ^ 1] = y;
38         a[y].fa = x;
39         pull(y);
40         pull(x);
41     }
42
43     void splay(int x)
44     {
```

```

45     while (a[x].fa)
46     {
47         int y = a[x].fa, z = a[y].fa;
48         if (z)
49         {
50             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
51             rotate(same ? y : x);
52         }
53         rotate(x);
54     }
55 }
56
57 // Splay the last visited node even when the value does not exist.
58 int find(ll value)
59 {
60     int x = root, last = 0;
61     while (x && a[x].val != value)
62     {
63         last = x;
64         x = a[x].ch[value > a[x].val];
65     }
66     if (x)
67         splay(x);
68     else if (last)
69         splay(last);
70     return x;
71 }
72
73 void insert(ll value)
74 {
75     int x = root, parent = 0;
76     while (x && a[x].val != value)
77     {
78         parent = x;
79         x = a[x].ch[value > a[x].val];
80     }
81     if (x)
82     {
83         ++a[x].cnt;
84         pull(x);
85     }
86     else
87     {
88         x = node(value, parent);
89         if (parent)
90             a[parent].ch[value > a[parent].val] = x;
91         else
92             root = x;
93     }
94     splay(x);
95 }
96
97 bool erase(ll value)
98 {
99     int x = find(value);
100    if (!x) return false;

```

```

101     if (a[x].cnt > 1)
102     {
103         --a[x].cnt;
104         pull(x);
105         return true;
106     }
107     int l = a[x].ch[0], r = a[x].ch[1];
108     if (l) a[l].fa = 0;
109     if (r) a[r].fa = 0;
110     if (!l)
111     {
112         root = r;
113         return true;
114     }
115     root = l;
116     x = l;
117     while (a[x].ch[1]) x = a[x].ch[1];
118     splay(x);
119     a[x].ch[1] = r;
120     if (r) a[r].fa = x;
121     pull(x);
122     return true;
123 }
124
125 int rank(ll value)
126 {
127     int x = root, last = 0, ans = 1;
128     while (x)
129     {
130         last = x;
131         if (value <= a[x].val)
132             x = a[x].ch[0];
133         else
134         {
135             ans += a[a[x].ch[0]].siz + a[x].cnt;
136             x = a[x].ch[1];
137         }
138     }
139     if (last) splay(last);
140     return ans;
141 }
142
143 ll kth(int k)
144 {
145     assert(1 <= k && k <= size());
146     int x = root;
147     while (true)
148     {
149         int left = a[a[x].ch[0]].siz;
150         if (k <= left)
151             x = a[x].ch[0];
152         else if (k <= left + a[x].cnt)
153         {
154             splay(x);
155             return a[x].val;
156         }

```

```

157         else
158         {
159             k -= left + a[x].cnt;
160             x = a[x].ch[1];
161         }
162     }
163 }
164
165 optional<ll> prev(ll value)
166 {
167     int x = root, last = 0, answer = 0;
168     while (x)
169     {
170         last = x;
171         if (a[x].val < value)
172         {
173             answer = x;
174             x = a[x].ch[1];
175         }
176         else
177             x = a[x].ch[0];
178     }
179     if (last) splay(last);
180     if (!answer) return nullopt;
181     splay(answer);
182     return a[answer].val;
183 }
184
185 optional<ll> next(ll value)
186 {
187     int x = root, last = 0, answer = 0;
188     while (x)
189     {
190         last = x;
191         if (a[x].val > value)
192         {
193             answer = x;
194             x = a[x].ch[0];
195         }
196         else
197             x = a[x].ch[1];
198     }
199     if (last) splay(last);
200     if (!answer) return nullopt;
201     splay(answer);
202     return a[answer].val;
203 }
204 };

```

# 第八十九章 状态序列维护

## 89.1 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树  $O(n)$  次 GCD，随机优先级下单次期望  $O(\log n)$  次 GCD；删除另需  $O(\text{删除长度})$  回收。一次 GCD 为  $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```
1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    int node(U value, int tag)
22    {
23        assert(tag == 0 || tag == 1);
24        int p;
25        if (recycled.empty())
26        {
27            p = a.size();
28            a.push_back(Node{});
29        }
30        else
31        {
32            p = recycled.back();
33            recycled.pop_back();
34            a[p] = Node{};
35        }
36        a[p].siz = 1;
37        a[p].val = value;
38        a[p].tag = tag;
39        a[p].cnt[tag] = 1;
40        a[p].g[tag] = value;
41        a[p].pri = rng();
```

```

42     return p;
43 }
44
45 void pull(int p)
46 {
47     int l = a[p].l, r = a[p].r;
48     a[p].siz = a[l].siz + a[r].siz + 1;
49     for (int t = 0; t < 2; t++)
50     {
51         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
52         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
53         if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
54     }
55 }
56
57 void split(int p, int k, int &l, int &r)
58 {
59     if (!p)
60     {
61         l = 0;
62         r = 0;
63         return;
64     }
65     int left = a[a[p].l].siz;
66     if (k > left)
67     {
68         l = p;
69         split(a[p].r, k - left - 1, a[p].r, r);
70     }
71     else
72     {
73         r = p;
74         split(a[p].l, k, l, a[p].l);
75     }
76     pull(p);
77 }
78
79 int merge(int l, int r)
80 {
81     if (!l || !r) return l ? l : r;
82     if (a[l].pri > a[r].pri)
83     {
84         a[l].r = merge(a[l].r, r);
85         pull(l);
86         return l;
87     }
88     a[r].l = merge(l, a[r].l);
89     pull(r);
90     return r;
91 }
92
93 void rebuild(int p)
94 {
95     if (!p) return;
96     rebuild(a[p].l);
97     rebuild(a[p].r);

```

```

98     pull(p);
99 }
100
101 void release(int p)
102 {
103     if (!p) return;
104     release(a[p].l);
105     release(a[p].r);
106     recycled.push_back(p);
107 }
108
109 // O(n) GCD calls; replaces the current sequence.
110 void build(const vector<pair<U, int>> &v)
111 {
112     a.assign(1, Node{});
113     recycled.clear();
114     root = 0;
115     vector<int> st;
116     for (auto [value, tag] : v)
117     {
118         int p = node(value, tag), last = 0;
119         while (!st.empty() && a[st.back()].pri < a[p].pri)
120         {
121             last = st.back();
122             st.pop_back();
123         }
124         a[p].l = last;
125         if (!st.empty()) a[st.back()].r = p;
126         st.push_back(p);
127     }
128     if (st.empty()) return;
129     root = st.front();
130     rebuild(root);
131 }
132
133 // Insert after the first k entries. Range operations below are 1-based.
134 void insert(int k, U value, int tag)
135 {
136     assert(0 <= k && k <= size());
137     int p = node(value, tag), l, r;
138     split(root, k, l, r);
139     root = merge(merge(l, p), r);
140 }
141
142 void erase(int l, int r)
143 {
144     assert(1 <= l && l <= r && r <= size());
145     int x, y, z;
146     split(root, r, y, z);
147     split(y, l - 1, x, y);
148     root = merge(x, z);
149     release(y);
150 }
151
152 void set(int k, U value)
153 {

```

```

154     assert(1 <= k && k <= size());
155     int x, y, z;
156     split(root, k, y, z);
157     split(y, k - 1, x, y);
158     a[y].val = value;
159     pull(y);
160     root = merge(merge(x, y), z);
161 }
162
163 void toggle(int k)
164 {
165     assert(1 <= k && k <= size());
166     int x, y, z;
167     split(root, k, y, z);
168     split(y, k - 1, x, y);
169     a[y].tag ^= 1;
170     pull(y);
171     root = merge(merge(x, y), z);
172 }
173
174 optional<U> query(int l, int r, int tag)
175 {
176     assert(1 <= l && l <= r && r <= size());
177     assert(tag == 0 || tag == 1);
178     int x, y, z;
179     split(root, r, y, z);
180     split(y, l - 1, x, y);
181     optional<U> answer;
182     if (a[y].cnt[tag]) answer = a[y].g[tag];
183     root = merge(merge(x, y), z);
184     return answer;
185 }
186 };

```

# 第九十章 一般图匹配

## 90.1 一般图最大基数匹配：带花树

Blossom(n), 点号 0..n-1; add(u,v) 加无向边, 忽略自环, 允许重边。solve() 从空匹配重新计算, 返回匹配边数; mate[u] 为配对点, 未匹配为 -1。结果支持直接输出方案。缩花维护交错树, 单次增广  $O(n^2+m)$ , 总计  $O(n^3+nm)$ , 简单图为  $O(n^3)$ ; 空间  $O(n+m)$ 。重复调用 solve 或加边后重算均可。不处理边权, 不能代替最大权匹配。

```
1 struct Blossom
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> mate, pre, base, q;
6     vector<bool> outer, flower, seen;
7
8     Blossom(int n)
9         : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
10    {
11    }
12
13    void add(int u, int v)
14    {
15        assert(0 <= u && u < n && 0 <= v && v < n);
16        if (u == v) return;
17        g[u].push_back(v);
18        g[v].push_back(u);
19    }
20
21    int lca(int u, int v)
22    {
23        fill(seen.begin(), seen.end(), false);
24        while (true)
25        {
26            u = base[u];
27            seen[u] = true;
28            if (mate[u] == -1) break;
29            u = pre[mate[u]];
30        }
31        while (!seen[base[v]]) v = pre[mate[base[v]]];
32        return base[v];
33    }
34
35    void mark(int u, int b, int v)
36    {
37        while (base[u] != b)
38        {
39            flower[base[u]] = true;
40            flower[base[mate[u]]] = true;
41            pre[u] = v;
42            v = mate[u];
43            u = pre[v];
44        }
```

```

45     }
46
47     bool augment(int root)
48     {
49         fill(outer.begin(), outer.end(), false);
50         fill(pre.begin(), pre.end(), -1);
51         iota(base.begin(), base.end(), 0);
52         q.assign(1, root);
53         outer[root] = true;
54         for (int head = 0; head < (int)q.size(); head++)
55         {
56             int u = q[head];
57             for (int v : g[u])
58             {
59                 if (base[u] == base[v] || mate[u] == v) continue;
60                 if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
61                 {
62                     int b = lca(u, v);
63                     fill(flower.begin(), flower.end(), false);
64                     mark(u, b, v);
65                     mark(v, b, u);
66                     for (int x = 0; x < n; x++)
67                     {
68                         if (!flower[base[x]]) continue;
69                         base[x] = b;
70                         if (!outer[x])
71                         {
72                             outer[x] = true;
73                             q.push_back(x);
74                         }
75                     }
76                 }
77                 else if (pre[v] == -1)
78                 {
79                     pre[v] = u;
80                     if (mate[v] == -1)
81                     {
82                         while (v != -1)
83                         {
84                             int p = pre[v];
85                             int next = mate[p];
86                             mate[v] = p;
87                             mate[p] = v;
88                             v = next;
89                         }
90                         return true;
91                     }
92                     int x = mate[v];
93                     outer[x] = true;
94                     q.push_back(x);
95                 }
96             }
97         }
98         return false;
99     }
100

```

```
101 // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
102 int solve()
103 {
104     fill(mate.begin(), mate.end(), -1);
105     int answer = 0;
106     for (int u = 0; u < n; u++)
107     {
108         if (mate[u] == -1 && augment(u)) answer++;
109     }
110     return answer;
111 }
112 };
```

# 第九十一章 线性递推

## 91.1 Berlekamp–Massey：最短线性递推

`berlekamp_massey(s,p)` 要求  $p$  为素数,  $2 \leq p \leq \text{INT\_MAX}$ , 输入均在  $[0,p)$ ; 返回  $c$ , 使  $s[i] = \sum c[j-1]s[i-j]$  ( $j=1..k, i \geq k$ )。空序列或全零序列返回空系数。有限前缀可能对应多种未来, 不能无条件外推; 若已知原序列满足阶数至多  $d$  的齐次线性递推, 提供至少  $2d$  项才能恢复可外推的递推。模运算次数  $O(Nk+N \log p)$ , 后项来自逐次求逆; 空间  $O(N)$ 。不适用于合数模数或浮点拟合。

```
1 // Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
2 vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
3 {
4     assert(p >= 2);
5     vector<int> c{1}, b{1};
6     int len = 0, shift = 1, last = 1;
7     for (int i = 0; i < (int)s.size(); i++)
8     {
9         assert(0 <= s[i] && s[i] < p);
10        long long d = s[i];
11        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
12        if (d == 0)
13        {
14            shift++;
15            continue;
16        }
17        long long inv = 1, x = last;
18        for (int e = p - 2; e; e >>= 1)
19        {
20            if (e & 1) inv = inv * x % p;
21            x = x * x % p;
22        }
23        long long factor = d * inv % p;
24        auto old = c;
25        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
26        for (int j = 0; j < (int)b.size(); j++)
27            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
28        if (2 * len <= i)
29        {
30            len = i + 1 - len;
31            b = old;
32            last = d;
33            shift = 1;
34        }
35        else
36            shift++;
37    }
38    c.resize(len + 1);
39    c.erase(c.begin());
40    for (int &x : c) x = x == 0 ? 0 : p - x;
41    return c;
42 }
```

## 91.2 特征多项式求线性递推第 k 项

`recurrence_nth(init,c,n,p)` 使用 0-based 下标,  $n$  为 unsigned long long。init 与  $c$  长度均为阶数  $k$ ;  $a[i] = \sum_{j=1}^k c[j-1]a[i-j]$ 。  $2 \leq p \leq \text{INT\_MAX}$ , 数值均在  $[0,p)$ , 本函数允许合数模数。空系数约定全零序列。计算  $x^n$  对特征多项式取余再与初值点乘,  $O(k^2 \log(n+1))$  时间、 $O(k)$  空间; 不依赖 BM 或其他模板。较大阶数可使用本库 `BostanMori::recurrence`, 须满足其 NTT 素数模数与容量条件; 本函数允许合数模数。

```

1 // init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
2 int recurrence_nth(const vector<int> &init,
3                   const vector<int> &c,
4                   unsigned long long n,
5                   int p = 998244353)
6 {
7     assert(p >= 2 && init.size() == c.size());
8     int k = c.size();
9     if (!k) return 0;
10    for (int x : init) assert(0 <= x && x < p);
11    for (int x : c) assert(0 <= x && x < p);
12    if (n < (unsigned)k) return init[n];
13    auto multiply = [&](const vector<int> &a, const vector<int> &b)
14    {
15        vector<int> t(2 * k - 1);
16        for (int i = 0; i < k; i++)
17        {
18            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
19        }
20        for (int i = 2 * k - 2; i >= k; i--)
21        {
22            for (int j = 1; j <= k; j++)
23                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
24        }
25        t.resize(k);
26        return t;
27    };
28    vector<int> answer(k), x(k);
29    answer[0] = 1;
30    if (k == 1)
31        x[0] = c[0];
32    else
33        x[1] = 1;
34    while (n)
35    {
36        if (n & 1) answer = multiply(answer, x);
37        x = multiply(x, x);
38        n >>= 1;
39    }
40    long long value = 0;
41    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
42    return value;
43 }

```

# 第九十二章 快速线性递推

## 92.1 快速递推与有理生成函数单项

BostanMori<mod=998244353,primitive=3>, 依赖参数化 NTT; 素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回  $[x^n]p(x)/q(x)$ , q 非空且  $q[0] \neq 0$ , p 可空或次数不小于 q; 令  $d=\max(p.size(),q.size())$ , 要求  $2d-1 \leq \text{NTT::max\_size}$ 。recurrence(init,c,n) 返回  $a[n]$ , 从下标 k 起  $a[i]=c[0]a[i-1]+\dots+c[k-1]a[i-k]$ , init、c 长度均为 k;  $k=0$  约定全零。非初值查询要求  $2k+1 \leq \text{NTT::max\_size}$ 。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项,  $O(d \log d \log(n+1))$  时间、 $O(d)$  空间; 递推接口  $d=k+1$ 。

```
1 template <int mod = 998244353, int primitive = 3> struct BostanMori
2 {
3     using N = NttConvolution<mod, primitive>;
4     using Z = typename N::Z;
5     using Poly = typename N::Poly;
6     using U = unsigned long long;
7
8     // Coefficient of  $x^n$  in  $p(x) / q(x)$ , with  $q[0] \neq 0$ .
9     static Z coefficient(Poly p, Poly q, U n)
10    {
11        assert(!q.empty() && q[0].v != 0);
12        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
13        while (n && !p.empty())
14        {
15            Poly r = q;
16            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
17            Poly a = N::multiply(p, r);
18            Poly b = N::multiply(q, r);
19            p.clear();
20            q.clear();
21            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
22            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
23            n >>= 1;
24        }
25        return p.empty() ? Z(0) : p[0] / q[0];
26    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
28 // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
29 static Z recurrence(const Poly &init, const Poly &c, U n)
30 {
31     assert(init.size() == c.size());
32     int k = c.size();
33     if (k == 0) return 0;
34     if (n < init.size()) return init[n];
35     assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
36     Poly q(k + 1);
37     q[0] = 1;
38     for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
39     Poly p = N::multiply(init, q);
40     p.resize(k);
41     return coefficient(p, q, n);
42 }
43 };
```

## **第二部分**

### **数学速查与维护**

## 第九十三章 数学速查：条件先于公式

本附录保留历史评测证据；其中传统版 AC 为归档记录，当前只提供 vector 模板。

### 93.1 整除、卷积与筛法

对正整数定义狄利克雷卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

单位元为  $\varepsilon(n) = [n = 1]$ ，常函数  $1(n) = 1$ ，恒等函数  $\text{id}(n) = n$ 。关键恒等式是  $\mu * 1 = \varepsilon$ 、 $\varphi * 1 = \text{id}$ 。若  $F(n) = \sum_{d|n} f(d)$ ，则  $f(n) = \sum_{d|n} \mu(d)F(n/d)$ 。

$$\sum_{i=1}^a \sum_{j=1}^b [\gcd(i, j) = 1] = \sum_{d=1}^{\min(a, b)} \mu(d) \lfloor a/d \rfloor \lfloor b/d \rfloor.$$

整除分块：当左端点为  $l$ 、商为  $q = \lfloor n/l \rfloor$ ，右端点为  $r = \lfloor n/q \rfloor$ 。同时包含多个商时，取各自右端点的最小值。杜教筛由卷积恒等式获得

$$M(n) = 1 - \sum_{l=2}^n M(\lfloor n/l \rfloor), \quad \Phi(n) = \frac{n(n+1)}{2} - \sum_{l=2}^n \Phi(\lfloor n/l \rfloor),$$

求和按等商区间合并；第二式求和中没有额外乘以  $l$ 。

### 93.2 杜教筛：预筛与缓存都不可省略

前节的两条卷积恒等式给出 DuJiao 的递推，见代码中的 `mertens` 与 `totient_sum`。把商相同的一段合并，系数为区间长度，递归参数严格变小；0 直接查表返回 0。超过预筛上限的结果在返回时写入缓存，后续相同参数直接复用。WIDA 收集的 `sumMu` 示例查询了 `fMu`，但未写入新结果；本库已有实现包含写回，不能照搬遗漏缓存的版本。

设最大查询为  $N$ ，预筛上限为  $B$ 。单次查询的常用上界为  $O(B + N/\sqrt{B})$ ，其中哈希访问按期望常数处理；取  $B$  约  $N^{2/3}$  得到  $O(N^{2/3})$ 。多个不同查询可能继续增加缓存，不能把单次空间估计当成任意多询问的上界。保留缓存可复用，主动清空仅影响重算开销；预筛表与上限应视为只读。

接口范围为  $0 \leq n \leq 10^{11}$ ，返回精确整数；欧拉函数前缀和用 128 位。WIDA 的 Z 模数结果可在精确莫比乌斯前缀和返回后取模得到。负值先求余，仅在余数为负时加模数，避免无条件加模数的溢出风险。

两套实现以独立试除构建到 200000 的前缀参考，检查四种预筛上限、零输入、乱序与重复查询、缓存写回及清空重建。上限  $10^{11}$  使用  $10^6$  与  $2 \times 10^6$  两种预筛规模核对双版本结果；这一项属于边界运行与一致性证据，不是独立答案对拍。普通与 ASan/UBSan 均通过，暂无该接口的在线 AC。

### 93.3 64 位判素与质因数分解

本库判素采用七个固定底数的 Miller-Rabin，覆盖无符号 64 位整数：

2, 325, 9375, 28178, 450775, 9780504, 1795265022.

将  $n-1 = d2^s$ 、 $d$  为奇数，检查  $a^d$  及后续平方链。小于 2 的输入不视为素数；底数模  $n$  为 0 时跳过该底数，不可误判为合数。模乘用无符号 128 位容纳完整的两个 64 位数之积。固定底数的适用范围依据公开 SPRP 记录，不外推到任意精度整数。

<https://miller-rabin.appspot.com/>

Pollard-Rho 用  $f(x) = x^2 + c \pmod n$  迭代，并求  $\gcd(|x - y|, n)$ 。只有  $1 < d < n$  的因子才返回；一次尝试超过轮数上限或得到  $d = n$ ，就换参数重试。其中模平方后的加法也在 128 位中执行，避免  $n$  接近  $2^{64}$  时加  $c$  溢出。该过程保证返回真因子，但随机尝试次数没有确定的最坏上界。递归分解两侧，直到判为素数，最后排序并保留重复因子。

调用 `factor(1)` 得到空数组；对  $n \geq 2$ ，返回单元素说明  $n$  为素数。POJ 1811 所需的最小质因子可取排序结果首项；P4718 所需的最大质因子可取末项。不要对空数组取首尾，也不要绕过 `factor` 把素数传给内部 `rho`。

双版本对百万范围以独立埃氏筛核验判素，对  $1 \dots 10000$  以试除核验完整因子列表。另验证三个随机种子下的大质数平方、重复因子、强伪素数和  $2^{64} - 1$ ；期望因子的素性均以试除独立验证。 $2^{61} - 1$  使用完整分解  $n - 1$  的 Lucas 素性证书检查；模乘与模幂另以任意精度运算对拍。完整 POJ 1811 与 P4718 驱动分别验证 350 组输入，检查素数标签与最小、最大质因子。P4718 压力批次含 300 个接近  $10^{18}$  的平衡半素数，本机普通编译运行约 1.99、1.83 秒；机器差异与随机路径都会影响耗时，本地运行时间不直接代表线上性能。两套代码随后通过 P4718 全部 20 个测试点，最长单点显示约 2.11、2.12 秒，低于当前页面的 4 秒限制。该 AC 检查素数标签与最大质因子，范围为  $2 \leq n \leq 10^{18}$ 。完整因子重数、最小质因子、 $n = 1$  与更大的无符号 64 位输入仍依据本地验证；普通与 ASan/UBSan 均通过，不构成随机运行时间保证。

来源：kuangbin 2.6；固定底数记录见上方链接。

## 93.4 单个数的欧拉函数

实现见第 114 页。对  $n \geq 1$ ，由容斥得到

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right),$$

其中只枚举不同质因子，故质数幂  $p^k$  的贡献为  $p^{k-1}(p - 1)$ 。每次更新按先除后乘的顺序： $\text{ans} \leftarrow (\text{ans}/p)(p - 1)$ 。当前尚未处理的质因子仍整除答案，而且答案只会减小，乘法结果不会超过原始  $n$ 。

无分解器参数的入口采用试除，循环上界用  $p \leq n/p$ ，剩下的  $n > 1$  再处理一次。这能避免  $p^2$  溢出，但不消除  $O(\sqrt{n})$  的耗时；不适合试除 64 位大素数。传入现有 PollardRho 对象时先分解，再在排序后的因子列表上去重更新；不重复构造分解器，后续调用沿用其随机状态。该入口没有确定的最坏运行时间。 $\varphi(1) = 1$ ，两个入口均接受完整的正无符号 64 位输入；0 不在定义域内。

本地双版本验证包含  $1 \dots 20000$  的线性筛交叉检查、小范围直接计数互质整数，以及重复质因子、大质数平方、 $2^{64} - 1$  和已独立认证的大素数。普通与 ASan/UBSan 测试均通过；P4718 的 AC 只验证底层分解用途，不是此欧拉函数的新 AC。来源：kuangbin 2.7.1、2.7.3；WIDA 的单个数欧拉函数。

## 93.5 线性筛：三张表的约定

LinearSieve 同时构造最小质因子、欧拉函数和莫比乌斯函数。内层遇到第一个整除  $i$  的素数后停止，因此每个合数  $j$  只由  $p = \text{lp}(j)$  与  $i = j/p$  生成一次，时间为  $O(n)$ 。若  $p \mid i$ ，则  $\varphi(ip) = p\varphi(i)$ 、 $\mu(ip) = 0$ ；否则二者分别为  $(p - 1)\varphi(i)$  和  $-\mu(i)$ 。

判素使用  $x \geq 2$  且  $\text{lp}(x) = x$ ，不能漏掉前半条件。分解表内正整数时，反复取  $p = \text{lp}(x)$ 、令  $x \leftarrow x/p$ ，直到  $x = 1$ ；重复因子会重复输出。0 不可分解，1 的质因子列表为空。上限要求  $0 \leq n < \text{INT\_MAX}$ ，避免构造时  $n + 1$  溢出，并保证内存可用。筛乘积前先检查  $p \leq n/i$ ，不直接计算可能溢出的  $ip$ 。

双版本验证至  $n = 10^7$ ，包括小前缀、试除最小质因子、因子还原和逐项约数和证书：

$$\sum_{d|x} \varphi(d) = x, \quad \sum_{d|x} \mu(d) = [x = 1].$$

这些恒等式在  $d = x$  项系数为 1，可按  $x$  递增唯一确定两张表。普通与 ASan/UBSan 均通过；不据此宣称莫比乌斯反演的所有应用已验证。

来源：kuangbin 2.1.1、2.1.2、2.7.2、2.7.4、2.13.1；WIDA 的全部欧拉函数表。

WIDA 的“使用莫比乌斯反演求解欧拉函数”先存储全部约数，再使用

$$\varphi(n) = n - \sum_{d|n, d < n} \varphi(d).$$

这与  $\varphi * 1 = \text{id}$  等价；再卷积  $\mu$  可得  $\varphi(n) = \sum_{d|n} \mu(d)(n/d)$ 。本库用线性筛提供同一张表，无需额外保存约数列表。百万范围另分别运行直接卷积和真约数扣减，逐项核对两套筛表；两种方法都不复用线性筛的递推分支。

### 93.6 区间筛：只保存目标区间

实现见第 103 页。设  $0 \leq L \leq R \leq \text{INT\_MAX}$ ，先筛出不超过  $\lfloor \sqrt{R} \rfloor$  的素数，再对每个素数  $p$  从

$$\max(p^2, \lceil L/p \rceil p)$$

开始标记区间内的倍数。任何合数都有不超过自身平方根的素因子，所以不会遗漏；从  $p^2$  开始也不会误删  $p$  自身。输出跳过 0 和 1，其余未标记数恰为素数。底筛与区间标记共用同一个外层素数循环，但使用两张独立的表。

令  $W = R - L + 1$ ，空间为  $O(\sqrt{R} + W)$ ，包含返回的素数数组；时间为  $O((\sqrt{R} + W) \log \log(R + 3))$ 。函数每次调用都重建底筛，适合题目中的少量区间查询。端点虽然是 `int`，向上取整的加法、 $p^2$ 、标记变量和输出遍历变量均用 `long long`；否则  $R = \text{INT\_MAX}$  时最后一次递增也会溢出。区间长度仍受可用内存约束，不适合一次保存整个 32 位值域。

POJ 2689 与 UVa 10140 的驱动读到 EOF，扫描相邻素数差。最近、最远都只在严格更优时更新，距离并列保留最先出现的一对；少于两个素数时输出题目指定句子。两套实现以试除和 Miller–Rabin 判素进行独立对拍，并检查平方边界、单点区间及右端点为 `INT_MAX` 的百万长度区间；完整驱动验证 208 组输入，含官方样例。普通测试与 ASan/UBSan 均通过；尚无本库在线 AC。

来源：kuangbin 2.1.3；<https://onlinejudge.org/external/101/10140.pdf>。

### 93.7 矩形 GCD 计数：缩放后容斥

实现见第 110 页。由  $[g = 1] = \sum_{d|g} \mu(d)$ ，前缀矩形互质数对数为

$$F(A, B) = \sum_{i=1}^{\min(A, B)} \mu(i) \left\lfloor \frac{A}{i} \right\rfloor \left\lfloor \frac{B}{i} \right\rfloor.$$

两个商同时不变的右端点取  $r = \min(A/\lfloor A/l \rfloor, B/\lfloor B/l \rfloor)$ ，用莫比乌斯前缀和合并  $[l, r]$ 。不能只按一个商分块。只需筛到  $\min(A, B)$ ，不要求较长的一边也在预处理范围内。 $A = 0$  或  $B = 0$  时循环为空，返回 0；正整数条件不包含坐标轴。

矩形内  $\gcd(x, y) = k$  等价于  $x, y$  均为  $k$  的倍数，且缩放后互质。因此答案为四个前缀矩形的容斥：

$$\begin{aligned} & F(\lfloor b/k \rfloor, \lfloor d/k \rfloor) - F(\lfloor (a-1)/k \rfloor, \lfloor d/k \rfloor) \\ & - F(\lfloor b/k \rfloor, \lfloor (c-1)/k \rfloor) + F(\lfloor (a-1)/k \rfloor, \lfloor (c-1)/k \rfloor). \end{aligned}$$

这里必须先减 1 再除以  $k$ ；零商无需特殊补偿。输入端点为正 `int`，最终答案不超过矩形面积，能用有符号 64 位保存。分块累加使用 128 位，避免依赖正负中间项的额外界。

双版本以小矩形逐对求  $\gcd$  为独立参考，另验证百万正方形的  $F(N, N) = 2 \sum_{i=1}^N \varphi(i) - 1$ ，以及按短边剩余类直接计数的 `INT_MAX` 长条。P2522 双版本完整程序验证五万次查询，包括缩放、窄边和无解；普通与 ASan/UBSan 均通过。尚无本库在线 AC。

来源：kuangbin 2.13.2 (BZOJ 2301)；<https://www.luogu.com.cn/problem/P2522>。

### 93.8 整数分拆：重数限制的差一问题

实现见第 116 页。分拆不区分加数顺序，部分均为正整数。无约束生成函数  $P(z) = \prod_{i \geq 1} (1 - z^i)^{-1}$ ，空分拆给出  $p(0) = 1$ 。五边形数定理为

$$\prod_{i \geq 1} (1 - z^i) = 1 + \sum_{j \geq 1} (-1)^j \left( z^{j(3j-1)/2} + z^{j(3j+1)/2} \right).$$

与  $P(z)$  相乘后比较系数，得到  $p(n)$  的递推，符号按  $+, +, -, -, +, +, \dots$  成对出现。每项只依赖更小的下标，每个  $n$  有  $O(\sqrt{n})$  个非零偏移。实现先用 64 位累加，再归一化到模数范围；无需逆元，合数模数与模数 1 均可使用。

每个正整数最多出现  $k$  次的生成函数是

$$\prod_{i \geq 1} (1 + z^i + \dots + z^{ki}) = P(z) \prod_{i \geq 1} (1 - z^{(k+1)i}).$$

因此用已有  $p$  表与放大  $k+1$  倍的五边形偏移相乘，符号为  $-, -, +, +, \dots$ 。本库 `limited(n,k)` 的上限包含  $k$ ； $k=0$  时非零  $n$  无解， $k=1$  为互异分拆。kuangbin 的注释写“不超过  $k$  个”，但其 `solve(n,k)` 使用放大  $k$  倍的公式，实际表示每种数少于  $k$  次，对应本库 `limited(n,k-1)` ( $k \geq 1$ )。HDU 原题页面本次无法读取，不能由这条注释推断原题措辞。

两套实现以任意精度完全背包验证到 1000，以逐种部分、逐个重数枚举验证有界情况到 100，覆盖素数与合数模数、0/1 重数限制和大于总和的限制；另检查  $N = 100000$  的边界。普通与 ASan/UBSan 均通过，暂无在线 AC。

来源：kuangbin 2.11；<https://oi-wiki.org/math/combinatorics/partition/>。

### 93.9 幂的约数和：不除以 $p-1$

实现见第 113 页。若  $a = \prod p_i^{e_i}$ ，则  $a^b$  的约数和是  $\prod_i (1 + p_i + \dots + p_i^{e_i b})$ 。几何级数的除法公式在模意义下要求分母可逆；合数模数或  $p \equiv 1$  时不能直接套用。

令  $G(a, n)$  同时返回  $P = a^n$  与  $S = \sum_{i=0}^{n-1} a^i$ 。先递归求一半项数  $h = \lfloor n/2 \rfloor$  的  $(P, S)$ ；偶数项的结果为  $(P^2, S(1+P))$ ，奇数项再补最后的  $a^{2h} = P^2$ ，得到  $(P^2 a, S(1+P) + P^2)$ ，全过程取模。 $n=0$  的结果为  $(1, 0)$ ，因此底数为 0 时，非空级数的和仍为 1。本接口参数是项数，不是最高指数；约数和必须传  $e_i b + 1$ 。

把幂与和一起返回，可使每层只做常数运算，总时间  $O(\log(n+1))$ 。长度允许无符号 128 位，避免  $e_i b$  在 64 位中溢出；模数及底数允许完整无符号 64 位。所有模乘与必要的加法在 128 位中执行，任意正模数均可用。底数  $a \geq 1$ ， $b=0$  或  $a=1$  都返回  $1 \bmod m$ ；0 的约数和不在接口定义域内。

双版本以直接枚举约数、逐项几何求和和任意精度矩阵幂独立核验，包括模数 1、合数模数、完整 128 位项数，以及指数乘重数超过 64 位的案例。P1593 完整程序各验证 126 组输入，包含  $p \equiv 0, 1 \pmod{9901}$  和最大输入；普通与 ASan/UBSan 均通过，在线 AC 待补。

来源：kuangbin 2.12；<https://www.luogu.com.cn/problem/P1593>。

### 93.10 类欧几里德：有符号整除求和

`NumberTheory::floor_sum(n,m,a,b)` 返回

$$F(n, m, a, b) = \sum_{i=0}^{n-1} \left\lfloor \frac{ai + b}{m} \right\rfloor.$$

范围约定  $0 \leq n \leq 10^9$ 、 $1 \leq m \leq 10^9$ ， $a, b$  支持完整有符号 64 位，返回 128 位。 $n$  是项数；若题目下标为  $0 \dots N$ ，需传  $N+1$  并检查仍在约定范围内。

先令  $a = q_a m + r_a$ 、 $b = q_b m + r_b$ ，其中余数属于  $[0, m)$ ，答案先加上  $q_a n(n-1)/2 + q_b n$ 。C++ 的负整数除法向零截断，因此余数为负时需要把商减 1，再用 128 位乘法计算规范余数。这一步不能简单用两次取模替代后丢弃的贡献。

对规范化后的  $0 \leq a, b < m$ , 设  $y = an + b$ 。若  $y < m$ , 余下贡献为 0; 否则交换格点计数的求和方向可得

$$F(n, m, a, b) = F(\lfloor y/m \rfloor, a, m, y \bmod m).$$

下一轮先提取新的整商, 再继续交换。进入交换分支时  $a > 0$ , 不会产生零分母。模数按欧几里得过程减小, 时间  $O(\log(m+1))$ , 额外空间  $O(1)$ 。

双版本以任意精度向下取整逐项求和对拍, 覆盖负系数、零项数和整除边界; 十亿项的常数、连续整数、反向连续整数及整倍数斜率使用闭式公式独立核验。同时检查模数 1 下两个系数的 64 位极值。普通与 ASan/UBSan 均通过, 暂无在线 AC。本节只验证基本整除和; 下节单独给出带下标权与平方和的接口及证据。

### 93.11 类欧几里德的三个求和量

实现见第 111 页。令  $y_i = \lfloor (ai + b)/m \rfloor$ , 维护  $f = \sum y_i$ ,  $g = \sum iy_i$ ,  $h = \sum y_i^2$ , 下标均为  $0 \leq i < n$ 。结果固定模 998244353, 不要更换模数却保留 2、6 的逆元常数。

先把  $a = xm + a'$ ,  $b = ym + b'$  归一化; 设规范化问题返回  $(f_0, g_0, h_0)$ , 并记  $S_1 = \sum i$ ,  $S_2 = \sum i^2$ , 则

$$\begin{aligned} f &= f_0 + xS_1 + yn, \\ g &= g_0 + xS_2 + yS_1, \\ h &= h_0 + x^2S_2 + y^2n + 2xyS_1 + 2xg_0 + 2yf_0. \end{aligned}$$

负系数的商仍须向下取整。规范化后的  $a' = 0$  时三个剩余量均为 0。

对  $0 < a' < m$ , 令  $t = \lfloor (a'(n-1) + b')/m \rfloor$ 。高度  $j = 1 \dots t$  首次出现的下标为  $r_j = \lceil (mj - b')/a' \rceil$ 。以  $q = j-1$  编号, 递归参数为  $(t, a', m, m + a' - b' - 1)$ ; 设返回  $(u, v, w)$ , 即  $u = \sum r_j$ ,  $v = \sum (j-1)r_j$ ,  $w = \sum r_j^2$ , 则

$$f_0 = nt - u, \quad g_0 = tS_1 - (w - u)/2, \quad h_0 = nt^2 - 2v - u.$$

这来自按高度计算每一层的贡献, 其中  $y_i^2 = \sum_{j=1}^{y_i} (2j-1)$ 。分母每次严格减小, 所以只有  $O(\log(m+1))$  层递归。

P5170 的上界包含  $n$ , 驱动传入  $n+1$ ; 题目输出顺序为  $f, h, g$ , 与本接口不同。核心函数支持十亿加一项, 并对完整有符号 64 位系数做任意精度逐项对拍; 大项数用一次多项式的和与平方和闭式检查。两套完整程序各验证十万组输入, 检查最大闭区间端点与全部三个输出。普通与 ASan/UBSan 均通过, 在线 AC 待补; 任意更高次矩尚不在这个接口内。

补充的大参数测试包含 3000 组随机参数、64 位系数端点和斐波那契型欧几里得链。反转  $i \mapsto n-1-i$  应保持  $f, h$  不变, 并使  $g$  变为  $(n-1)f - g$ 。将每项替换为  $-y_i - 1$ , 三个结果应变为  $-f - n$ ,  $-g - S_1$ ,  $h + 2f + n$ 。相邻前缀还应满足增量  $(y_n, ny_n, y_n^2)$ , 其中新增项以任意精度计算。第一结果另与独立接口 `floor_sum` 交叉核对。这些恒等式测试是大参数一致性证据, 与前述逐项独立答案对拍区分记录; 普通与 ASan/UBSan 均通过。

来源与题目: <https://www.luogu.com.cn/problem/P5170>。

### 93.12 模运算的适用条件

模逆元存在当且仅当  $\gcd(a, m) = 1$ 。费马逆元  $a^{p-2}$  仅适用于素数模数及  $a \not\equiv 0$ 。对正模数  $m, n$ , 同余式  $x \equiv a \pmod{m}$ 、 $x \equiv b \pmod{n}$  可合并当且仅当  $\gcd(m, n) \mid (b-a)$ ; 新模数为  $\text{lcm}(m, n)$ 。

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p} \quad (p \text{ 为素数, } n_i, k_i \text{ 是 } p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。阶乘与逆阶乘预处理要求上限小于模数; 超过模数会出现阶乘为零。扩展欧拉降幂中, 非互质情况不能直接将指数对  $\varphi(m)$  取模; 需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

### 93.13 任意数组的批量逆元

设  $s_i = \prod_{j < i} a_j \pmod{m}$ ,  $s_0 = 1$ . 仅对总乘积  $s_n$  求一次逆元, 随后从右向左处理. 若当前  $t = s_{i+1}^{-1}$ , 则

$$a_i^{-1} = ts_i \pmod{m}, \quad s_i^{-1} = ta_i \pmod{m}.$$

先保存原  $a_i$  再覆盖输出, 避免误用已写入的逆元更新  $t$ . 总乘积与  $m$  互素当且仅当每一项均与  $m$  互素, 因此一次扩展欧几里得也能判断整个批次是否可行; 模数无需为素数. 实现见第 104 页.

接口返回可选向量: 无解是空的可选值, 空输入是有效的空向量, 两者不同. 模数为 1 时按接口约定返回全零. 调用者可以移动不再使用的输入数组, 避免额外复制. P5431 的驱动程序计算  $\sum_{i=1}^n k^i/a_i$ , 注意权重从  $k^1$  开始. 五百万项规模采用缓冲的字符读取, 并移动输入以控制峰值内存. 双版本已做乘法证书和非互素拒绝的穷举检查, 以及 64 位极值、空数组、模数 1 的检查; 完整驱动另用逐项逆元和几何级数进行对拍. P5431 双版本均通过全部 5 个测试点; 动态版记录 297561197, 传统版记录 297561804. 最慢单点分别为 386、384 毫秒, 峰值内存均为 76.80 MB. 合数模数、不可逆输入和完整 64 位范围以上本地测试验证.

该方法可处理任意可逆数组; 连续整数的专用递推见下一节.

来源: <https://oi-wiki.org/math/number-theory/inverse/>.

### 93.14 连续整数的线性逆元表

素数模数  $p$  下, 对  $1 < i < p$ , 写  $p = qi + r$ , 其中  $q = \lfloor p/i \rfloor$ ,  $r = p \bmod i$ .  $r$  非零, 且  $r < i$ . 将等式模  $p$  后乘以  $i^{-1}r^{-1}$ , 得到

$$i^{-1} \equiv -\lfloor p/i \rfloor (p \bmod i)^{-1} \pmod{p}.$$

从  $1^{-1} = 1$  开始递推, 右侧总是已算过的较小下标, 故总时间为  $O(N)$ . 实现见第 105 页; 第 0 项只作占位, 允许  $N = 0$ .  $N \geq p$  时区间包含无逆元的  $p$ , 不能套用该表.

递推还依赖  $r$  可逆, 不能仅凭  $i$  与合数模数互素就照搬. 例如  $i = 3, m = 8$  时  $r = 2$  不可逆. kuangbin 的“简洁写法”递归式也有这一限制, 本书以一般扩展欧几里得接口代替其不成立的合数推广. 若  $\gcd(a, m) = 1$ , 扩展欧几里得给出  $ax + my = 1$ , 归一化  $x$  即为逆元; 若不互素, 则整数方程无解. 素数下也可用  $a^{p-2}$  求逆元; kuangbin 标为“利用欧拉函数”的代码实际使用这一费马形式.

P3811 双版本完整程序逐行输出逆元, 已在本地对三百万行结果逐项检验  $ix_i \equiv 1$ . P3811 双版本均通过全部 6 个测试点, 单点最长分别为 221 ms 与 219 ms. 单个逆元另用小模数穷举与费马乘法证书交叉验证; 这部分不由 P3811 的 AC 覆盖.

来源: <https://oi-wiki.org/math/number-theory/inverse/>; WIDA 数论逆元节与 kuangbin 2.4.

### 93.15 一次不定方程：特解与正整数解范围

实现见第 101 页. 对不全为零的  $a, b$ , 令  $g = \gcd(|a|, |b|) > 0$ . 递归扩展欧几里得先求  $|a|u + |b|v = g$ ; 当且仅当  $g \mid c$  时有整数解. 把系数乘以  $c/g$ , 再恢复  $a, b$  的符号, 就得到  $ax_0 + by_0 = c$ . 两个特解相减, 再利用  $a/g, b/g$  互素, 可知所有解恰为

$$x = x_0 + \frac{b}{g}t, \quad y = y_0 - \frac{a}{g}t, \quad t \in \mathbb{Z}.$$

若  $a = b = 0$ , 则  $c = 0$  时所有整数对都是解, 否则无解; 此时不能使用除以  $g$  的通解式.

底层 `exgcd` 的本库调用约定为非负系数, 上限  $2^{63}$ ; 返回非负的  $g$ ,  $(0, 0)$  约定返回 0. 包装函数先转为 128 位再取相反数, 因而可以处理 `LLONG_MIN`. 缩放后的特解也用 128 位保存; 不要提前转回 `long long`. 即使特解可表示, 直接验证  $ax + by$  的两个乘积仍可能超过 128 位, 本地验证使用任意精度整数.

对 P5656 的正系数  $a, b, c$ , 设  $d_x = b/g, d_y = a/g$ , 先将  $x_0$  调整到  $x_{\min} \in [1, d_x]$ , 并计算与它配对的  $y_{\max} = (c - ax_{\min})/b$ . 若  $y_{\max} > 0$ , 正整数解个数为

$$N = \left\lfloor \frac{y_{\max} - 1}{d_y} \right\rfloor + 1, \quad x_{\max} = x_{\min} + (N - 1)d_x, \quad y_{\min} = y_{\max} - (N - 1)d_y.$$

否则不存在同时为正的解，但  $x$  和  $y$  仍各自有最小正值；分别按周期归一化即可，它们通常不是同一组解。正值归一化使用  $(v \% d + d - 1) \% d + 1$ ，不要把余数 0 当作正整数。驱动中  $a, b, c \leq 10^9$ ，归一化后的乘积不超过  $10^{18}$ ，故该部分可用 64 位。

双版本以小范围有符号输入穷举、任意精度等式证书、64 位端点和斐波那契递归链验证。完整 P5656 驱动另验证 200000 组输入，包含正解数量与四个极值、无解和无正解分支；普通与 ASan/UBSan 测试均通过，在线 AC 待补。

来源：kuangbin 2.3；WIDA 数论打印稿的 `exgcd` 与不定方程例题。

<https://oi-wiki.org/math/number-theory/bezouts/>。

### 93.16 线性同余：最小解与最小周期

实现见第 102 页，接口约定为  $ax + b \equiv 0 \pmod{m}$ 。先把  $a$  归一化到  $[0, m)$ ，令  $g = \gcd(a, m)$ 。有解当且仅当  $g \mid b$ ；若  $au + mv = g$ ，则一个特解为  $u(-b/g)$ ，所有解为  $x = x_0 + (m/g)t$ 。归一化到  $[0, m/g)$  后得到最小非负解。除以  $g$  后  $a/g$  与  $m/g$  互素，因而  $m/g$  是最小正周期。当  $a = 0$  且  $m \mid b$  时所有整数均为解，返回  $(0, 1)$ ；否则无解。模数  $m = 1$  始终返回  $(0, 1)$ 。

输入常数项支持 `LLONG_MIN`，内部用 `-i128(b)`，不能先执行 64 位的 `-b` 再转换。若题目给出  $ax \equiv c \pmod{m}$ ，可先将  $c$  归一化到  $[0, m)$  再取负传入，避免  $c = \text{LLONG\_MIN}$  时在调用点溢出。本接口依赖的 `exgcd` 与前节相同，不必重复抄写。

P1516 中相遇条件为  $(m - n)t + (x - y) \equiv 0 \pmod{L}$ 。传入速度差与起点差，第一返回值就是最早非负相遇时间；失败时输出 `Impossible`。不能擅自把负的速度差取绝对值，也不能把两者同余的初始位置强制判为需要跳跃。

小模数测试逐个枚举全部剩余类，检查解集、最小解和最小周期；大数用任意精度等式证书验证。完整驱动以直接模拟、交换青蛙和构造大周期最早相遇时间验证 242 组输入，包括等速和初始位置同余。两套代码均通过普通与 ASan/UBSan 测试，在线 AC 待补。

来源：WIDA 收集的 `jiangly exgcd` 文件内 `sol`；<https://www.luogu.com.cn/problem/P1516>。

### 93.17 扩展 CRT：逐式合并

kuangbin 2.5 的 `solve` 对应单次合并，MLES 对应下面的逐式循环。实现见第 94 页的 `crt`，初始取  $r = 0, m = 1$ ，依次合并  $x \equiv b \pmod{n}$ 。空方程组保持  $(0, 1)$ ，表示任意整数；一旦无解即停止，不能忽略失败继续合并。先把余数归一化。设  $g = \gcd(m, n)$ ，由扩展欧几里得得到  $mu + nv = g$ 。写  $x = r + mt$ ，原问题化为  $mt \equiv b - r \pmod{n}$ 。若  $g \nmid (b - r)$  则无解，否则

$$q = n/g, \quad t \equiv \frac{b - r}{g}u \pmod{q}, \quad 0 \leq t < q.$$

新周期为  $mq$ ，新余数为  $(r + mt) \bmod (mq)$ 。所有可行  $t$  相差  $q$ ，故新余数加新周期的任意整数倍恰好是全部解；逐式合并时这一不变量保持成立。模数不必互素，模数 1 也可直接参与。

参考实现使用 `int` 保存合并状态，本库扩展为 64 位并显式报告周期越界；避免可能溢出的 `abs(a - a0)`。

代码要求正模数且最终周期不超过有符号 64 位上限；中间乘法与扩展欧几里得系数使用 128 位整数。无解返回假；有解而周期越界抛出异常。失败时旧周期不变，旧余数已归一化。

$k$  条方程的时间为  $O(k \log M)$ ，其中  $M$  是输入及中间模数的上界；递归欧几里得使用  $O(\log M)$  栈空间。

P4777 的完整程序按“模数、余数”读入，题目保证有解且周期不超过  $10^{18}$ 。本地验证包含双版本小模数穷举、任意精度整数对拍、有符号极值、无解与越界，以及十万条同余式的完整程序测试。另穷举至多三个小模数方程组成的所有有序系统，检查空系统与后续矛盾。双版本均已通过 P4777 全部 15 个测试点；动态版记录 297521102，传统版记录 297521173。

kuangbin 的“解同余方程组”是模 7 的线性方程组消元应用，见后面的工厂工期一节。来源：<https://oi-wiki.org/math/number-theory/crt/>；WIDA 数论模板的 `exCRT` 节。

## 93.18 Garner: 总周期可以超过整数范围

设正模数  $m_i$  两两互素,  $P = \prod_i m_i$ 。经典构造为  $x = \sum_i b_i P_i (P_i^{-1} \bmod m_i) \bmod P$ , 其中  $P_i = P/m_i$ 。模  $m_i$  时其他项都为零, 本项为  $b_i$ ; 两解之差被所有  $m_i$  整除, 因而被  $P$  整除。但直接写此式需要存储整个  $P$ , 并防止乘法溢出。

Garner 改用混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \quad 0 \leq d_i < m_i.$$

在确定第  $i$  位前, 对每个后续模数维护已确定部分的余数  $v_j$ , 以及当前位权  $c_j = \prod_{t < i} m_t \bmod m_j$ 。于是

$$d_i = (b_i - v_i) c_i^{-1} \bmod m_i.$$

把  $c_j d_i$  加到  $v_j$ , 再将  $c_j$  乘  $m_i$  即可进入下一位。互素性保证逆元存在; 模数 1 的位固定为 0, 可直接保留。位数范围保证重建的  $x$  恰在  $[0, P)$ , 因此就是最小非负解。实现见第 106 页。

若只需  $x \bmod T$ , 从高位向低位做 Horner 计算: 每步将当前值乘对应模数, 加上该位, 再对  $T$  取模。 $T$  无需与  $m_i$  互素, 因为这里不对  $T$  求逆。这返回的是规范解  $x$  的余数; 一般解  $x + kP$  对  $T$  的余数可能不同。例如模 3 余 2、模 5 余 3, 规范解是 8; 对 7 取模为 1, 但另一个解 23 的余数是 2。若用多模 NTT 恢复整数系数, 仍须证明真实系数位于所选规范区间, 不能在乘积不够大时直接把 CRT 的代表元当成原整数。

双版本本地验证使用任意精度整数的经典 CRT 求和作为独立对拍, 检查混合进制的完整重建、负余数、有符号极值、非互素输入拒绝和 1000 个模数。P1495 完整程序按题目保证的  $P \leq 10^{18}$  输出规范解, 双版本均通过全部 11 个测试点; 动态版记录 297521594, 传统版记录 297521658。超出该题范围的能力以上述本地测试验证。来源: <https://oi-wiki.org/math/number-theory/crt/>。

## 93.19 素数模平方根

奇素数  $p$  下, 对  $n \not\equiv 0 \pmod{p}$ , Euler 判别式  $n^{(p-1)/2} \equiv 1 \pmod{p}$  当且仅当存在平方根, 否则为  $-1$ 。当  $p \equiv 3 \pmod{4}$  且有解时,  $r = n^{(p+1)/4}$  为一根, 另一根是  $p - r$ 。

Cipolla 选择  $a$  使  $w = a^2 - n$  是非平方, 令  $\omega^2 = w$ 。在扩域中取  $\alpha = a + \omega$ , 有  $\alpha^p = a - \omega$ , 故  $\alpha^{p+1} = n$ 。令  $\beta = \alpha^{(p+1)/2}$ , 则  $\beta^2 = n$  且  $\beta^{p-1} = n^{(p-1)/2} = 1$ , 所以  $\beta$  落在原素数域中。实现见第 121 页; 零和模数 2 单独处理。模数为合数时不能照搬此判别和扩域构造。

## 93.20 乘法阶与原根

对  $\gcd(a, n) = 1$ , 阶是最小的正整数  $d$  使  $a^d \equiv 1 \pmod{n}$ , 且  $d \mid \varphi(n)$ 。从  $d = \varphi(n)$  开始, 对每个素因子  $q$ , 只要  $a^{d/q} \equiv 1$  就继续除去  $q$ , 所得即为实际阶。对于不可逆的  $a$ , 接口返回 0, 表示此处没有定义乘法阶。

原根要求阶达到  $\varphi(n)$ 。只需对其每个不同素因子  $q$  检查  $a^{\varphi(n)/q} \not\equiv 1$ : 若实际阶是真因子, 商中必含某个这样的  $q$ 。必须先检查  $\gcd(a, n) = 1$ ; 例如模 9 的候选 3 虽通过上述非 1 判定, 却不是原根。 $n \geq 2$  时, 原根恰存在于  $2, 4, p^k, 2p^k$ , 其中  $p$  是奇素数。原根存在且取一个原根  $g$  后,  $g^t$  的阶是  $\varphi(n)/\gcd(t, \varphi(n))$ , 所以枚举与  $\varphi(n)$  互素的指数即可得到全部原根, 共  $\varphi(\varphi(n))$  个; 这些数按大小输出时还需排序。

实现见第 107 页。本地验证穷举小模数的幂循环, 与双版本的阶、最小值和完整集合逐项比较; 另验证大素数、奇素数幂、无原根合数和有符号输入边界。P6091 的完整程序输出排序后第  $d, 2d, \dots$  个原根, 零个输出时仍保留空行。双版本均通过全部 11 个测试点; 动态版记录 297558740, 传统版记录 297559082。求阶、最小原根的最小性及更大输入范围仍以上述本地证据验证。若  $d = \text{ord}_n(a)$ , 则  $\text{ord}_n(a^k) = d/\gcd(d, k)$ 。两个可逆元素乘积的阶整除各自阶的最小公倍数; 两阶互素时, 乘积的阶等于两阶之积。来源: <https://oi-wiki.org/math/number-theory/primitive-root/>; kuangbin 2.17。

### 93.21 Carmichael: 最小公共指数

$\lambda(n)$  是模  $n$  的所有可逆元素之阶的最小公倍数, 约定  $\lambda(1) = 1$ 。它整除  $\varphi(n)$ , 但可以严格更小, 例如  $\lambda(8) = 2$  而  $\varphi(8) = 4$ 。按互素的素数幂分解  $n = \prod_i p_i^{e_i}$  后, 由 CRT, 对每个分量同时成立的幂等式也对  $n$  成立, 因此

$$\lambda(n) = \text{lcm}_i \lambda(p_i^{e_i}), \quad \lambda(p^e) = \begin{cases} 2^{e-2}, & p = 2, e \geq 3, \\ p^{e-1}(p-1), & \text{其他情形.} \end{cases}$$

奇素数幂的单位群有原根, 公共指数即其大小。模  $2^e$  的奇数可表示为  $\pm 5^k$ , 当  $e \geq 3$  时 5 的阶为  $2^{e-2}$ , 这给出二幂情形的上界和达到上界的元素。实现见第 115 页, 试除版本适用于有符号 32 位正模数。

对可逆底数可把指数对  $\lambda(n)$  取模; 非互素底数仍须另行处理。例如模 8 下  $2^2 = 4$ , 但  $2^{2 \bmod \lambda(8)} = 1$ , 直接降幂会错误。另有  $\lambda(\text{lcm}(a, b)) = \text{lcm}(\lambda(a), \lambda(b))$ , 这也提供了独立的组合检查条件。

合数  $n$  是 Carmichael 数当且仅当  $\lambda(n) \mid (n-1)$ 。等价的 Korselt 判别是:  $n$  无平方因子, 且每个素因子  $p$  都满足  $(p-1) \mid (n-1)$ 。素数也满足前一整除式, 判定时必须排除素数。这些合数对所有互素底数都通过 Fermat 判定, 不能用单纯的 Fermat 检验代替素性判定。无平方因子再配合 CRT, 也使所有整数  $a$  满足  $a^n \equiv a \pmod{n}$ 。

双版本以小模数的全部实际阶之最小公倍数对拍, 另检验大二幂、奇素数幂、32 位边界及公共指数证书。

UVa10006 完整程序对全部合法输入进行本地 Korselt 对拍; 在线 AC 待补。

来源: <https://oi-wiki.org/math/number-theory/primitive-root/>;

应用题: <https://onlinejudge.org/external/100/10006.pdf>。

### 93.22 Lucas: 按素数进制逐位计算

在特征为素数  $p$  的域中,  $(1+x)^p = 1+x^p$ 。若  $n = \sum_i n_i p^i$ , 则

$$(1+x)^n = \prod_i (1+x^{p^i})^{n_i}.$$

每个因子选择的次数在  $0, \dots, n_i$  之间, 严格小于  $p$ , 因此位数表示唯一。比较  $x^k$  的系数即得  $\binom{n}{k} = \prod_i \binom{n_i}{k_i} \pmod{p}$ ; 若某位  $k_i > n_i$ , 答案立即为零。仅预处理  $0!, \dots, (p-1)!$  及其逆元, 这些数均非零。 $p!$  为零, 不能把阶乘表延长后继续用除法公式。实现见第 117 页。

预处理后每位只做常数乘法, 适合固定小素数模数下的多次查询。接口接收  $n, k$ , 不接收 “ $n, m$  表示选  $n$  个” 的隐含约定; P3807 的驱动程序显式调用 `choose(n+m, n)`。题目范围内求和安全, 完整 64 位输入自行求和时仍要检查溢出。本地验证包含 Pascal 三角、精确大整数、小模数重建和最大无符号 64 位下标; P3807 双版本均已通过全部 25 个测试点; 动态版记录 297556209, 传统版记录 297556596。完整 64 位参数及额外边界以上述本地对拍验证。来源: <https://oi-wiki.org/math/number-theory/lucas/>。

### 93.23 合数模数组数与插值

对素数幂  $q = p^e$ , 记  $v_p(n!) = \sum_{j \geq 1} \lfloor n/p^j \rfloor$ ,  $U_p(n) = n!/p^{v_p(n!)}$ 。设  $t = v_p(n!) - v_p(k!) - v_p((n-k)!)$ 。当  $t \geq e$  时组合数模  $q$  为零; 否则

$$\binom{n}{k} \equiv U_p(n) U_p(k)^{-1} U_p(n-k)^{-1} p^t \pmod{q}.$$

这里仅对与  $p$  互素的单位部分求逆。各素数幂上的结果通过 CRT 合并, 对应第 119 页的实现。

在素数域中,  $n$  个模  $p$  两两不同的横坐标确定唯一的次数小于  $n$  的多项式。定义  $w_i = y_i / \prod_{j \neq i} (x_i - x_j)$ , 则

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j < i} (k - x_j) \prod_{j > i} (k - x_j).$$

该式不除以  $k - x_i$ , 查询点命中样本点时仍然成立。当  $x_i = i$  时, 分母为  $i!(n-1-i)!(-1)^{n-1-i}$ , 要求  $n \leq p$ 。实现见第 130 页。对整数幂前缀和  $S_d(k) = \sum_{i=1}^k i^d$ , 当  $0 \leq d \leq p-2$  时, 它在模  $p$  意义下可用次数至多  $d+1$  的多项式表示; 取  $0, 1, \dots, d+1$  的样本即可。不能只因某序列前几项能插值, 就断言后续仍由该多项式给出。

## 93.24 阶乘表：组合数与排列数

当素数模数为  $p$  且  $0 \leq k \leq n < p$  时，阶乘均非零，因此

$$C(n, k) = n! (k!)^{-1} ((n-k)!)^{-1}, \quad P(n, k) = n! ((n-k)!)^{-1} \pmod{p}.$$

将表从旧上限扩展到  $N$ ，先正向补齐阶乘，只对  $N!$  求一次逆元，再用  $(i-1)!^{-1} = i(i!)^{-1}$  反向补齐新逆阶乘项即可。不需要每一项分别快速幂求逆。较小上限的初始化请求保留已有表。查询前显式扩表，之后查询为常数时间；内部两表重分配时旧元素引用不能继续使用。实现见第 100 页。

WIDA 的“组合数 (Lucas)”文件实际提供的是普通阶乘、逆阶乘、组合数和排列数，没有逐位 Lucas 递推。本书以此节覆盖该文件的数学功能，用显式扩表取代查询时自动扩表，适用范围仍须限制在模数以内。验证使用独立 Pascal 三角和下降阶乘，另检查百万项表及扩表前后的原查询结果。

## 93.25 组合计数

容斥： $|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$ 。二项式反演：若  $b_n = \sum_{k=0}^n \binom{n}{k} a_k$ ，则  $a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} b_k$ 。

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}, \quad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以  $n+1$ ，在模数下尤其有用，但组合数本身仍须正确求模。第二类 Stirling 数满足

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad S(0, 0) = 1.$$

第一类无符号 Stirling 数把系数  $k$  换为  $n-1$ 。

| 球与盒     | 空盒允许                   | 空盒禁止               |
|---------|------------------------|--------------------|
| 不同球，不同盒 | $k^n$                  | $k!S(n, k)$        |
| 相同球，不同盒 | $\binom{n+k-1}{k-1}$   | $\binom{n-1}{k-1}$ |
| 不同球，相同盒 | $\sum_{j=0}^k S(n, j)$ | $S(n, k)$          |

上表默认  $k \geq 1$ ； $n=0, k=0$  等边界应按原问题约定单独处理。

## 93.26 群作用计数

有限群  $G$  作用在有限集合  $X$  上时，轨道数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

对位置置换的  $q$  色染色， $|\text{Fix}(g)| = q^{c(g)}$ ，其中  $c(g)$  为循环数。环形项链只商去旋转；手链还商去反射，二者群不同，不能共用分母。模数整除  $|G|$  时不能使用模逆元，须先在合适的整数模数下求和再做精确除法。

## 93.27 生成函数与递推

普通生成函数  $A(x) = \sum a_n x^n$  的乘法对应普通卷积。指数生成函数  $A(x) = \sum a_n x^n / n!$  的乘法对应  $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ 。形式幂级数求逆要求常数项可逆； $\log A$  取  $A(0) = 1$ ， $\exp A$  取  $A(0) = 0$ 。有限域中积分涉及  $1/i$ ，必须避免  $i$  被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \quad (\log A)' = A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用，必须另外证明序列确实具有不超过相应阶数的线性递推；少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

## 93.28 线性代数与生成树

域上的线性方程组一致时，解集为一个特解加零空间，零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值；模素数消元是精确域运算；合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数，非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树；重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向，不能直接复用无向封装。

## 93.29 模线性方程组：工厂工期

POJ2947 (同题 UVa1564) 给出每位工人制造的产品类型和起止星期。设第  $j$  种产品需  $x_j$  天，第  $i$  条记录中出现  $a_{ij}$  次，以星期一为 0、星期日为 6，则

$$\sum_j a_{ij} x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod{7}.$$

起止两天均计入；同一种产品重复出现必须累加系数。跨星期无需特判，模数类型负责归一化负数。每个余数在  $[3, 9]$  中恰有一个代表，因此模方程的解数就是原问题的解数。

直接使用第 146 页的模高斯消元，模数取 7。增广矩阵有  $m$  行、 $n+1$  列，最后一列是右端项。先判断 **consistent**：假时输出无解；为真且秩小于  $n$  时输出多解；否则读取特解，余数小于 3 的分量加 7。不能先以秩不足判断多解，因为零系数行仍可能有非零右端项。本题所有非零系数均可逆；合数模数不具备这一性质，不能直接套用此消元。

完整双版本程序位于 `verify/poj/2947.*.cpp`。验证包含所有起止星期组合、产品重复 7 次导致系数归零、矛盾的零行、独立枚举所有 3~9 天赋值，以及用可逆行变换构造的 300 阶稠密系统。该应用验证覆盖分类及唯一解输出，不代替零空间基等其他接口的验证。来源：<https://onlinejudge.org/external/15/1564.pdf>；kuangbin 2.10.2。

## 93.30 概率、期望与博弈

期望线性性不要求独立： $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ 。方差可加需要协方差为零；独立是充分条件。非负整数变量  $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$ 。吸收过程的期望方程  $E_i = 1 + \sum_j p_{ij} E_j$  需要检查是否几乎必然吸收及期望是否有限；不能对闭合非吸收分量直接求唯一有限解。有限无环公平组合游戏在正常玩法下

$$\text{SG}(u) = \text{mex}\{\text{SG}(v) : u \rightarrow v\}.$$

独立子游戏的 SG 值异或为零当且仅当先手必败。反常玩法、含环游戏、双方可走步不同的游戏不能直接套这个结论。

## 93.31 数值算法与几何

排序必须使用严格弱序；不能以“差小于  $\epsilon$  则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。对于简单格点多边形（无孔），Pick 定理  $A = I + B/2 - 1$ ，边界格点数  $B = \sum \gcd(|\Delta x|, |\Delta y|)$ ；不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

**来源索引** 本章为重新组织的公式速查，扩展范围依据 <https://oi-wiki.org/math/>。小模数 exLucas 与有限域拉格朗日插值已有双风格实现和本地验证；素数模平方根已有 Cipolla 实现；仍需补齐二次剩余的其他方法与定理、Min25、Newton 插值、子集卷积、行列式进阶、数值积分等内容及验证。本章公式不能代替这些待完成代码。

### 93.32 线性递推：系数方向与外推条件

约定  $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，初值是  $a_0, \dots, a_{k-1}$ 。对应特征多项式为

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \dots - c_{k-1}.$$

若  $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$ ，则  $a_n = \sum_{i=0}^{k-1} r_i a_i$ 。代码由高次向低次消去多项式项，第  $j$  个系数写入原次数减  $j$  的位置，避免倒置系数。例如 Fibonacci 初值为  $[0, 1]$ ，递推系数为  $[1, 1]$ 。

BM 在域上求有限前缀的最短递推。本书实现要求素数模数；一般的模合数环不能保证非零偏差存在逆元。仅凭若干样本无法证明真实序列以后一直遵循同一个规律。若事先知道序列满足阶数至多  $d$  的齐次线性递推，至少给出前  $2d$  项，再用恢复出的系数求后续项。系数向量末尾的零不能擅自删除：它可能对应非平稳的初始段，改变阶数会改变递推开始适用的位置。

### 93.33 素数模高次剩余与单位根

实现见第 123 页；依赖本书离散对数、线性同余和快速幂。设  $p$  为素数， $g$  是原根， $k > 0$ 。 $a = 0$  时根只有零；否则写  $a = g^b$ 、 $x = g^y$ ，方程转为

$$ky \equiv b \pmod{p-1}.$$

令  $d = \gcd(k, p-1)$ 。当  $d \nmid b$  时无解；否则指数解为  $y = y_0 + i(p-1)/d$ ， $0 \leq i < d$ 。因此用三个数存全部根：

$$\text{first} = g^{y_0}, \quad \text{ratio} = g^{(p-1)/d}, \quad \text{count} = d.$$

依次乘 ratio 即可枚举；不保证数值递增，也不保证 first 是最小根。ratio 是阶为  $d$  的单位根，因而枚举恰有  $d$  个不同的根。模  $p = 2$  同样有效；指数同余的模数是 1，唯一非零根为 1。

接口支持  $p \leq 10^{12}$ ，需要调用者提供已验证的素数与原根； $k$  支持完整 unsigned long long，先对  $p-1$  取模后解同余。 $a$  支持完整 signed long long，先正规化。返回空 optional 表示无解，返回  $\{0, 1, 1\}$  表示唯一零根。给定原根后的期望时间、空间为  $O(\sqrt{p})$ ；枚举全部根另外需要  $O(d)$  时间。模乘须先扩展到 128 位，不可用 64 位乘积直接取模。

测试穷举小于 100 的每个素数、全部原根及小指数，用任意精度模幂逐个枚举候选根并比较整个集合；覆盖负数、64 位边界、模 2、无解与零根。另检查 998244352 个根的紧凑表示，以及素数 999999999989 上的植根测试；后一个素数与原根的条件由试除及乘法阶证书独立确认。普通与 ASan/UBSan 均通过，尚无专门在线 AC。

这是 [OI Wiki 高次剩余](#) 中离散对数转线性同余方法的素数模实现。素数幂及非互素被开方数见下一节；一般合数模数见后文；改良 Tonelli-Shanks 尚需另补，不能把本节状态解读为该页面全部覆盖。

### 93.34 素数幂模高次剩余与根的提升

实现见第 124 页。令  $M = p^e$ 。当  $a = 0$  时，全部根是  $p^{\lceil e/k \rceil}$  的倍数。否则令  $t = v_p(a) < e$ ，必须有  $k \mid t$ ，否则无解。取  $s = t/k$ ， $q = p^{e-t}$ ，写  $x = p^s y$ ，只需求单位方程

$$y^k \equiv a/p^t \pmod{q}.$$

每个单位根  $y$  对应原模数下的根

$$x = p^s y + \ell p^s q, \quad 0 \leq \ell < p^{t-s}.$$

这些提升保持互不相同，也解释了非互素被开方数可能产生大量根。

奇素数时，提供模  $p^e$  的原根，约化后仍是模  $q$  的原根，沿用离散对数转线性同余的方法。 $q = 2$  时单位根只有 1； $q \geq 4$  且为二的幂时，单位表示为  $(-1)^z 5^y$ 。符号指数模 2，5 的指数模  $q/4$ ； $q = 4$  时只有符号分支。分别解两个指数同余即可，不可假设二的高次幂存在原根。

返回结构体保存 first、两个 ratio、两个 count，以及 step、lifts。全部根由下式给出：

$$(\text{first} \cdot \text{ratio}[0]^i \cdot \text{ratio}[1]^j \bmod \text{step}) + \ell \cdot \text{step}.$$

其中  $0 \leq i < \text{count}[0]$ 、 $0 \leq j < \text{count}[1]$ 、 $0 \leq \ell < \text{lifts}$ 。get 已处理模乘溢出，size 返回三者乘积；不保证根按数值排序。零根情形复用同一表示。

模数上限  $10^{12}$ ， $k$  支持完整无符号 64 位。只检查范围，不检查  $p$  的素性和奇素数原根的阶，调用者须保证。默认参数  $g = 0$  仅用于  $p = 2$ ；奇素数不能漏传原根。时间和空间由 BSGS 决定，期望  $O(\sqrt{q})$ ；单次按索引取根使用快速幂，需  $O(\log M)$ 。

已穷举多个素数的全部不超过 256 的素数幂、所有被开方数与小指数，用任意精度模幂直接枚举所有根，比较整个集合。另测有符号与无符号 64 位边界、 $2^{39}$  的非平凡离散对数、 $3^{20}$  的非互素方程，以及数万亿根的紧凑表示。普通与 ASan/UBSan 均通过；尚无专门在线 AC。一般合数的 CRT 组合见下一节；改良 Tonelli-Shanks 仍未完成，OI Wiki 页面继续标记部分覆盖。

### 93.35 一般合数模高次剩余：CRT 组合

实现见第 128 页。调用者提供完整分解  $M = \prod_{t=0}^{F-1} m_t$ ，其中  $m_t = p_t^{e_t}$  两两互素，并提供每个奇素数幂的原根。先分别求局部根族；任一局部无解，整体即无解。若局部根数为  $c_t$ ，则整体根数为  $\prod_t c_t$ 。只保存局部根族，不创建这个大小的数组。

对每个因子预处理

$$E_t = \frac{M}{m_t} \left( \left( \frac{M}{m_t} \right)^{-1} \bmod m_t \right) \bmod M.$$

对一组局部根  $r_t$ ，唯一对应的标准代表是

$$x = \sum_t r_t E_t \bmod M.$$

逆元存在是因为因子两两互素。`get(index)` 将索引按各个  $c_t$  转成混合进制数字，再把局部索引拆成两个乘法根族索引和加法提升索引，最后用上述公式组合。CRT 双射保证不重复、不遗漏，输出顺序不保证递增。

$M \leq 10^{12}$ ， $k > 0$ ；空因子列表表示  $M = 1$ ，返回唯一代表 0。`mod` 保存模数，`total` 保存总解数，`parts` 和 `weight` 保存局部根族与  $E_t$ 。模数乘积检查上界，CRT 的乘积先扩展至 128 位。单次按索引查询为  $O(F \log M)$ ；求解的主要时间和空间为局部 BSGS，因子列表接口不包含整数分解与原根搜索的成本；自动准备重载见下一节。

已对  $1 \leq M \leq 180$ 、所有被开方数与小指数直接枚举所有根，与两套代码的完整输出集合比较，并反转因子顺序重新检查。另覆盖有符号与无符号 64 位边界、混合素数幂上的非互素方程和大量零根，以及十一种不同素因子组成模数时的全部 1024 个平方根。普通与 ASan/UBSan 均通过；尚无专门在线 AC。至此已有已知分解及原根时的 BSGS 求根路线，更快的改良 Tonelli-Shanks 等内容仍未覆盖，OI Wiki 条目保留部分完成状态。

### 93.36 自动准备模数分解与原根

实现见第 127 页。`root_factors(M, rho)` 把  $M \leq 10^{12}$  分解为互异素数的幂，返回合数求根需要的  $(p, e, g)$  列表；模数 1 返回空列表。二的幂不需要原根，固定填零。奇素数幂  $m = p^e$  的乘法群是循环群，设  $\varphi = m/p \cdot (p-1)$ 。分解  $\varphi$  后，候选  $g$  为原根当且仅当

$$p \nmid g, \quad g^{\varphi/q} \not\equiv 1 \pmod{m} \quad \text{对每个不同素因子 } q \mid \varphi.$$

从 2 起枚举候选，返回最小原根；所有奇素数幂均有原根。这里检查的是模  $p^e$  的阶，不能仅取一个模  $p$  的原根而不验证提升。

重载 `CompositeRoots::solve(a, k, M, rho)` 自动调用此准备过程。

随机分解工作区类型为 `PollardRho`。如果需要在同一个模数下处理多个方程，先保存因子列表，调用原有列表接口，避免重复分解和原根搜索。准备成本包括  $M$  与各  $\varphi$  的因数分解，以及逐个候选的模幂检验；不能把这部分直接算作常数时间。

独立试除验证  $1 \dots 500$  的全部模数与多组接近  $10^{12}$  的边界值，并用任意精度模幂验证返回原根的阶。小模数进一步检查更小候选不能生成全部单位。直接传模数的两套求根接口，在小模数上与全部候选根的枚举结果比较。普通与 `ASan/UBSan` 均通过；尚无自动准备入口的在线 AC。

### 93.37 带权矩阵树定理与方向约定

实现见第 152 页。返回的是每棵树的边权乘积，再对所有树求和，并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择，自环不属于任何生成树；单点图的空树乘积为 1。零权和负权都按域上的运算处理，因此结果为零可能来自抵消，不能作为连通性判定。

无向边  $uv$  权值  $w$  对拉普拉斯矩阵贡献为  $L_{uu}, L_{vv}$  各加  $w$ ， $L_{uv}, L_{vu}$  各减  $w$ 。删除 root 行和列后求行列式，与 root 的选择无关。对于每个点都能沿有向边走到根的树形图，边  $u \rightarrow v$  贡献  $L_{uu} += w, L_{uv} -= w$ ；删除根行列后得到内向树形图的权值和。外向树形图先反转所有边，再使用同一构造。

例如只有一条边  $0 \rightarrow 1$  时，内向根 1 的答案为该边权，内向根 0 为零；外向根 0 的答案为该边权。P6178 指定的是以 1 为根的外向树，驱动把点号减一，并选择 `away_from_root`。只直接构造  $(n-1) \times (n-1)$  余子式，时间  $O(n^3 + m)$ 、额外空间  $O(n^2)$ 。行列式复用本书的素数模高斯消元，不支持将 mod 随意改成合数。

测试独立枚举  $n-1$  条边的所有选择，检查连通性或根向约束，用任意精度整数计算边权乘积之和，再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界，并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点：[动态版记录 297619886](#)、[传统版记录 297620263](#)。单点最大用时分别为 64 ms、62 ms，内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树；其他根、内向树和负权仍使用本地证据。定理背景可参考 [OI Wiki 矩阵树定理](#)。

## 93.38 任意正模数行列式

实现见第 151 页。素数模高斯消元可以除以任意非零主元，合数模数没有这个性质。本节只使用倍加行和交换行，不求逆元。先把元素正规化到  $[0, m)$ ，对第  $i$  列的两个元素  $a, b$ ，用整数商  $q = \lfloor a/b \rfloor$  执行  $R_i \leftarrow R_i - qR_j$ ，再交换  $R_i, R_j$ 。该列的新数对为  $(b, a \bmod b)$ ，所以循环按欧几里得算法终止。其余元素每步对  $m$  取模，只改变整数代表，不改变行列式模  $m$  的值。

倍加行不改变行列式；交换行改变符号。消去所有下三角元素后，乘上对角元素及累计符号即得结果。剩余部分的该列全零时直接返回零，不需要判断某个非零元素是否与模数互素。例如  $\text{diag}(2, 3)$  在模 8 下的行列式为 6，不能因为第一个主元没有逆元就判定失败。

支持任意正 signed 64 位模数与完整 signed 64 位输入元素。正规化后单个乘积小于  $2^{126}$ ，行更新差值和结果乘积都放入有符号 128 位。空矩阵按约定返回  $1 \bmod m$ ；模 1 的所有返回值为零。时间上界  $O(n^3 \log m)$ ，矩阵副本占  $O(n^2)$  空间。本节可以用于合数模的拉普拉斯余子式，但之前的 `MatrixTree<mod>` 接口仍使用素数模高斯，不能仅把其模板参数改成合数；合数模使用下一节的独立接口。

测试用排列定义穷举所有元素属于  $\{-1, 0, 1\}$  的三阶矩阵，并用任意精度 Bareiss 消元验证随机矩阵。覆盖不可逆主元、奇异矩阵、模 1、有符号 64 位边界、Fibonacci 长欧几里得过程，以及 300 阶对角矩阵和完全图余子式的 Cayley 公式。两套版本普通与 ASan/UBSan 均通过；尚无专门在线 AC。行变换依据可参考 [OI Wiki 行列式](#)。

### 93.39 任意正模数的矩阵树计数

实现见第 153 页。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用无向、内向与外向三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`。

先把边权正规化到  $[0, m)$ ，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回  $1 \bmod m$ ，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

## 93.40 斯特林数的快速整行计算

第二类实现见第 137 页，第一类实现见第 138 页。本节默认模数  $p = 998244353$ ，允许用模板参数指定素数与原根，复用参数化 NTT。第二类  $S(n, k)$  计数无标号非空集合划分；第一类  $c(n, k)$  计数排列的循环数。两者不能互换，第一类本节返回无符号版本。

第二类的容斥公式为

$$S(n, k) = \sum_{i=0}^k \frac{i^n (-1)^{k-i}}{i! (k-i)!}.$$

分别构造  $a_i = i^n/i!$  和  $b_i = (-1)^i/i!$ ，一次卷积取前  $m+1$  项即可。 $n$  支持无符号 64 位； $i > 0$  时指数对  $p-1$  取模后快速幂， $i = 0$  时单独区分  $n = 0$  和  $n > 0$ ，不可直接套用费马降幂。 $S(0, 0) = 1$ ，正  $n$  的  $S(n, 0) = 0$ ， $k > n$  时为零。令  $L$  为 NTT 长度上限，要求  $2m+1 \leq L$ ；默认模数下即  $m < 2^{22}$ 。

无符号第一类是升阶乘多项式的系数：

$$x(x+1)\cdots(x+n-1) = \sum_{k=0}^n c(n, k)x^k.$$

令  $F_m(x) = x(x+1)\cdots(x+m-1)$ ，倍增式为  $F_{2m}(x) = F_m(x)F_m(x+m)$ ；奇数长度再乘  $x+2m$ 。平移系数满足  $[x^i]F_m(x+m) = \sum_{j=i}^m f_j j! m^{j-i}/(i!(j-i)!)$ ，将  $f_j j!$  倒序后与  $m^j/j!$  卷积即可。每层只递归计算一次半长多项式，总时间  $O(n \log n)$ ，空间  $O(n)$ 。要求  $n+1 \leq L$ ，默认模数下即  $n < 2^{23}$ ； $n = 0$  返回常数多项式 1。如果需要下降阶乘对应的带符号数，将第  $k$  项乘  $(-1)^{n-k}$ 。

验证使用小规模集合划分、排列循环的直接枚举，以及 300 行的递推表；大指数第二类另用独立转移矩阵快速幂验证。规模检查包含二十万项第二类行与两万项第一类行，用幂函数的下降阶乘展开和升阶乘求值恒等式检查。普通与 ASan/UBSan 均通过。P5395 和 P5408 使用模 167772161，现已支持指定模板参数 167772161。两题均获两套码风在线 AC、各 10 个测试点全过：第二类动态版与 传统版，第一类动态版与 传统版。OJ 覆盖题面模数与行长，其他模数和第二类完整无符号指数仍由本地证据支持。该模数已测试二十万项第二类行与 262143 阶第一类行，普通与消毒器均通过。按列计算及 OI Wiki 的其他斯特林数应用仍待补，[对应条目](#)保留部分覆盖状态。

### 93.41 参数化 NTT 与模数适配

实现见第 132 页。传入素数  $p$  和原根  $g$ ；长度  $n$  必须为二次幂且整除  $p-1$ 。代码自动计算  $L = 2^{v_2(p-1)}$  作为长度上限，不代表分配这个长度的数组没有成本。两个非空数组的普通卷积需要补齐到不小于长度和减一的二次幂。模 998244353 的  $L = 2^{23}$ ，模 167772161 的  $L = 2^{25}$ ，两者均可用原根 3。不能只改模数而忽略原根和长度条件；接口只检查基本范围，不检查素性或原根的阶。

正变换按  $g$  的相应单位根计算，逆变换使用逆单位根并乘  $n^{-1}$ 。模数支持 `int` 范围内的素数，模整数的加减和乘积使用 64 位中间量。蝶形层以半长度递增，避免最后一层之后的有符号整数溢出。

独立参考用任意精度整数直接计算 DFT 和朴素卷积，同时验证正逆变换复原。已检查七组素数和原根，包括模 2、3、17、97，斯特林数的默认模数与洛谷题目模数，以及 2013265921、原根 31。这些测试中的素性及原根条件另用试除和乘法阶证书确认。大规模全一数组卷积逐项比较已知三角形系数，检查补齐长度的边界。两套实现普通与 ASan/UBSan 均通过。

## 93.42 Bostan–Mori 与快速递推单项

代码见第 227 页。对于  $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，令  $Q(x) = 1 - \sum_{j=1}^k c_{j-1} x^j$ ， $P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$ ，则  $[x^n]P/Q = a_n$ 。注意系数按前一项、前两项的顺序给出，初值从  $a_0$  开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)}.$$

分母的奇数次项抵消，因此只保留分子与  $n$  同奇偶的项，分母保留偶数项，再将  $n$  右移一位。最终  $n = 0$  时答案是  $P(0)/Q(0)$ 。实现不需要分子次数小于分母次数，空分子直接返回零；分母常数项必须非零。每轮系数长度不增加，初始最大长度  $d$  必须满足  $2d - 1$  不超过 NTT 容量。总时间  $O(d \log d \log(n + 1))$ ，空间  $O(d)$ ； $n = 0$  直接取常数项。

独立验证包括按定义做形式幂级数除法、友矩阵快速幂，以及  $(1 - x)^{-32000}$  的二项式系数闭式；覆盖完整无符号 64 位下标、常数分母、不等于 1 的非零常数项、空分子、不真分式、零递推系数和多种 NTT 素数。P4723 驱动读取有符号系数并归一化；两套版本均获在线 AC：动态版与 传统版 各 5 个测试点全部通过，最慢单点分别为 907 ms、905 ms。OJ 覆盖模 998244353、 $n \leq 10^9$ 、 $k \leq 32000$ ；其他模数、完整 64 位下标与通用有理函数接口的扩展范围由本地测试支持。OI Wiki 线性递推 中的连续区间取项与其他快速算法仍待补，不将单项算法视为整页覆盖。

### 93.43 集合变换与不相交子集卷积

代码见第 139 页与第 141 页。长度  $N = 2^m$  确定全集的位数，数组第  $s$  项对应集合  $s$ 。OR 变换计算  $\hat{a}_s = \sum_{t \subseteq s} a_t$ ，AND 变换计算所有超集的和；逆变换将逐位加法改成减法。XOR 变换的系数是  $(-1)^{\text{popcount}(s \& t)}$ ，蝶形将  $(u, v)$  变为  $(u + v, u - v)$ ，逆变换每层再除以 2。因此 OR、AND 可用于任意模数，XOR 需要 2 可逆；本实现限制为奇数模数，使用  $(\text{mod} + 1)/2$ ，不对合数模数使用费马小定理。三类卷积均是变换后逐项相乘，再逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \quad \text{要求两个集合不相交。}$$

普通 OR 卷积允许重叠，不能直接用于上式。将输入按集合大小分层， $A_{i,s} = [|s| = i]a_s$ ， $B$  同理。每层 OR 变换后，对层数做卷积： $H_{k,s} = \sum_{i=0}^k \hat{A}_{i,s} \hat{B}_{k-i,s}$ 。对每个  $k$  逆 OR 变换，得到并集恰为  $s$ 、两集合大小之和为  $k$  的贡献。取  $k = |s|$ ，由  $|u| + |v| = |u \cup v| + |u \cap v|$  得到交集为空。时间  $O(m^2 2^m)$ ，空间  $O(m 2^m)$ ；本实现逐层复用结果缓冲区，不用第三组二维数组。 $m = 20$  时两组分层数组本身占 168 MiB，另计输入和线性缓冲。

验证独立枚举变换系数、所有下标对与子掩码，并检查复合模数与 `INT_MAX` 模数。大规模使用  $a_s = x^{|s|}$ 、 $b_s = y^{|s|}$  的逐位乘积恒等式：OR 为  $(x + y + xy)^{|s|}$ ，AND 为  $(xy)^{|s|}(1 + x + y)^{m - |s|}$ ，XOR 为  $(x + y)^{|s|}(1 + xy)^{m - |s|}$ ，不相交子集卷积为  $(x + y)^{|s|}$ 。P4717 两套版本各 10 点在线 AC：动态版和 传统版，最慢单点 84 ms、81 ms。

P6097 两套版本各 5 点在线 AC：动态版和 传统版，最大测试点显示 2.70 s、2.69 s，内存 188.83 MB、189.00 MB。两题 OJ 仅验证各自题面模数和长度范围，其他模数及长度 1 仍由本地独立测试覆盖。OI Wiki FWT 的其他运算及更广泛应用仍待覆盖。

## 93.44 Taylor 平移与等比点求值

代码见第 134 页和第 135 页。多项式使用升幂系数，输入长度  $n$  表示次数小于  $n$ ，允许末尾零。平移输出的第  $i$  项满足

$$[x^i]f(x+c) = \frac{1}{i!} \sum_{j=i}^{n-1} f_j j! \frac{c^{j-i}}{(j-i)!}.$$

倒序  $f_j j!$ ，与  $c^j/j!$  卷积，再取第  $n-1-i$  项并乘  $1/i!$ 。保留原数组长度；零多项式可以用空数组表示，返回空数组。使用 NTT 素数及原根，要求  $2n-1$  不超过变换容量；此时所需阶乘非零。时间  $O(n \log n)$ 、空间  $O(n)$ 。这与给出  $f(0), f(1), \dots$  后平移点值的任务不同，后者尚待补充。

Chirp Z 返回  $f(ar^i)$ ， $0 \leq i < m$ 。设  $T_j = j(j-1)/2$ ，由  $ij = T_{i+j} - T_i - T_j$  得

$$f(ar^i) = r^{-T_i} \sum_{j=0}^{n-1} f_j a^j r^{-T_j} r^{T_{i+j}}.$$

将第一项数组倒序，与  $r^{T_j}$  卷积，抽取第  $n-1+i$  项。三角数幂通过累乘生成，不计算可能溢出的  $j(j-1)$ 。令  $L$  为不小于  $n+m-1$  的最小二次幂，直接做模  $x^L-1$  的循环卷积。普通乘积最高次数为  $2n+m-3$ ，而所需区间任意下标  $t \geq n-1$  满足  $t+L \geq 2n+m-2$ ，因此不会收到高次项回卷，所取系数仍然精确。要求  $n+m-1$  不超过 NTT 容量，总时间  $O((n+m) \log(n+m))$ 、空间  $O(n+m)$ 。

约定  $r^0 = 1$ ，故  $r=0$  时第一项为  $f(a)$ ，余项均为  $f(0)$ ； $a=0$  时所有点为零， $r=1$  时所有点相同。这些情况单独计算，避免对零求逆。 $m=0$  返回空数组，空输入返回  $m$  个零。同一份算法支持小阶公比、重复点与有限域中多次绕回，输出保持原顺序。

独立验证采用 Pascal 二项式展开和 Horner 求值，涵盖 7 种 NTT 素数、零公比与单位公比、小素数完整容量边界。大规模平移用  $(x+b)^d$  检查每个系数；等比点求值用几何级数闭式逐点检查。普通与 ASan/UBSan 均通过。Library Checker 的 [Polynomial Taylor Shift](#) 和 [等比点求值](#) 已准备 vector 完整驱动；本地检查各题 524288 的最大规模，尚无在线 AC。参考OI Wiki [平移](#)及 [Chirp Z](#)，更广泛的点值平移和应用另行覆盖。

## 93.45 可持久化区间拼接的懒标记

代码见第 167 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足  $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$ 。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为  $\text{offset} + \text{source.lazy} - \text{destination.lazy}$ 。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为  $O(\log n)$  新节点，查询为  $O(\log n)$  时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 INT\_MAX。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 INT\_MAX 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的应用已获 QOJ8240 双 AC，其余通用接口范围仍由本地验证覆盖。来源需求是[2023 杭州站题解](#) 第 24–26 页的 K 题，完整驱动及验证见下节。

## 93.46 杭州 2023 K：跨版本拼接应用

[Card Game](#) 的完整驱动在 `verify/qoj/8240/*.cpp`，使用前节的可持久化区间模板。将下标转为从 0 开始，设  $F_i(r)$  为区间  $[i, r]$  的剩余牌数， $p$  为  $i$  之后第一张与  $a_i$  相同的牌，若不存在则  $p = n$ 。当  $r < p$  时，第一张牌始终保留，剩余牌的过程与从  $i + 1$  开始相同；当  $r \geq p$  时，到达  $p$  会将此前区间全部清空。因此

$$F_i(r) = \begin{cases} 1 + F_{i+1}(r), & r < p, \\ F_{p+1}(r), & r \geq p. \end{cases}$$

每个版本的坐标是右端点  $r$ ，值是  $F_i(r)$ 。从  $i = n - 1$  向前处理：复制版本  $i + 1$  并在  $[i, p)$  加 1，再把  $[p, n)$  拼接自版本  $p + 1$ 。 $p = n$  时省去后一步。询问原问题的  $[l, r]$  时，从版本  $l - 1$  读取坐标  $r - 1$  的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n, q \leq 300000$ ，牌号位于  $[1, n]$ 。构建及查询时间分别为  $O(n \log n)$ 、 $O(q \log n)$ 。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为  $O(n \log n)$ 。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、 $n = q = 300000$  的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。两套驱动均在线 AC，各 41 个正式测试和 3 个额外测试全过：[动态版](#) 最慢 789 ms、452688 KB，[传统版](#) 最慢 752 ms、452492 KB。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

## 93.47 非负权树的点集直径摘要

代码见第 66 页。一个非空点集只需保存任意一对直径端点  $a, b$ ，即可回答任意树上点  $x$  到该点集的最远距离： $\max(d(x, a), d(x, b))$ 。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点  $u$  投影到路径  $a-b$ ，记投影距  $a$  为  $t$ ，离路径的距离为  $h$ ，直径长为  $D$ 。由于  $d(a, u), d(b, u) \leq D$ ，有  $h \leq \min(t, D - t)$ 。再将查询点投影到同一路径，投影距  $a$  为  $s$ ，离路径的距离为  $g$ 。经两个投影点连接得到  $d(x, u) \leq g + h + |s - t| \leq g + \max(s, D - s)$ ，右侧恰为  $\max(d(x, a), d(x, b))$ 。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 `distance` 接口仍返回边数。带权距离请显式调用 `weighted_distance / Weighted_Distance`；采用  $(D_u - D_z) + (D_v - D_z)$  避免不必要的两倍根距离中间值。所有路径权和须能用 `long long` 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权  $10^9$  的递归链也通过普通与 ASan/UBSan 检查。

杭州 2023 F 中，点权互异。令  $g$  为全树点权的 mex，只需按点权  $0, 1, \dots, g - 1$  建立前缀摘要。某个前缀全部位于以  $x$  为中心、半径  $k$  的范围内，当且仅当该摘要的最远距离不超过  $k$ 。二分最长满足条件的前缀长度即为答案； $g = 0$  时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 mex 集合参照， $n = q = 500000$  的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

## 93.48 函数图的人环链与到达查询

代码见第 4 页。每个点有且仅有一个后继，故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点，剩下的点恰在环上；沿后继枚举各环。反向处理移除序列时，后继的环编号、入环点和入环距离均已确定，可直接传给当前点。这里使用的是入度剥离队列，不是用显式栈模拟 DFS。

跳转  $k$  步时，若  $k$  小于入环距离，用普通倍增；否则先到入环点，减去入环步数，再对环长取模。因此即使  $k$  达到无符号 64 位上界，也只需  $O(\log n)$  层倍增表。环上位置相加使用无符号宽整数，避免两个大下标的 `int` 中间值溢出。

判断从  $u$  到  $v$  的最少步数时：不同环编号必不可达。若  $v$  在入环树上，先检查  $u$  的深度至少为  $v$  的深度，再跳过深度差，看是否恰好到  $v$ ，从而排除同分量的不同分支。若  $v$  在环上，答案为  $u$  的入环步数加上沿后继方向的环上距离。不可达返回  $-1$ ，从自己到自己为零步。字段 `position` 只对环点有意义，不要将树点的  $-1$  当作环坐标。

独立参照逐步行走直至首次重复，同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射，并检查跳转、最少步数、入环点与环长；另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应 [CSES 1750](#)、[CSES 1160](#) 和 [P2921](#)，均准备两套码风。前两题检查跳转与可达步数，后一题仅使用“入环步数 + 环长”，不能替代查询接口的验证。线上评测尚待完成。

## 93.49 杭州 2023 H：随机事件顺序与函数图

[Sugar Sweet II](#) 的完整驱动为 `verify/qoj/8237/*.cpp`，复用函数图分解与逆阶乘表。事件  $u$  发生前，自己的数量仍为  $a_u$ ；目标  $v = b_u$  只可能为  $a_v$  或  $a_v + w_v$ 。因此  $a_u < a_v$  时必定发生， $a_u \geq a_v + w_v$  时必定不发生。其余情况下，事件  $u$  成功当且仅当事件  $v$  更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为  $d$ ，沿途  $d + 1$  个事件必须按唯一的顺序出现。均匀随机全排列对这  $d + 1$  个事件诱导均匀相对顺序，因此成功概率为  $1/(d + 1)!$ 。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为  $a_u + w_u/(d + 1)!$ ，在模  $10^9 + 7$  下计算。

题面所有增量为正， $\sum n \leq 500000$ ；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为  $O(n \log n)$ ；概率传播本身只读取 `entry` 和 `depth`，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：[动态版](#) 162 ms、84608 KB，[传统版](#) 158 ms、84612 KB。本应用不能验证函数图的 `advance` 或 `steps` 接口。

## 93.50 杭州 2023 G：开放时间的单位边最短路

驱动 `verify/qoj/8236*.cpp` 直接枚举网格邻居，不建立四倍规模的边表。初始蛇身从头到尾编号  $0, \dots, k-1$ ，第  $i \geq 1$  节所在点的最早进入时间为  $k-i$ ，头与其他空格为零。旧蛇尾允许在同一步进入，所以尾格的限制必须是 1。长度为 4 的满  $2 \times 2$  网格样例距离为 0, 3, 2, 1，平方和为 14。

每步只能删除或移动一个尾节，故这些时间限制必要。最短路可以取不重复顶点的路径；行走前把途中等待的总时间全部用于缩短蛇身，随后沿路径连续移动。进入初始蛇身第  $i$  节时，已删除的尾节数量至少为  $k-i$ ；新形成的蛇身不会与无重复顶点的路径相撞。因此这些限制也充分。等待超过  $k-1$  没有必要，因为此时全部初始蛇身均已释放。

转移为  $d_v = \max(d_u + 1, \text{open}_v)$ 。按距离递增处理顶点时，第一个发现  $v$  的前驱就给出最小候选值，故不需要后续松弛。候选等于  $d_u + 1$  的顶点进入普通 FIFO；其余进入对应开放时间的桶。每次取两者中距离较小的一项。桶指针只能扫描到下一个 FIFO 距离，否则可能越过之后才被发现较早开放时间。令  $R = \max \text{open}$ ，复杂度  $O(V + E + R)$ ，空间  $O(V + R)$ 。此性质依赖所有边权为 1；一般非负权仍需优先队列。

本地以独立堆 Dijkstra 对拍，并穷举  $2 \times 3$  网格障碍和全部 744 种初始蛇形，对真实蛇身序列逐步执行移动与缩尾 BFS，核对每个点的最早到达时间。另验证正式样例、满长链和  $3000 \times 3000$  空网格及隔断网格；普通与 ASan/UBSan 均通过。答案用 unsigned long long 累加距离平方，自然实现模  $2^{64}$ 。两套驱动在 [QOJ8236](#) 各通过 58 个正式与 3 个额外测试：[动态版](#) 324 ms、123080 KB，[传统版](#) 294 ms、122768 KB。线上证据对应单位网格、源点时间为零及初始蛇身开放时间；一般有向图与非零源点时间由本地对拍覆盖。

## 93.51 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间  $t$  表示前  $t$  次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针  $(l, r, t)$ ，回调统计恰好对应当前版本的半开区间  $[l, r)$ 。前进修改  $i$  时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为  $B$ ，询问数为  $Q$ ，修改数为  $U$ ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有  $O(Q \log Q)$  排序与  $O(Q)$  调度空间。三者同阶时取  $B \approx n^{2/3}$ ，得到  $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

[P1903 数颜色](#) 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 [P1903 记录 297653743](#) 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

## 93.52 SCC 模板题与速度榜口径

[QOJ906](#) 来自 Library Checker，要求输出强连通分量的拓扑序。本库 Tarjan 编号按逆拓扑序产生，驱动将输入点号加一，再按分量编号从大到小输出，输出点号减一。[CCF\\_NOI 的记录 2940638](#) 通过全部 11 个测试，203 ms、58636 KB。验证覆盖 add、run、bel 和完整分量输出，不调用 dag。本地独立可达性参考核对分量等价关系及每条跨分量边方向，并测试 50 万点链、环和多分量。普通与 ASan/UBSan 均通过；本地测试线程使用 1 GiB 栈容纳 sanitizer 扩大的递归栈帧，提交代码保持普通 main 和递归 DFS。

2026-09-12 核对速度榜时，页面第 66 行处于 203 ms 的第 65–67 行并列区间。但该表合并同一用户的提交，不能将行号与满分提交数 578 配对。以 CCF\_NOI 逐页读取页面链接的满分列表，共取得 143 条，其中 93 条严格更快、3 条同为 203 ms。可见范围内按耗时并列排名为 94 out of 143，位置区间为 94–96。统计页显示所有提交 1053、满分提交 578；与可见列表的计数差异原因尚未核实，因此全站排名仍为待查。记录清单与证据见 docs/ranking-audits.json，不得将该局部排名改写为全站排名。

## 93.53 并查集模板题的接口范围

[P3367](#) 输入点号为 1-based，驱动统一减一后调用 dsu。[记录 297668102](#) 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 merge、same 及其调用的 find，不验证 size、groups；后两者已由独立标签参考和 50 万点长链测试核对。

## 93.54 ACL 的组合约定

固定参考 AtCoder Library 提交 864245a00b00，公开模块分为数据结构、图论、数学与字符串。学习短小的运算接口及明确的组合契约，不直接复制全部工程支持层。尤其注意：懒标记 composition(f,g) 表示先作用 g、再作用 f；SCC 分量方向与 2-SAT 的真假编码不能分别替换。本库已补齐 ModInt 的四种复合赋值，Fenwick 的模数区间和组合通过独立整数余数参考及 ASan/UBSan。新增运算只有本地验证，不能套用历史 OJ AC；取模和仍不适用于 kth。完整 12 模块、6 内部头文件的对照及缺项清单见 docs/ACL-REVIEW.md。

## 第九十四章 计算几何：精确判定与适用范围

### 94.1 整数系数的有界半平面交

实现见第 188 页。输入是不等式  $ax + by \leq c$ ，三个系数均为绝对值不超过  $10^9$  的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标  $(X, Y, D)$  表示：

$$D = a_1b_2 - b_1a_2, \quad X = c_1b_2 - b_1c_2, \quad Y = a_1c_2 - c_1a_2.$$

若  $D < 0$ ，三者同时取负，使  $D > 0$ 。测试  $aX + bY \leq cD$  即可判断交点位于半平面内，无需除法。系数界保证  $|X|, |Y|, D \leq 2 \times 10^{18}$ ，可用 64 位保存；相等判断的交叉乘积至多为  $4 \times 10^{36}$ ，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。两套版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。将有理数坐标转换为浮点、计算面积或处理浮点输入属于另一个数值问题，尚未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

## 94.2 最近点对、Minkowski 和与三维整数判定

最近点对与 Minkowski 和见第 199 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间  $O(n \log n)$ ，额外空间  $O(n)$ 。坐标绝对值上限为  $10^{12}$ ，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在  $[-10^{12}, 10^{12}]$  内。非退化凸包按边方向归并，时间  $O(n + m)$ ；退化分支枚举点和后求凸包，时间  $O((n + m) \log(n + m))$ 。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 202 页。点的三个坐标为  $x, y, z$ ，绝对值均不超过  $10^9$ 。`orient(a, b, c, d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多  $2 \times 10^9$ ，叉积分量至多  $8 \times 10^{18}$ ，三重积绝对值至多  $4.8 \times 10^{28}$ ，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按  $4 \times 4$  行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

## 94.3 浮点最近点对

实现见第 197 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过  $10^{100}$ 。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间  $O(n \log n)$ ，额外空间  $O(n)$ 。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 `epsilon`；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、 $10^{-100}$  至  $10^{100}$  的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：[动态版记录 297602650](#)、[传统版记录 297603238](#)。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

## 94.4 公切线与过点切线

实现见第 195 页，依赖浮点几何的点、圆和向量运算。令两圆心差为  $v$ ,  $d = |v|$ ,  $u = v/d$ 。外公切线取  $s = 1$ , 内公切线取  $s = -1$ , 单位法向量满足

$$n \cdot u = \frac{r_1 - sr_2}{d} = c, \quad n = cu \pm \sqrt{(1-c)(1+c)} u^\perp.$$

切点为  $p = o_1 + r_1 n$ 、 $q = o_2 + sr_2 n$ 。直线以  $n \cdot (x - p) = 0$  表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 **side** 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一次。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，**side**=0。严格同心且等半径返回 **infinite=true**，包括重合的两个点圆；严格同心但半径不同没有公切线。**from\_point(p,c)** 把第一圆设为点圆，圆上的切点读返回的 **b**。点在圆上时返回一条，点在圆内时空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过  $10^{100}$ ，半径非负。容差  $0 < \varepsilon < 1$  默认  $10^{-12}$ ：当  $|r_1 - sr_2|$  与  $d$  的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算  $n \cdot (x - p)$ ，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

## 94.5 随机增量最小覆盖圆

实现见第 193 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为  $O(n)$ ，最坏时间仍为  $O(n^3)$ 。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。`through` 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。[P1742](#) 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过  $10^4$ ，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 P1742 通过全部 10 个测试点：[动态版记录 297608403](#)、[传统版记录 297608801](#)。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 [Project Nayuki 的算法说明](#)。

## 94.6 圆与简单多边形相交面积

实现见第 191 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是  $\text{cross}(p, q)/2$ ；否则贡献是有向扇形面积

$$\frac{r^2}{2} \text{atan2}(\text{cross}(p, q), p \cdot q).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间  $O(n)$ 、额外空间  $O(1)$ 。

求交点时令边的单位方向为  $u$ ，有向距离  $h = \text{cross}(u, a)$ ，垂足直接构造为  $hu^\perp$ ，交点为

$$hu^\perp \pm u\sqrt{(r - |h|)(r + |h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

# 第九十五章 来源与覆盖维护

## 95.1 参考基线

- kuangbin, ACM 常用算法模板, 2018 年版本: <https://kuangbin.github.io/2018/08/01/ACM-template/>。
- WIDA XCPC: <https://github.com/hh2048/XCPC>, 参考快照 484e9e0。
- OI Wiki 数学: <https://oi-wiki.org/math/>。

本项目重新实现的代码与公式速查不能冒用原作者的验证记录。完整覆盖审计应为每个来源条目记录实现文件、替代关系与验证证据, 保留遗漏项直至完成。

## 95.2 后续覆盖项

尚需完成的范围以覆盖表为准, 包括动态树、平衡树、一般图匹配、最小树形图、上下界流、高阶几何构造、半平面交、三维凸包、数值积分、复杂数论及多项式、搜索与动态规划等。不能用通用模块的存在推断其覆盖了所有原书例题。

# 算法索引

AffineSegTree, 29  
AhoCorasick, 82  
Arborescence, 179  
  
basis\_intersection, 27  
basis\_sum\_intersection, 28  
batch\_inverse, 104  
Berlekamp-Massey, 239  
berlekamp\_massey, 225  
Bezout identity, 235  
Biconnected, 7  
Binomial, 100  
BipartiteMatching, 40  
Blossom, 222  
BostanMori, 227  
BoundedCirculation, 35  
bridge\_augmentation, 15  
Burnside, 239  
  
Carmichael, 238  
carmichael, 115  
Catalan, 239  
CentroidPairs, 72  
chirp\_z, 135  
Cipolla, 237  
CirclePolygon, 191  
CircleTangents, 195  
ClosestPair, 197  
CompositeRoots, 128  
coprime pairs, 232  
CoprimePairs, 110  
CRT, 234  
cut\_tree\_values, 185  
  
determinant\_mod, 151  
Dijkstra, 37  
Dinic, 31  
DirectedEuler, 49  
Dirichlet convolution, 230  
DiscreteLog, 150  
divisor sum, 233  
divisor\_sum\_power, 113  
dsu, 16  
Du Jiao sieve, 230

DuJiao, [149](#)  
DynamicKth, [173](#)

EdgeCompression, [13](#)  
EnclosingCircle, [193](#)  
Euler theorem, [234](#)  
Euler totient, [231](#)  
euler\_phi, [114](#)  
EulerLCA, [63](#)  
exCRT, [236](#)  
ExLucas, [119](#)  
exLucas, [238](#)  
expectation, [240](#)

Fenwick, [18](#)  
floor moments, [234](#)  
floor sum, [233](#)  
floor\_moments, [111](#)  
FunctionalGraph, [4](#)

Garner, [237](#)  
garner, [106](#)  
GcdSequenceTreap, [218](#)  
generating functions, [239](#)  
geometric series, [233](#)  
GeometryExtra, [199](#)  
gomory\_hu, [184](#)

HLD, [59](#)  
Hungarian, [178](#)

inclusion-exclusion, [239](#)  
integer partitions, [233](#)  
IntegerGeometry, [154](#)  
IntegerGeometry3D, [202](#)  
IntegerHalfplanes, [188](#)  
inverse\_table, [105](#)

kmp\_match, [80](#)  
Korselt criterion, [238](#)  
KthResidue, [123](#)

Lagrange, [130](#)  
Lagrange interpolation, [238](#)  
LazySeg, [19](#)  
LexTwoSAT, [47](#)  
LiChao, [160](#)  
LiftingLCA, [68](#)  
linear congruence, [236](#)  
linear sieve, [231](#)

linear\_congruence, 102  
linear\_equation, 101  
LinearAlgebra, 146  
LinearSieve, 98  
LinkCutTree, 205  
Lowlink, 43  
Lucas, 117, 238  
Lucas theorem, 234  
  
manacher, 81  
Matrix Tree theorem, 240  
MatrixTree, 152  
MatrixTreeMod, 153  
maximum\_closure, 36  
MaxPlusMatrix, 160  
Miller–Rabin, 230  
MinCostFlow, 33  
minimum\_rotation, 82  
mixed\_euler\_orientation, 54  
mixed\_euler\_trail, 56  
Mobius inversion, 230  
mod\_sqrt, 121  
ModifiedMo, 23  
ModInt, 99  
multiplicative order, 237  
  
NttConvolution, 132  
NumberTheory, 94  
numerical stability, 240  
  
odd\_cycle\_vertices, 10  
OfflineLCA, 61  
OrderedSplay, 214  
OrderedTreap, 208  
orient\_edges, 14  
  
PalindromicTree, 92  
Partitions, 116  
path\_intersection, 71  
pentagonal number theorem, 233  
PersistentArray, 164  
PersistentDistinct, 176  
PersistentKth, 162  
PersistentRange, 167  
Pick theorem, 240  
Pollard–Rho, 230  
PollardRho, 97  
Polya, 239  
Polynomial, 142

[polynomial\\_shift](#), 134  
[PositionBasis](#), 25  
[power\\_sum](#), 112  
[prefix\\_function](#), 80  
[prefix\\_lcs](#), 90  
[PrimePowerRoots](#), 124  
[PrimitiveRoot](#), 107  
  
[quadratic residue](#), 237  
  
[rank](#), 240  
[RealGeometry](#), 156  
[recurrence\\_nth](#), 226  
[release\\_bfs](#), 46  
[removal\\_components](#), 12  
[RollbackDSU](#), 17  
[root\\_factors](#), 127  
  
[SCC](#), 39  
[segmented sieve](#), 232  
[segmented\\_primes](#), 103  
[SequenceTreap](#), 210  
[SetConvolution](#), 139  
[Sprague-Grundy](#), 240  
[square\\_counts](#), 91  
[Stirling numbers](#), 239  
[stirling\\_first\\_row](#), 138  
[stirling\\_second\\_row](#), 137  
[StoerWagner](#), 182  
[subset\\_convolution](#), 141  
[SubtreeColors](#), 75  
[SuffixArray](#), 85  
[SuffixAutomaton](#), 87  
[SuffixLCP](#), 89  
[SupportHull](#), 203  
  
[TarjanSCC](#), 2  
[TreeDiameter](#), 66  
[TreePathKth](#), 170  
[TwoSAT](#), 40  
  
[UndirectedEuler](#), 52  
  
[VirtualTree](#), 77  
  
[WeightedMatching](#), 186  
[Widget Factory](#), 240  
[word\\_chain](#), 51  
  
[XorBasis](#), 20  
[XorWalk](#), 57

[z\\_function, 81](#)

[二元一次不定方程, 235](#)

[区间筛素数, 232](#)

[原根, 237](#)

[带修莫队, 259](#)

[开放时间最短路, 259](#)

[扩展中国剩余定理, 236](#)

[批量逆元, 235](#)

[排列数, 239](#)

[模高斯消元, 240](#)

[混合进制, 237](#)

[矩形 GCD 计数, 232](#)

[类欧几里德, 233](#)

[线性同余方程, 236](#)

[线性逆元表, 235](#)

[线性递推, 241](#)

[贪吃蛇, 259](#)

[阶乘表, 239](#)